

JavaScript Notes

1. Installation

What You Need

Tool	Purpose	Download
Node.js	Runs JavaScript on your computer	https://nodejs.org
VS Code	Code editor for writing JavaScript	https://code.visualstudio.com

What is Node.js?

- JavaScript was originally only used inside a browser (like Chrome, Firefox)
- Node.js allows you to run JavaScript directly on your computer, without a browser
- This is why we install Node.js before writing JavaScript

Quick Setup Check

After installing, open a terminal and type:

```
node -v    # Shows version like v20.x.x
npm -v     # Shows version like 10.x.x
```

If you see a version number, the installation was successful.

Real-life analogy: Node.js is like the engine of a car. VS Code is the dashboard. Without the engine, the car won't move. Without Node.js, JavaScript won't run on your computer.

2. Creating a Project Folder and JavaScript File

Steps

1. Create a new folder on your computer — e.g., `MyFirstProject`
2. Open VS Code → File → Open Folder → Select your folder
3. In the Explorer panel on the left, click the New File icon
4. Name the file `hello.js`
 - `.js` is the file extension for all JavaScript files

Example Folder Structure

MyFirstProject/

```
|
└─ hello.js
```

Real-life analogy: A project folder is like a drawer in your office. Each file inside is a document in that drawer. Keeping everything organized makes your work easier.

3. How to Execute (Run) JavaScript

There are two easy ways to run a JavaScript file.

Method 1: Using VS Code Extension — Code Runner

1. Open the Extensions panel → press `Ctrl+Shift+X`
2. Search for "Code Runner" → Click Install
3. Open your `hello.js` file
4. Press `Ctrl+Alt+N` or click the ▶ Run button at the top right
5. Output appears in the Output panel at the bottom

Method 2: Using the Terminal

1. Open the terminal in VS Code: Terminal → New Terminal
2. Type the following command and press Enter:

```
node hello.js
```

Real-life analogy: Running code is like pressing the Start button on a washing machine. You've loaded clothes (written code), selected a program (saved the file), now press Start (run the command).

4. Printing Statements Using `console.log()`

What is `console.log()`?

- It is used to print or display output in the terminal
- It is the very first thing every JavaScript programmer learns
- It helps you see what your code is doing, step by step

Syntax

```
console.log("Your message here");
```

Examples

```
console.log("Hello, World!");
```

```
console.log("My name is Alice");
```

```
console.log("I am learning JavaScript");
```

```
console.log(100);
```

```
console.log(10 + 20); // Output: 30
```

Output in Terminal

```
Hello, World!
```

```
My name is Alice
```

```
I am learning JavaScript
```

```
100
```

```
30
```

5. Different Ways to Run Code

Way 1 — Run Button (Code Runner Extension)

- Click the ▶ Play button at the top-right corner in VS Code
- Output appears in the Output panel at the bottom of VS Code

Way 2 — Terminal

node hello.js

- Output appears directly in the terminal window

Which one to use?

Situation	Recommended Way
Quick test or small file	Run Button (Code Runner)
Proper project file	Terminal (node filename.js)

6. Comments in JavaScript

What Are Comments?

- Comments are lines in your code that JavaScript does NOT execute (run)
- They are written for humans to read, not for the computer
- JavaScript completely skips and ignores comments

Why Use Comments?

1. To explain what a piece of code does
2. To remind yourself of your logic when you come back later
3. To help teammates understand your code
4. To disable a line temporarily without deleting it — useful while fixing bugs

What Happens If You DON'T Use Comments?

- The code still runs — comments don't affect how code works
- But the code becomes very hard to understand
- After a few weeks, even you may forget what a block of code does
- Other people in your team won't know what the code is trying to do
- It becomes harder to find and fix mistakes

Real-life analogy: Imagine a recipe with only a list of ingredients and no steps or explanation. You wouldn't know what to cook or in what order. Comments are the instructions that explain your code.

7. Types of Comments

Type 1 — Single-Line Comment (//)

- Use two forward slashes //
- Everything written after // on that line is a comment
- Used for short, one-line explanations

// This is my first JavaScript program

console.log("Hello, World!");

console.log("My name is Alice"); // This line prints my name

```
// Printing numbers
```

```
console.log(100);
```

```
console.log(200);
```

Output:

Hello, World!

My name is Alice

100

200

Notice: The comment lines (`//`) do not appear in the output at all.

Type 2 — Multi-Line Comment (`/* ... */`)

- Starts with `/*` and ends with `*/`
- Everything written between them is a comment
- Used when your explanation is long or needs multiple lines

```
/*
```

```
Program: My First JavaScript Program
```

```
Author: Alice
```

```
Date: April 28, 2026
```

```
Description: This program prints a welcome message.
```

```
*/
```

```
console.log("Welcome to JavaScript!");
```

```
/*
```

```
The lines below print  
some numbers to the terminal.
```

```
These are just for practice.
```

```
*/
```

```
console.log(10);
```

```
console.log(20);
```

```
console.log(30);
```

Output:

Welcome to JavaScript!

10

20

30

Single-Line vs Multi-Line — When to Use?

Use	Comment Type
Short explanation next to a line	Single-line <code>//</code>
Explaining a whole block of code	Multi-line <code>/* */</code>
Writing author name, date, description at top of file	Multi-line <code>/* */</code>

Program 1

```
// =====
// 📖 JavaScript Printing Statements
// =====

// console.log() is used to print output in the console

console.log("Your message here"); // Prints a custom message
console.log("Hello World!");      // Basic output
console.log("My name is Rahul");   // String output
console.log("I am learning JavaScript"); // Informational message

console.log(10);                  // Prints a number
console.log(10 + 20);             // Prints result of expression → 30
```

```
// =====
// 📖 Comments in JavaScript
// =====
```

```
// Why do we use comments?
// 1. To explain what a piece of code does
// 2. To make code easy to understand for others
// 3. To improve readability and maintainability
```

```
// 💎 Types of Comments
```

```
// 1. Single-line comment
// Starts with two forward slashes (//)
```

```
// 2. Multi-line comment
// Used for longer descriptions
```

```
/*
=====
```

```
📖 Program: My First JavaScript Program
```

```
Author: Ketan
```

```
Date: 29 April 2026
```

```
Description: This program prints a welcome message
```

```
*/
```

```
// =====
```

```
// 📖 Java vs JavaScript (Basic Difference)
// =====

// Java (Strict typing - need to define data type)
// Example:
// String name = "Program";
// int number = 10;

// JavaScript (Dynamic typing - no need to define type)
let name = "Program"; // String variable
const number = 10;    // Constant variable
```

What is JavaScript?

JavaScript is a **lightweight, interpreted, high-level programming language** and one of the core technologies of the web, along with HTML and CSS.

- Created in **1995 by Brendan Eich** at Netscape
- Designed to make web pages **interactive** (buttons, forms, animations, etc.)
- Runs on:
 - **Browser (Frontend)**
 - **Server (Backend)** using Node.js

Features of JavaScript

1. Interpreted Language

- JavaScript runs **line by line** in the browser or runtime
- No compilation needed (unlike Java, C++)
- Faster development cycle

2. Dynamically Typed

- No need to define variable type
- Type is decided at runtime

```
let x = 10;    // Number
x = "Hello";  // String
x = true;     // Boolean
```

3. Single-Threaded with Event Loop

- Executes one task at a time
- Uses **event loop** to handle multiple operations efficiently
- Enables async behavior (timers, API calls)

4. First-Class Functions

- Functions behave like variables

```
let greet = function () {
  console.log("Hello!");
};
```

```
greet(); // Output: Hello!
```

👉 You can:

- Store functions in variables
- Pass them as arguments
- Return them from other functions

5. Object-Oriented + Functional

- Supports:
 - **OOP** → objects, classes
 - **Functional programming** → functions as values

👉 Makes JavaScript very flexible

6. Platform Independent

- Runs on any device with a browser:
 - Windows
 - Mac
 - Linux
 - Android
 - iOS

👉 Write once, run anywhere

8. Variables

What Are Variables?

- A variable is a named container that stores a value
- You give it a name, and store something inside it (a number, text, true/false, etc.)
- You can use the name later anywhere in your code to get the stored value
- Variables make your code flexible and reusable

Real-life analogy: A variable is like a labelled box. You write a name on the outside (variable name) and place something inside it (value). Whenever you need that thing, you just look at the label and open the box.

How to Create a Variable

```
let age = 25;  
console.log(age); // Output: 25
```

Here:

- `let` → keyword to create a variable
- `age` → name of the variable
- `25` → value stored inside it

Types of Variables: `var`, `let`, `const`

`var` — Old Way (Avoid Using)

- Used in old JavaScript (before 2015)
- Can be redeclared (declared again with the same name) — this causes confusion
- Not recommended in modern JavaScript

```
var city = "New York";  
console.log(city); // Output: New York
```

```
var city = "London"; // Declaring the same variable again — no error, but confusing  
console.log(city); // Output: London
```

Why avoid `var`? Imagine labelling two boxes with the same name. You won't know which box has what. It leads to hard-to-find mistakes.

`let` — Modern Way (Use when value will CHANGE)

- Introduced in 2015 (ES6)
- The value can be changed (reassigned) later
- Cannot be redeclared with the same name in the same block — safer
- Use `let` when the value needs to update during the program

```
let studentName = "Alice";  
console.log(studentName); // Output: Alice
```

```
studentName = "Bob"; // Changing the value
```

```
console.log(studentName); // Output: Bob
```

```
studentName = "Charlie";
console.log(studentName); // Output: Charlie
```

Another Example:

```
let score = 0;
console.log("Starting score: " + score); // Output: Starting score: 0
```

```
score = 10;
console.log("After round 1: " + score); // Output: After round 1: 10
```

```
score = 25;
console.log("Final score: " + score); // Output: Final score: 25
```

const — Use when value will NOT CHANGE

- Short for constant — meaning fixed, unchangeable
- Once a value is set, it cannot be changed
- Use `const` for values that should stay the same throughout the program

```
const country = "India";
const pi = 3.14159;
const appName = "MyApp";
```

```
console.log(country); // Output: India
console.log(pi);      // Output: 3.14159
console.log(appName); // Output: MyApp
```

What happens if you try to change a `const`?

```
const country = "India";
country = "USA"; // ERROR: Assignment to constant variable
```

JavaScript will immediately throw an error. This is actually a good thing — it protects values that should never change.

var vs let vs const — Comparison Table

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Can change value?	Yes	Yes	No
Can declare again?	Yes	No	No
Modern JavaScript?	No — Avoid	Yes — Use	Yes — Use
When to use?	Never	Value changes	Value stays fixed

Variable Naming Rules (Must Follow)

These are strict rules — breaking them causes an error.

1. Must start with a letter, underscore `_`, or dollar sign `$`
2. Cannot start with a number
3. Cannot contain spaces
4. Cannot use reserved words like `let`, `const`, `if`, `function`
5. JavaScript is case-sensitive — `name` and `Name` are two different variables

// Correct variable names

```
let firstName = "Alice";
let age = 25;
let _score = 100;
let $amount = 500;
```

// Wrong variable names — these will cause errors

```
let 1name = "Bob";      // Cannot start with a number
let first name = "Bob"; // Cannot have a space
let let = "Bob";        // Cannot use a reserved keyword
```

Naming Conventions — Best Practices

These are not strict rules but good habits that all professional programmers follow.

camelCase — Recommended for JavaScript

- First word is all lowercase
- Every following word starts with an uppercase letter
- No spaces, no underscores

```
let firstName = "Alice";
let lastName = "Johnson";
let dateOfBirth = "01-01-2000";
let totalMarks = 95;
let isPassedExam = true;
let numberOfStudents = 30;
```

Use Meaningful Names

The name of a variable should clearly describe what it stores.

```
// Bad naming — no one knows what these mean
let a = "Alice";
let b = 25;
let c = true;
```

// Good naming — clear and self-explanatory

```
let studentName = "Alice";
let studentAge = 25;
let isEnrolled = true;
Full Example with Good Naming:
```

Program 1

/*

What Are Variables?

A variable is a named container that stores a value

You give it a name, and store something inside it (a number, text, true/false, etc.)

You can use the name later anywhere in your code to get the stored value

Variables make your code flexible and reusable

Types of Variables

1.let

2.var

3.const

How to create a variable

variables varName = varValue;

*/

/*

let age = 30

let - Keyword to create a variable

age - name of the variable

30 - Value stored inside it

*/

let age = 30;

console.log(age);

// Var - old way (Avoid using)

// Used old javascript (before 2015)

// Can be redeclared

// Not recommended in modern javascript

var city = "New York";

console.log(city);

var city = "London";

console.log(city);

var box = "Book"

console.log(box);

var box = "Bat";

console.log(box);

//let - Modern way (use when value will change)

//The value can be changed (reassigned) later

//use "let" when the value needs to be updated during the program

let studentName = "Rohit";

//let studentName = "Virat";//SyntaxError: Identifier 'studentName' has already been declared

```
console.log(studentNname);
```

```
studentNname = "Virat";
console.log(studentNname);
```

```
studentNname = "Rahul";
console.log(studentNname);
```

```
let score = 0;
console.log("Starting Score = ", score);
console.log("Starting Score = " + score);
```

```
score = 10;
console.log("After Round 1 = ", score)
```

```
score = 25;
console.log("After Round 2 = ", score)
```

```
//3. Const - Use when value not change
//once a value is set it can not be changes
```

```
const country = "India"; //final
// const country = "USA";
const pi = 3.14;
const appName = "MyApp"
```

```
console.log(country);
console.log(pi);
console.log(appName);
```

```
// country = "USA"; //TypeError: Assignment to constant variable
console.log(country);
```

```
let test = true;
let test1 = false;
console.log(test);
console.log(test1);
```

```
//diff          var    let    const
```

```
// Can change value ? - Yes    yes    No
// Can declare Again. - Yes.    No.    No
// Whne to use.      - Never. Value Change Value stays Fixed
// Can be reaasing   - Yes.    Yes.    No
```

Program 2

```
//Variable Naming Rules
//1.Must start with letter ,underscore(_), dollar sign ($)
//2.Cannot start with a number
//3.Cannot contains spaces
//4.Cannot use reserved word like let,const,if ,for
//5. Javascript is case sensitive - name and Name are two diff variables
```

```
//1 Correct variableNames
```

```
let firstName = "Rohit";
```

```
let age = 30;
```

```
let _score = 100
```

```
let $amount = 500
```

```
// let #rp = 500;
```

```
console.log(firstName);
```

```
console.log(age);
```

```
console.log(_score);
```

```
console.log($amount);
```

```
//Wrong variables names
```

```
// let 1name = "Test";
```

```
// let firstName lastName = "Bob";
```

```
// let var = "Demo";
```

```
// let 1Test123 = "number"
```

```
//Naming convenations - Best Practices
```

```
// camelCase -
```

```
// First word is alllowercase
```

```
// Every following word tarts withn an uppercase letter
```

```
// No spaces no underscore
```

```
let firstName = "Alice";
```

```
let lastName = "John";
```

```
let dateOfBirth = "1-01-2026"
```

```
//Bad Naming
```

```
let a = "Rohit";
```

```
let b = 25;
```

```
let c = true;
```

```
console.log(a)
```

Data Types

1.1 What Are Data Types?

A **data type** tells JavaScript what kind of value a variable holds — is it a number, a piece of text, true/false, or a complex object?

Think of data types like containers in real life:

- A **bottle** holds liquid (Number)
- A **book** holds text (String)
- A **light switch** is either ON or OFF (Boolean)
- An **empty box** is undefined or null

Every value in JavaScript has a data type.

1.2 Why Are Data Types Important?

Reason	Example
Determines what operations are valid	You can multiply numbers, not names
Prevents bugs	Adding a number to undefined gives NaN
Controls memory allocation	Primitives → Stack, Objects → Heap
Affects how values are compared	"5" == 5 is true, "5" === 5 is false

1.2 Why Do Data Types Matter?

```
let a = 5;  
let b = 3;  
console.log(a + b); // 8 — adds the numbers
```

```
let x = "5";  
let y = 3;  
console.log(x + y); // "53" — joins as text!
```

The same + symbol behaves differently depending on the data type.
This is why knowing data types is very important.

1.3 JavaScript is Dynamically Typed

In JavaScript, you do NOT declare the type. JavaScript figures it out automatically from the value.

```
let data = 42;
```

```
console.log(typeof data); // "number"
```

```
data = "Hello";  
console.log(typeof data); // "string"  
data = true;  
console.log(typeof data); // "boolean"
```

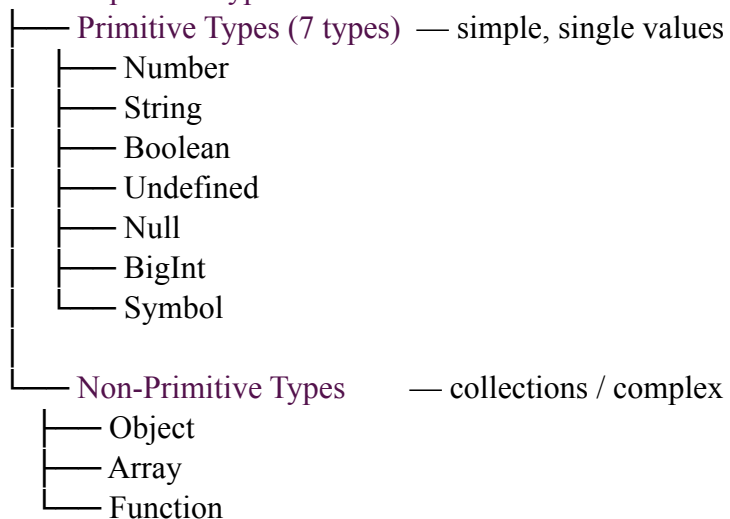
The same variable data changed its type three times — this is called **dynamic typing**.

Compare with Java (a statically typed language):

```
int age = 25;    // must declare type  
age = "twenty"; // ❌ Error — cannot change type
```

2. Two Categories of Data Types

JavaScript Data Types



Simple rule:

- **Primitive** → stores one single value (a number, a word, true/false)
- **Non-Primitive** → stores a group of values (many items together)

3. Primitive Data Types

All primitive types are **immutable** — once a value is created in memory, it cannot be changed. When you reassign a variable, a new value is created.

3.1 Number

The Number type represents **all numeric values** — whole numbers, decimals, positive, negative.

⚠ JavaScript has only ONE number type for everything. There is no separate int or float.

```
let age = 25;
let price = 19.99;
let temperature = -5;
let bigNum = 1000000;
```

Code Example 1 — Basic Arithmetic

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7

console.log(typeof a); // "number"
```

Code Example 2 — Real-Time: Bill Calculation

```
let itemPrice = 200;
let quantity = 3;
let discount = 50;

let subtotal = itemPrice * quantity;
let totalAfterDiscount = subtotal - discount;
let tax = totalAfterDiscount * 0.18;

console.log("Subtotal: " + subtotal);
console.log("After Discount: " + totalAfterDiscount);
console.log("Tax (18%): " + tax);
console.log("Final: " + (totalAfterDiscount + tax));
```

3.2 String

A String is used to store **text** — any characters: letters, numbers, symbols, spaces. Strings are always wrapped in quotes.

Three Ways to Write a String

```
let a = 'Single quotes';  
let b = "Double quotes";  
let c = `Backticks`;
```

Syntax

```
let firstName = "John";  
let city = 'Mumbai';  
let greeting = `Hello, John!`;
```

Code Example 1 — Basic String Operations

```
let firstName = "John";  
let lastName = "Doe";  
  
let fullName = firstName + " " + lastName;  
console.log(fullName);  
  
let modern = `Hello, ${firstName} ${lastName}!`;  
console.log(modern);  
  
console.log(firstName.length);  
console.log(firstName.toUpperCase());  
console.log(firstName.toLowerCase());  
console.log(" hello ".trim());  
console.log("hello world".includes("world"));  
console.log("hello".replace("hello", "hi"));  
  
console.log(typeof firstName);
```

Code Example 2 — Real-Time: User Display Card

```
let userName = "Priya Sharma";  
let userEmail = "PRIYA@GMAIL.COM";  
let userPlan = "premium";  
let accountYear = 2026;  
  
let displayName = userName.toUpperCase();
```

```
let formattedEmail = userEmail.toLowerCase();
let planLabel = userPlan.toUpperCase();

console.log("Name: " + displayName);
console.log("Email: " + formattedEmail);
console.log("Plan: " + planLabel);
console.log(`Welcome to Farm Pass, ${userName}!`);
console.log("Member since: " + accountYear);
```

3.3 Boolean

A Boolean holds only **one of two values**: true or false.

Syntax

```
let isLoggedIn = true;
let hasDiscount = false;
let isAdult = true;
let isSubscribed = false;
```

Code Example

```
let age = 20;

let isAdult = age >= 18;
let isTeenager = age < 18;

console.log(isAdult);
console.log(isTeenager);
console.log(typeof isAdult);
```

3.4 Undefined

undefined means a variable was declared but no value was assigned yet.

```
let userName;
console.log(userName);
console.log(typeof userName);
```

Example

```
let productName = "Maize Seeds";
let productPrice = 500;
let productColor;
```

```
let productReview;

console.log(productColor);
console.log(productReview);
console.log(typeof productColor);
```

3.5 Null

null is an intentional empty value.
let selectedItem = null;

```
console.log(selectedItem);
console.log(typeof selectedItem);
```

Feature	Undefined	Null
Meaning	Not assigned	Intentionally empty
Assigned by	JavaScript	Developer/Tester
Type	undefined	object !
Use case	Default state	Reset / no value explicitly

3.6 BigInt

Used for very large integers.

Add n at end to create BigInt. Cannot mix with normal numbers **!**

```
let bigNum = 9007199254740992n;
console.log(bigNum + 1n);
console.log(typeof bigNum);
```

Program 1

```
// =====
// Arithmetic Operations
// =====

let a = 5;
let b = 3;

// Addition of two numbers
console.log("Addition of two numbers:", a + b); // 8
```

```
// =====
// Type Coercion (Important Concept)
// =====
```

```
let x = "5"; // String
let y = 3; // Number
```

```
// JS converts number → string and performs concatenation
console.log(x + y); // "53" ! (NOT 8)
```

```
// =====
// Static vs Dynamic Typing
// =====
```

```
// Java → Statically Typed (type cannot change)
/*
int number = 40;
number = "Hello"; // ✗ Error
*/
```

```
// JavaScript → Dynamically Typed (type can change)
let data = 40;
console.log(typeof data); // number
```

```
data = "Hello";
console.log(typeof data); // string
```

```
data = true;
console.log(typeof data); // boolean
```

```
// =====
// Types of Data Types
// =====
```

```
/*
1. Primitive Data Types (Single Value)
- Number
- String
- Boolean
- Undefined
- Null
- BigInt
- Symbol
```

2. Non-Primitive Data Types (Multiple Values)

- Object
- Array
- Function

*/

```
// =====  
// Number Data Type  
// =====
```

```
let age = 25;           // integer  
let price = 20.23;      // decimal  
let temperature = -6;   // negative  
let bigNumber = 10000000; // large number
```

```
console.log(typeof age);  
console.log(typeof price);  
console.log(typeof temperature);  
console.log(typeof bigNumber);
```

```
// =====  
// Arithmetic Example  
// =====
```

```
let a1 = 10;  
let b1 = 2;
```

```
console.log(a1 + b1); // 12  
console.log(a1 - b1); // 8
```

```
// =====  
// Real-time Example (Shopping)  
// =====
```

```
let itemPrice = 200;  
let quantity = 3;  
let discount = 50;
```

```
let subTotal = itemPrice * quantity;    // 200 * 3 = 600  
let totalAfterDiscount = subTotal - discount; // 600 - 50 = 550  
let tax = totalAfterDiscount * 0.15;    // 15% tax
```

```
console.log("Subtotal:", subTotal);  
console.log("After Discount:", totalAfterDiscount);  
console.log("Final Amount:", totalAfterDiscount + tax);
```

```
// =====
// String Data Type
// =====

// String = text (letters, numbers, symbols, spaces)

// 3 ways to declare string
let s1 = 'Single quotes';
let s2 = "Double quotes";
let s3 = `Backticks`;

console.log(typeof s1);
console.log(typeof s2);
console.log(typeof s3);
```

```
// =====
// String Example
// =====
```

```
let firstName = "John";
let cityName = "Mumbai";

// Template literal (best practice)
let greeting = `Hello, ${firstName}!`;

console.log(firstName);
console.log(cityName);
console.log(greeting);
console.log(0.1 + 0.2);
```

Program 2

```
// =====
// ◆ STRING IN JAVASCRIPT
// =====

// 3 ways to declare string
let s1 = 'Single quotes'; // Using single quotes
let s2 = "Double quotes"; // Using double quotes
let s3 = `Backticks`;    // Using backticks (template literals)

// Declaring first and last name
let firstName = "John";
let lastName = `Doe`;    // Backticks can also store normal string
```

```
// typeof checks the data type
console.log(typeof firstName); // string
console.log(typeof lastName); // string

// Printing multiple values
console.log(firstName, lastName); // John Doe

// String concatenation using +
console.log(firstName + " " + lastName); // John Doe

// Template literal (best modern way)
let fullName = `Hello, ${firstName} ${lastName}`;
// ${} is used to insert variables inside string
console.log(fullName); // Hello, John Doe
```

```
// =====
// ♦ REAL-TIME EXAMPLES
// =====
```

```
let name = "Rahul";
let message = `Hello ${name}`; // Dynamic message
console.log(message); // Hello Rahul
```

```
let age = 25;
```

```
// Embedding number inside string
let text = `I am ${age} years old`;
console.log(text); // I am 25 years old
```

```
let product = "Laptop";
let price = 50000;
```

```
// Using template literal (recommended)
console.log(`The ${product} price is ${price}`);
```

```
// Old way (concatenation)
console.log(`The` + " " + product); // The Laptop
```

```
// 2. Backticks allow multi-line strings
let multiLine = `This is line 1
This is line 2`;
console.log(multiLine);
```

```
// 3. Template literals improve readability
// Avoid using + for long strings
```


// 4. Always prefer backticks in real projects

// 5. typeof string always returns "string"

Program 3

// =====

//  BOOLEAN IN JAVASCRIPT

// =====

// A Boolean can hold only two values:

// true or false

```
let isLoggedIn = true;    // User login status
let hasDiscount = false; // Discount availability
let isAdult = true;      // Age check
let isSubscribed = false; // Subscription status
```

// typeof checks the data type

```
console.log(typeof isLoggedIn); // boolean
```

```
console.log(typeof hasDiscount); // boolean
```

// Reassigning value

```
isLoggedIn = true; // Valid
```

//  Incorrect (JavaScript is case-sensitive)

```
// hasDiscount = False; // Error → should be 'false'
```

// Checking types again

```
console.log(typeof isLoggedIn); // boolean
```

```
console.log(typeof hasDiscount); // boolean
```

// =====

//  BOOLEAN WITH CONDITIONS

// =====

```
let age = 20;
```

// Comparison operators return boolean values

```
let isAdult1 = age >= 18; // true (20 >= 18)
```

```
let isTeenager = age <= 19; // false (20 <= 19)
```

// Printing results

```
console.log(isAdult1); // true
```

```
console.log(isTeenager); // false
```

```
// Checking type
console.log(typeof isAdult1); // boolean

// =====
// ♦ IMPORTANT POINTS
// =====

// 1. Boolean values are always lowercase
// true ✔
// false ✔
// True ✗
// False ✗

// =====
// ♦ TRICKY INTERVIEW POINTS
// =====

console.log(Boolean(1)); // true
console.log(Boolean(0)); // false
console.log(Boolean("")); // false
console.log(Boolean("Hi")); // true
console.log(Boolean(null)); // false
console.log(Boolean(undefined)); // false
```

Program 3

```
// =====
// ♦ PRIMITIVE DATA TYPES
// Number, String, Boolean, Undefined, Null, BigInt
// =====

// =====
// ♦ UNDEFINED
// undefined means a variable is declared but no value is assigned
// =====

let userName; // declared but not assigned

console.log("Test : " + userName); // Test : undefined
```

```
console.log(typeof userName); // undefined
```

```
let productName = "Seeds"; // string
let productPrice = 200; // number
let productColor; // undefined
let productReview; // undefined
```

```
console.log(typeof productColor); // undefined
console.log(typeof productName); // string
console.log(typeof productPrice); // number
```

```
// Assigning value later
productColor = "Red";
```

```
console.log(productColor); // Red
console.log(typeof productColor); // string
```

```
// =====
// ◆ NULL
// null means intentionally empty value (assigned by developer)
// =====
```

```
let selectedItem = null;
```

```
console.log(selectedItem); // null
console.log(typeof selectedItem); // object ! (JavaScript known bug)
```

```
let currentUser = "TestUser";
```

```
// User logs out
console.log(currentUser); // TestUser
```

```
currentUser = null; // intentionally clearing value
```

```
console.log(typeof currentUser); // object
console.log(currentUser); // null
```

```
// =====
// ◆ DIFFERENCE
// undefined → value not assigned
// null → value intentionally empty
// =====
```

```
// =====
// ♦ BIGINT
// BigInt is used for very large numbers
// It is written by adding 'n' at the end (NOT 'm')
// =====
```

```
let bigNum = 9838n; // Correct BigInt
```

```
console.log(typeof bigNum); // bigint
console.log(bigNum);
```

```
// =====
// ♦ FLOATING POINT ISSUE
// =====
```

```
// JavaScript follows IEEE 754 standard
// So decimal calculations may be slightly inaccurate
```

```
console.log(0.1 + 0.2); // 0.30000000000000004 !
```

```
// More examples
console.log(0.1 + 0.4); // 0.5 (sometimes correct, sometimes precision issue)
```

```
// =====
// ♦ FIXING PRECISION ISSUE
// =====
```

```
// toFixed() rounds the number and returns STRING
let result = (0.1 + 0.2).toFixed(2);
```

```
console.log(result); // "0.30"
console.log(typeof result); // string !
```

```
// Convert back to number if needed
let finalResult = Number(result);
```

```
console.log(finalResult); // 0.3
console.log(typeof finalResult); // number
```

```
// =====
// ♦ IMPORTANT NOTES
// =====
```

```
// 1. undefined → automatic by JavaScript
// 2. null → manually assigned
// 3. typeof null → "object" (bug)
// 4. BigInt must end with 'n'
// 5. Floating point issues happen due to IEEE 754
// 6. toFixed() returns string
```

```
// =====
// 💎 INTERVIEW TIP
// =====
```

```
console.log(undefined == null); // true
console.log(undefined === null); // false
```

3.7 Symbol

Creates a unique value.

```
let sym1 = Symbol("id");
let sym2 = Symbol("id");
```

```
console.log(sym1 === sym2);
console.log(typeof sym1);
```

💎 Why do we use Symbol?

✅ 1. To create unique keys in objects

No other property can accidentally overwrite it.

✅ 2. To avoid naming conflicts

Very useful in large applications or frameworks.

4. Non-Primitive (Reference) Data Types

4.1 Object

An Object stores **key-value pairs** — like a form with labeled fields. Each piece of data has a **name (key)** and a **value**.

Real-Life Analogy

An employee ID card:

- Name: John Doe
- Department: Engineering
- Salary: 85000

```
let employee = {  
  name: "John Doe",  
  department: "Engineering",  
  salary: 85000  
};
```

Syntax

```
let objectName = {  
  key1: value1,  
  key2: value2,  
  key3: value3  
};
```

Code Example 1 — Basic Object

```
let person = {  
  name: "Alice",  
  age: 30,  
  city: "Mumbai",  
  isEmployed: true  
};
```

```
// Accessing values — dot notation  
console.log(person.name);    // "Alice"  
console.log(person.age);     // 30  
console.log(person.city);    // "Mumbai"  
console.log(person.isEmployed); // true
```

```
// Accessing values — bracket notation  
console.log(person["name"]);  // "Alice"  
console.log(person["age"]);   // 30
```

```
// Changing a value
person.age = 31;
console.log(person.age);    // 31

// Adding a new property
person.email = "alice@gmail.com";
console.log(person.email);  // "alice@gmail.com"

// Deleting a property
delete person.city;
console.log(person.city);   // undefined — it's gone

console.log(typeof person); // "object"
```

Code Example 2 — Real-Time: Product Listing

```
let product = {

  id:      1001,
  name:    "Maize Seeds",
  sku:     "ABC123",
  price:   50,
  units:   20,
  category: "Seeds",
  isAvailable: true
};

let totalValue = product.price * product.units;

console.log("Product: " + product.name);    // Product: Maize Seeds
console.log("SKU: " + product.sku);         // SKU: ABC123
console.log("Price: ₹" + product.price);    // Price: ₹50
console.log("Units: " + product.units);     // Units: 20
console.log("Total: ₹" + totalValue);       // Total: ₹1000
console.log("Available: " + product.isAvailable); // Available: true
console.log("Category: " + product.category); // Category: Seeds
```

4.2 Array

Definition

An Array is an **ordered list** of values.

Think of it like a numbered shelf — each slot has a number (called an **index**) starting at **0**.

Real-Life Analogy

A shopping list:

- Slot 0 → Maize Seeds
- Slot 1 → DT Fertilizer
- Slot 2 → Urea

```
let shoppingList = ["Maize Seeds", "DT Fertilizer", "Urea"];  
let arrayName = [item1, item2, item3];
```

Code Example 1 — Basic Array

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];  
  
// Access by index (starts at 0)  
console.log(fruits[0]); // "Apple"  
console.log(fruits[1]); // "Banana"  
console.log(fruits[3]); // "Orange"  
  
// Length  
console.log(fruits.length); // 4  
  
// Last item (using length)  
console.log(fruits[fruits.length - 1]); // "Orange"  
  
// Add to end  
fruits.push("Grapes");  
console.log(fruits.length); // 5  
console.log(fruits[4]); // "Grapes"  
  
// Remove last item  
fruits.pop();  
console.log(fruits.length); // 4  
  
// Check type  
console.log(typeof fruits); // "object" ← arrays are objects in JS!  
console.log(Array.isArray(fruits)); // true
```



```
console.log("Total Items in Order: " + orderItems.length); // 5
```

4.3 Function

Functions are **reusable blocks of code**. We will cover them in detail in the Functions chapter. For now, just understand that functions are also a data type in JavaScript:

```
// This is a function — notice typeof says "function"
function sayHello() {
  console.log("Hello!");
}

console.log(typeof sayHello); // "function"
```

JavaScript Operators

1. What is an Operator?

An **operator** is a symbol that performs an operation on one or more values (operands) and produces a result.

Example:

```
let result = 10 + 5;
```

- $+$ $\rightarrow$ Operator
- 10, 5 $\rightarrow$ Operands
- 15 $\rightarrow$ Result

2. Real-Time Document Example (E-commerce Bill System)

 This example simulates a **shopping bill calculation system**

```
// Product details
let productName = "Laptop";
let price = 50000;
let quantity = 2;

// Discount and tax
let discount = 5000;
let taxRate = 0.18;

// Step 1: Calculate subtotal
let subtotal = price * quantity;

// Step 2: Apply discount
let afterDiscount = subtotal - discount;

// Step 3: Calculate tax
let tax = afterDiscount * taxRate;

// Step 4: Final bill
let finalAmount = afterDiscount + tax;

// Print bill
console.log("Product:", productName);
console.log("Subtotal:", subtotal);
console.log("After Discount:", afterDiscount);
console.log("Tax:", tax);
```

```
console.log("Final Amount:", finalAmount);
```



Output Explanation

- Subtotal $\rightarrow$ price $\times$ quantity
- Discount applied $\rightarrow$ reduces amount
- Tax added $\rightarrow$ final payable amount



3. Arithmetic Operators

Used in the above example:

+ $\rightarrow$ Addition
- $\rightarrow$ Subtraction
* $\rightarrow$ Multiplication
/ $\rightarrow$ Division
% $\rightarrow$ Remainder
** $\rightarrow$ Power



4. Assignment Operators

```
let total = 100;  
total += 50; // 150  
total -= 20; // 130
```



5. Comparison Operators

```
let price = 500;  
  
console.log(price > 300); // true  
console.log(price === "500"); // false
```



6. Logical Operators (Real Example)

```
let age = 22;  
let hasID = true;  
  
console.log(age >= 18 && hasID); // true
```

7. Increment Example (Stock Update)

```
let stock = 10;

// One item sold
stock--;
console.log(stock); // 9

// New stock added
stock++;
console.log(stock); // 10

let stock1 = 10;

// One item sold
stock--;
console.log(stock1); // 9

// New stock added
stock++;
console.log(stock1); // 10
```

8. Difference Between == and ===

```
console.log(10 == "10"); // true
console.log(10 === "10"); // false
```

👉 Rule:

Always use === for accurate results.

// Ternary Operator

The only operator that takes THREE operands
Short-hand version of if...else

// Syntax:

condition ? valueIfTrue : valueIfFalse

```
let age = 20;
```

```
let status = age >= 18 ? "Adult" : "Minor";
```

```
console.log(status); // Adult
```

Operators Program 1

1.

// 📌 Operator: A symbol that performs an operation on one or more values (operands)

// Example:

```
let result = 10 + 5; // 10 and 5 are operands, + is operator
```

```
console.log(result); // 15
```

```
/*
```

```
=====
```

◆ Arithmetic Operators

```
=====
```

1. Addition (+)

2. Subtraction (-)

3. Multiplication (*)

4. Division (/)

5. Modulus (%) → remainder

6. Exponentiation (**)

```
*/
```

```
let a = 17;
```

```
let b = 5;
```

```
console.log(a + b); // 22 → Addition
```

```
console.log(a - b); // 12 → Subtraction
```

```
console.log(a * b); // 85 → Multiplication
```

```
console.log(a / b); // 3.4 → Division
```

```
console.log(a % b); // 2 → Remainder
```

```
console.log(a ** b); // 17^5 → Exponentiation
```

// 📌 Real-time Example (Billing Calculation)

```
let itemPrice = 250;
```

```
let quantity = 7;
```

```
let discount = 100;
```

```
let taxRate = 0.12;
```

```
let subtotal = itemPrice * quantity; // 250 * 7
```

```
let afterDisc = subtotal - discount; // apply discount
```

```
let taxAmount = afterDisc * taxRate; // calculate tax
```

```
let finalBill = afterDisc + taxAmount; // total amount
```

```
console.log('Final bill = ' + finalBill);
```

/*

=====

◆ Increment & Decrement

=====

++ → Increase by 1

-- → Decrease by 1

*/

let count = 5;

// Prefix → change first, then return

console.log(++count); // 6

console.log(count); // 6

count = 5;

// Postfix → return first, then change

console.log(count++); // 5

console.log(count); // 6

/*

=====

◆ Assignment Operators

=====

= → assign value

+= → add and assign

-= → subtract and assign

*= → multiply and assign

/= → divide and assign

%= → modulus and assign

*/

let score = 100;

score += 50; console.log(score); // 150 → score = score + 50

score -= 20; console.log(score); // 130

score *= 2; console.log(score); // 260

score /= 4; console.log(score); // 65

score %= 100; console.log(score); // remainder

/*

=====

◆ Comparison Operators

=====

Used to compare values → returns true/false

== → loose equality (ignores type)
 === → strict equality (checks type + value)
 != → loose not equal
 !== → strict not equal
 > → greater than
 < → less than
 >= → greater than or equal
 <= → less than or equal
 */

```
let a1 = 10;    // number
let b1 = "10";  // string
let c1 = 20;
```

```
// Loose comparison (==)
console.log(a1 == b1); // true (string "10" converted to number)
console.log(a1 == 10); // true
```

```
console.log("-----");
```

```
// Strict comparison (===)
console.log(a1 === b1); // false (different types)
console.log(a1 === 10); // true
```

```
// Relational operators
console.log(a1 > c1); // false
console.log(a1 < c1); // true
console.log(a1 >= 10); // true
console.log(a1 <= 9); // false
```

```
console.log("-----");
```

```
// Type coercion examples with ==
console.log(0 == false); // true
console.log(1 == true);  // true
console.log("" == false); // true
console.log("1" == 1);   // true
```

```
console.log("-----");
```

```
// Strict equality (===)
console.log(0 === false); // false
console.log("" === false); // false
console.log(1 === true);  // false
console.log("1" === 1);   // false
```

```
console.log(1 === 1);    // true
```

```
// 🖱️ Rule: Always use === and !== in real projects
```

```
// Not equal operators
```

```
console.log(1 !== 1);    // false
console.log(1 !== 5);    // true
console.log("1" !== 3);  // true
console.log("1" !== 3);  // true
console.log("1" !== 1);  // false
console.log("1" !== 1);  // true
```

```
/*
```

```
=====
```

◆ Logical Operators

```
=====
```

```
&& → AND (both must be true)
```

```
|| → OR (any one true)
```

```
*/
```

```
// AND (&&)
```

```
// true && true → true
```

```
// true && false → false
```

```
// false && true → false
```

```
// false && false → false
```

```
// OR (||)
```

```
// true || true → true
```

```
// true || false → true
```

```
// false || true → true
```

```
// false || false → false
```

```
let age = 20;
```

```
let hasID = true;
```

```
console.log(age >= 18 && hasID); // true
```

```
console.log(age >= 18 && !hasID); // false
```

```
/*
```

Operator	Name	Checks	Type Conversion
'=='	Loose Equality	Only value	✅ Yes
'==='	Strict Equality	Value + Type	❌ No


```
*/
```

```
console.log(age >= 18 || hasID); // true
console.log(age >= 18 || !hasID); // true
```

Program 2

```
// =====
// ♦ Short Circuit Evaluation in JavaScript
// =====
```

```
// AND (&&) operator
// → Stops at the FIRST falsy value
// → Evaluates from LEFT to RIGHT
```

```
console.log(false && "hello"); // false
// false is falsy → stops immediately
```

```
console.log(true && "hello"); // hello
// true → move next → returns last value
```

```
console.log(0 && "hello"); // 0
// 0 is falsy → stops immediately
```

```
// =====
// ♦ Falsy Values in JavaScript
// =====
```

```
// false
// 0
// ""
// null
// undefined
// NaN
```

```
// =====
// ♦ OR (||) Operator
// =====
```

```
// → Stops at FIRST truthy value
```

```
let isAdmin = false;
let isEditor = true;
```

```
console.log(isAdmin || isEditor); // true
// false || true → true
```

```
console.log(false || false); // false
// both are false
```

```
console.log(false || "default"); // default
// false is falsy → returns next value
```

```
console.log("saved" || "default"); // saved
// "saved" is truthy → stops immediately
```

```
// =====
// ♦ NOT (!) Operator
// =====
// → Reverses boolean value
```

```
console.log(!true); // false
console.log(!false); // true
console.log(!0); // true
// 0 is falsy → !0 becomes true
```

```
// =====
// ♦ Ternary Operator
// =====
```

```
// The only operator that takes THREE operands
// Short-hand version of if...else
```

```
// Syntax:
// condition ? valueIfTrue : valueIfFalse
```

```
let age = 20;
```

```
let status = age >= 18 ? "Adult" : "Minor";
console.log(status); // Adult
```

```
age = 10;
```

```
let status1 = age >= 18 ? "Adult" : "Minor";
console.log(status1); // Minor
```

```
let marks = 75;
```

```
let result = marks >= 50 ? "PASS" : "FAIL";
console.log(result); // PASS
```

```
marks = 45;
```

```
let result1 = marks >= 50 ? "PASS" : "FAIL";
console.log(result1); // FAIL
```

```
let stock = 1;
```

```
let availability = stock > 1 ? "In Stock" : "Out of stock";
console.log(availability); // Out of stock
```

```
// =====
// ♦ typeof Operator
// =====
// Used to check datatype of variable
```

```
let name = "Rohit";
let age1 = 30;
let score = 100;
let pass = true;
let list = [1, 2, 3];
let obj = { key: "value" };
```

```
console.log(typeof name); // string
console.log(typeof age1); // number
console.log(typeof score); // number
console.log(typeof pass); // boolean
console.log(typeof list); // object
console.log(typeof obj); // object
```

```
// =====
// ♦ Boolean Conversion
// =====
```

```
console.log("=====");
```

```
console.log(Boolean(false)); // false
console.log(Boolean(0)); // false
console.log(Boolean("")); // false
```

```
console.log(Boolean(undefined)); // false
console.log(Boolean(NaN));      // false
```

```
console.log(Boolean("Test"));  // true
console.log(Boolean([]));      // true
// Empty array is truthy
```

```
// =====
// ♦ Type Coercion Examples
// =====
```

```
console.log(1 + "2" + 3); // 123
// 1 + "2" → "12"
// "12" + 3 → "123"
```

```
console.log(1 + 2 + "3"); // 33
// 1 + 2 → 3
// 3 + "3" → "33"
```

```
console.log("3" + 2 + 1); // 321
// String concatenation
```

```
// =====
// ♦ Boolean Values in Arithmetic
// =====
```

```
console.log(true + true); // 2
// true = 1
```

```
console.log(true + false); // 1
// 1 + 0
```

```
console.log(false + false); // 0
// 0 + 0
```

```
// =====
// ♦ null and undefined in Arithmetic
// =====
```

```
console.log(null + 1); // 1
// null becomes 0
```

```
console.log(undefined + 1); // NaN
// undefined cannot convert to number
```

```
// =====
// ◆ == vs ===
// =====
```

// == → compares VALUE only
 // === → compares VALUE + DATATYPE

```
console.log(0 == ""); // true
// "" becomes 0
```

```
console.log(0 === ""); // false
// number !== string
```

```
console.log("" == false); // true
// both converted to 0
```

```
console.log("" === false); // false
// string !== boolean
```

```
// =====
// ◆ Post Increment (x++)
// =====
```

```
let x = 5;
```

```
console.log(x++); // 5
// first prints → then increments
```

```
console.log(x); // 6
```

```
// =====
// ◆ Pre Increment (++y)
// =====
```

```
let y = 5;
```

```
console.log(++y); // 6
// first increments → then prints
```

```
console.log(y); // 6
```

```
// =====
```

```
//  More Type Coercion Example
```

```
// =====
```

```
console.log("3" + 1 + 2); // 312
```

```
// "3" + 1 → "31"
```

```
// "31" + 2 → "312"
```

```
// =====
```

```
//  IMPORTANT POINTS
```

```
// =====
```

```
//  && returns first falsy value OR last truthy value
```

```
//  || returns first truthy value
```

```
//  ! reverses boolean value
```

```
//  Ternary operator is shorthand for if...else
```

```
//  typeof checks datatype
```

```
//  JavaScript automatically converts datatypes
```

```
// (called Type Coercion)
```

```
//  == checks only value
```

```
//  === checks value + datatype
```

```
//  Empty array [] is truthy
```

```
//  null converts to 0 in arithmetic
```

```
//  undefined gives NaN in arithmetic
```

```
// =====
```

```
//  Interview One-Liners
```

```
// =====
```

```
// “JavaScript uses short-circuit evaluation in && and || operators.”
```

```
// “=== is recommended over == because it checks both value and datatype.”
```

```
// “JavaScript performs automatic type coercion during operations.”
```

Introduction — JavaScript's Type System (Type Conversion/Type Casting)

JavaScript is a **dynamically typed** language.

This means you do NOT declare the type of a variable — JavaScript figures it out automatically based on the value you assign.

```
let x = 42;    // x is a Number
let x = "hello"; // now x is a String — no error!
let x = true;  // now x is a Boolean — still no error!
```

Why does this matter?

Because JavaScript sometimes **automatically converts** a value from one type to another behind the scenes — which can lead to surprising results if you don't understand what's happening.

```
console.log("5" + 2); // "52" ← became a string!
console.log("5" - 2); // 3   ← became a number!
```

That is JavaScript's **type conversion** in action.

PART 1 — TYPE CONVERSION

Type conversion = changing a value from one data type to another.

There are **two kinds**:

Feature	Implicit (Automatic)	Explicit (Manual)
Who does it?	JavaScript, behind the scenes	You, the developer
Also called	Type Coercion	Type Casting
Example	"5" - 2 → 3	Number("5") → 5

Topic 1.1 — Implicit Type Conversion (Type Coercion)

JavaScript automatically converts types when you mix them in an expression.

String Coercion — The + Trap

When one side of + is a string, JavaScript converts the other side to a string too (**concatenation wins over addition**).

```

console.log("5" + 2);    // "52" ← 2 → "2", then joined
console.log("5" + 2 + 1); // "521" ← left to right: "52" + 1 = "521"
console.log(5 + 2 + "1"); // "71" ← left to right: 7 + "1" = "71"
console.log("Age: " + 20); // "Age: 20"
console.log(true + "!");  // "true!"
console.log(null + " value");// "null value"

```

⚡ Rule of Thumb

👉 + is "infected" by strings.

Any string in the expression turns everything into string concatenation.

📖 Numeric Coercion — -, *, /, %

All other arithmetic operators force conversion to Number.

```

console.log("5" - 2); // 3
console.log("5" * 2); // 10
console.log("10" / 2); // 5
console.log("10" % 3); // 1

```

📖 Boolean Coercion

Any value used in a boolean context (like if) is automatically converted to true or false.

🔥 Falsy Values — Only 6 Exist

```

console.log(Boolean(false)); // false
console.log(Boolean(0));     // false
console.log(Boolean(""));    // false
console.log(Boolean(null));   // false
console.log(Boolean(undefined)); // false
console.log(Boolean(NaN));    // false

```

⚠ Interview Trap: "0", [], {} are ALL truthy even though they look "empty".

📖 Topic 1.2 — Explicit Type Conversion

You manually convert a value using built-in functions.

 Convert to String **Method 1 — String()**

```
console.log(String(123)); // "123"
console.log(String(3.14)); // "3.14"
```

 Method 2 — .toString()

```
let age = 25;
console.log(age.toString()); // "25"
```

 Convert to Number **Number()**

```
console.log(Number("42")); // 42
console.log(Number("3.14")); // 3.14
console.log(Number("")); // 0
```

 Convert to Boolean **Boolean()**

```
console.log(Boolean(1)); // true
console.log(Boolean(0)); // false
console.log(Boolean("yes")); // true
```

 Falsy Values (Memorize These 6)

false | 0 | "" | null | undefined | NaN

 + Operator Quick Rules

```
number + number = number    (5 + 3 = 8)
string + anything = string    ("5" + 3 = "53")
number + boolean = number     (5 + true = 6)
null + number = number        (null + 5 = 5)
undefined + number = NaN       (undefined + 5 = NaN)
```

 == vs ===

```
=== → same TYPE + same VALUE → true (preferred)
==  → ignores type, converts then compares (avoid)
```

🔥 Examples

```
console.log(5 == "5"); // true
console.log(5 === "5"); // false
```

🎯 Final Interview Tip

✅ Always prefer: ===

❌ Avoid: ==

Because == performs implicit type conversion and can produce unexpected results.

Program 1

```
// =====

// 🔥 TYPE CONVERSION / TYPE COERCION IN JAVASCRIPT
// =====

// Types of Conversion:
// 1. Implicit Conversion (Automatic conversion by JavaScript)
// 2. Explicit Conversion (Manual conversion by developer)

// =====
// ♦ IMPLICIT TYPE CONVERSION (AUTOMATIC)
// =====

// JavaScript automatically converts one data type into another
// when different data types are used in an expression.

// =====
// ♦ STRING COERCION
// =====

// Rule:
// If one side of + operator is string,
// JavaScript converts everything into string concatenation.

console.log("5" + 2 + 1);
// Output: "521"
// Step:
// "5" + 2 => "52"
// "52" + 1 => "521"

console.log(5 + 2 + "1");
// Output: "71"
// Step:
// 5 + 2 => 7
```

```
// 7 + "1" => "71"

console.log("Age :" + 20);
// Output: "Age :20"

console.log(true + "!");
// Output: "true!"
// Boolean true converted into string

// =====
// 🔥 IMPORTANT POINT
// =====

// "+" operator behaves differently:
// ✓ With strings -> concatenation
// ✓ With numbers -> addition

// Any string in expression can convert remaining values into
string.

// =====
// 💎 NUMERIC COERCION
// =====

// Arithmetic operators except "+" force values into numbers.

console.log("10" - 2);
// Output: 8
// "10" converted to number 10

console.log("10" * 2);
// Output: 20

console.log("10" / 5);
// Output: 2

console.log("10" % 3);
// Output: 1

console.log("5" - true);
// Output: 4
// true = 1
// 5 - 1 = 4

console.log("5" - false);
// Output: 5
// false = 0
```

```

console.log("5" - undefined);
// Output: NaN
// undefined cannot convert properly into number

// =====
// ♦ BOOLEAN COERCION
// =====

// Any value used in boolean context
// is automatically converted into true or false.

console.log("-----");

// Falsy Values
console.log(Boolean(false));
console.log(Boolean(0));
console.log(Boolean(""));
console.log(Boolean(null));
console.log(Boolean(undefined));
console.log(Boolean(NaN));

// All outputs => false

console.log("-----");

// Truthy Values
console.log(Boolean(2));
console.log(Boolean("hi"));
console.log(Boolean("-1"));
console.log(Boolean({}));
console.log(Boolean([]));

// All outputs => true

// =====
// 🔥 IMPORTANT INTERVIEW POINT
// =====

// Falsy values in JavaScript:
// false
// 0
// ""
// null
// undefined
// NaN

// Everything else is mostly truthy.

```

```
// =====  
// ◆ NULL COERCION  
// =====  
  
console.log(null + 10);  
// Output: 10  
// null converts to 0 in numeric operation  
  
// =====  
// ◆ COERCION WITH COMPARISON OPERATORS  
// =====  
  
// == -> compares only values  
// === -> compares value + datatype  
  
console.log("5" == 5);  
// true  
// "5" converted to number 5  
  
console.log(0 == false);  
// true  
// false converted to 0  
  
console.log("" == false);  
// true  
// both become 0 internally  
  
console.log(null == 0);  
// false  
// null only loosely equals undefined  
  
console.log(null == undefined);  
// true  
  
console.log(null === undefined);  
// false  
// datatype also checked  
  
// =====  
// 🔥 IMPORTANT POINT  
// =====  
  
// Always prefer ===  
// because it checks:  
// ✓ value  
// ✓ datatype
```

```
// =====  
// ◆ EXPLICIT TYPE CONVERSION (MANUAL)  
// =====  
  
// Developer manually converts datatype using functions.  
  
// =====  
// ◆ CONVERT TO STRING  
// =====  
  
console.log(String(123));  
// "123"  
  
console.log(String(3.14));  
// "3.14"  
  
console.log(String(true));  
// "true"  
  
let age = 25;  
  
console.log(age.toString());  
// "25"  
  
// Extra Examples  
// console.log((255).toString(16)); // FF (Hexadecimal)  
// console.log((8).toString(2)); // 1000 (Binary)  
  
// =====  
// ◆ CONVERT TO NUMBER  
// =====  
  
let number = "42";  
  
console.log(typeof number);  
// string  
  
let No = Number("42");  
  
console.log(typeof No);  
// number
```

```
console.log(Number(""));
// 0

console.log(Number("1+2"));
// NaN
// Invalid numeric string

console.log(Number("Hi"));
// NaN

console.log(Number(true));
// 1

console.log(Number(false));
// 0

// =====
// ♦ parseInt()
// =====

// Used to extract integer values

console.log(parseInt(3.14));
// 3
// Decimal removed

console.log(parseInt("Abc"));
// NaN

console.log(parseInt("45.5px"));
// 45

// =====
// ♦ parseFloat()
// =====

// Used for decimal numbers

console.log(parseFloat("3.14test"));
// 3.14

console.log(parseFloat("1.5"));
// 1.5

// =====
// 🔥 IMPORTANT POINTS
// =====
```

```
// ✓ JavaScript is dynamically typed language
// ✓ JavaScript automatically converts datatypes
// ✓ "+" operator prefers string concatenation
// ✓ Other arithmetic operators prefer number conversion
// ✓ Use === instead of ==
// ✓ NaN means "Not a Number"
// ✓ typeof NaN => number
```


4.JavaScript Conditional Statements Notes

Types of Conditional Statements

- 1.if
- 2.if...else
- 3.else if
- 4.switch
- 5.ternary operator

1. if Statement

Definition

The if statement executes a block of code only when the condition is true. If the condition is false, the block is skipped.

Syntax

```
if(condition){  
    // code executes when condition is true  
}
```

Flow

- Condition → true → execute block
- Condition → false → skip block

Example

```
let marks = 70;  
  
if (marks >= 50) {  
    console.log("PASS");  
}
```

Important Points

- Executes only when condition is true
- Curly braces {} define the block
- Code outside block always executes
- Prefer === for comparison

- Avoid semicolon after if

2. if...else Statement

Definition

The if...else statement provides two execution paths.

- if block runs when condition is true
- else block runs when condition is false

Syntax

```
if(condition){  
    // runs when condition is true  
}  
else{  
    // runs when condition is false  
}
```

Flow

- Condition true → if block executes
- Condition false → else block executes

Example

```
let age = 16;  
  
if (age >= 18) {  
    console.log("Eligible to vote");  
}  
else {  
    console.log("Not eligible to vote");  
}
```

Important Points

- Only one block executes at a time
- Used when there are two possibilities
- else block is optional
- Helps in decision making

3. if...else if...else Ladder

Definition

Used when there are multiple conditions.

JavaScript checks conditions from top to bottom.

The first matching condition executes and remaining conditions are skipped.

Syntax

```
if(condition1){  
    // code  
}  
else if(condition2){  
    // code  
}  
else if(condition3){  
    // code  
}  
else{  
    // code  
}
```

Flow

- Checks conditions one by one
- Executes first true condition
- Remaining conditions are skipped
- else runs when no condition matches

Example

```
let marks = 75;  
  
if (marks >= 90) {  
    console.log("Grade A+");  
}  
else if (marks >= 80) {  
    console.log("Grade A");  
}  
else if (marks >= 70) {  
    console.log("Grade B");  
}
```

```
else {
    console.log("Failed");
}
```

Important Points

- Used for multiple possibilities
- Order of conditions is important
- Only first matching block executes
- else block is optional

Program 1:

```
// =====
// ♦ Types of Conditional Statements in JavaScript
// =====

// 1. if
// 2. if...else
// 3. else if
// 4. switch
// 5. ternary operator

// =====
// ♦ IF Statement
// =====

// The if statement executes a block of code
// only when the condition is TRUE.

// If the condition is FALSE,
// the block is skipped completely.

/*
Syntax:

if(condition){
    // code executes only when condition is true
}

Condition Result:
true -> enter block -> execute code
false -> skip block -> continue next line
*/

// =====
// ♦ Example 1 : Student Marks Validation
```

```
// =====

let marks = 40;

// Check whether marks are greater than or equal to 50
if (marks >= 50) {

    // This block runs only if condition is true
    console.log("Congratulations! You PASSED");
}

// This line always executes
console.log("Exam result checked");

// =====
// ♦ Dry Run
// =====

// Case 1:
// marks = 70
// 70 >= 50 → true

// Output:
// Congratulations! You PASSED
// Exam result checked

// Case 2:
// marks = 40
// 40 >= 50 → false

// Output:
// Exam result checked

// =====
// ♦ Example 2 : Stock Availability Check
// =====

let stockCount = 0;

// Check whether stock is available
if (stockCount > 0) {

    // Runs only when stock is greater than 0
    console.log("Item is available");
    console.log("You can add it to cart");
}

// This line always executes
console.log("Program completed");

// =====
```

```
// ♦ Dry Run
// =====

// stockCount = 0
// 0 > 0 → false

// Output:
// Program completed

// =====
// ♦ Example 3 : Discount Eligibility
// =====

let orderValue = 1500;

// Check whether order value is eligible for discount
if (orderValue >= 1000) {

    // Executes because condition is true
    console.log("You qualify for 10% discount");
}

// =====
// ♦ Dry Run
// =====

// orderValue = 1500
// 1500 >= 1000 → true

// Output:
// You qualify for 10% discount

// =====
// ♦ Common Mistake 1 : Using = instead of == or ===
// =====

/*

let x = 100;

if (x = 10) {

    console.log("Wrong condition");
}

*/

// Problem:
// = is assignment operator
```

```
// It assigns value 10 to x

// Correct operators:
// == → value comparison
// === → value + datatype comparison

// =====
// ♦ Common Mistake 2 : Semicolon after if
// =====

/*

let score = 0;

if (score >= 50); {
    console.log("This always runs");
}

*/

// Problem:
// Semicolon ends the if statement immediately

// This creates an empty if block

// So the code inside {} runs always

// Important Point:
// ✗ semi-colon kills the if condition

// =====
// ♦ Correct Version
// =====

let score = 0;

if (score >= 50) {
    console.log("Passed");
}

console.log("Program Ended");

// =====
// ♦ Important Interview Points
// =====

// ✓ if executes only when condition is true
// ✓ Curly braces {} define the block of code
```

```
// ✓ Code outside the if block always executes
// ✓ Avoid semicolon after if condition
// ✓ Prefer === for comparisons

// =====
// ♦ Real-Time Example
// =====

let isLoggedIn = true;

// Check whether user is logged in
if (isLoggedIn === true) {
    console.log("Welcome to Dashboard");
}

console.log("Website loaded");

// =====
// ♦ Output
// =====

// Welcome to Dashboard
// Website loaded
```

Program 2

```
// =====
// ♦ if...else Statement in JavaScript
// =====

// The if...else statement provides TWO paths:
// 1. One path runs when condition is TRUE
// 2. Another path runs when condition is FALSE

/*
Syntax:

if(condition){
    // Runs when condition is TRUE
}
else{
    // Runs when condition is FALSE
}

Condition Result:
TRUE → if block executes
FALSE → else block executes
*/
```



```

console.log("=====");

// =====
// ♦ Example 1 : Pass or Fail Result
// =====

let marks = 45;

// Check whether student passed or failed
if (marks >= 50) {
    // Executes when condition is true
    console.log("Result : PASS");
}
else {
    // Executes when condition is false
    console.log("Result : FAIL");
}

// This line always executes
console.log("Program completed");

// =====
// ♦ Dry Run
// =====

// marks = 45
// 45 >= 50 → false

// Output:
// Result : FAIL
// Program completed

// =====
// ♦ Example 2 : Pass Case
// =====

marks = 55;

// Check whether student passed
if (marks >= 50) {
    console.log("Result : PASS");
}
else {
    console.log("Result : FAIL");
}

console.log("Program completed");

// =====
// ♦ Dry Run

```

```
// =====  
  
// marks = 55  
// 55 >= 50 → true  
  
// Output:  
// Result : PASS  
// Program completed  
  
// =====  
// ♦ Example 3 : Even or Odd Number  
// =====  
  
let number = 7;  
  
// % operator returns remainder  
// If remainder is 0 → Even Number  
// Otherwise → Odd Number  
  
if (number % 2 === 0) {  
    console.log(number + " is Even number");  
}  
else {  
    console.log(number + " is Odd number");  
}  
  
// =====  
// ♦ Dry Run  
// =====  
  
// number = 7  
// 7 % 2 = 1  
// 1 === 0 → false  
  
// Output:  
// 7 is Odd number  
  
// =====  
// ♦ Example 4 : Even Number  
// =====  
  
number = 20;  
  
if (number % 2 === 0) {  
    console.log(number + " is Even number");  
}  
else {  
    console.log(number + " is Odd number");  
}
```

```

// =====
// ♦ Dry Run
// =====

// number = 20
// 20 % 2 = 0
// 0 === 0 → true

// Output:
// 20 is Even number

// =====
// ♦ Example 5 : Password Validation
// =====

let password = "Correct123";
let input = "wrong456";

// Check whether entered password matches original password
if (password === input) {
    console.log("Login successful!");
    console.log("Redirecting to dashboard");
}
else {
    console.log("Incorrect password!");
    console.log("Please try again");
}

// =====
// ♦ Dry Run
// =====

// password = Correct123
// input = wrong456

// "Correct123" === "wrong456" → false

// Output:
// Incorrect password!
// Please try again

// =====
// ♦ Example 6 : Correct Password
// =====

password = "Correct123";
input = "Correct123";

// Password comparison
if (password === input) {

```

```

        console.log("Login successful!");
        console.log("Redirecting to dashboard");
    }
    else {
        console.log("Incorrect password!");
        console.log("Please try again");
    }

    // =====
    // ♦ Dry Run
    // =====

    // password = Correct123
    // input = Correct123

    // "Correct123" === "Correct123" → true

    // Output:
    // Login successful!
    // Redirecting to dashboard

    // =====
    // ♦ Example 7 : Age Validation
    // =====

    let age = 16;

    // Check whether user is adult
    if (age >= 18) {
        console.log("Access Granted – You are an adult");
    }
    else {
        console.log("Access denied – You must be 18 or older");
    }

    console.log("PC");

    // =====
    // ♦ Dry Run
    // =====

    // age = 16
    // 16 >= 18 → false

    // Output:
    // Access denied – You must be 18 or older
    // PC

    // =====
    // ♦ Example 8 : Discount Calculation

```

```
// =====

let orderAmount = 1200;

// Apply discount only if order amount is ₹1000 or more
if (orderAmount >= 1000) {

    // Calculate 10% discount
    let discount = orderAmount * 0.10;

    // Final amount after discount
    let finalAmount = orderAmount - discount;

    console.log("Discount applied :", discount);

    console.log("Final amount :", finalAmount);
}
else {
    console.log("No discount for order below ₹1000");
    console.log("Add more items to unlock discount");
}

// =====
// ♦ Dry Run
// =====

// orderAmount = 1200
// 1200 >= 1000 → true

// discount = 1200 * 0.10 = 120
// finalAmount = 1200 - 120 = 1080

// Output:
// Discount applied : 120
// Final amount : 1080

// =====
// ♦ Example 9 : No Discount Case
// =====

orderAmount = 800;

// Check eligibility for discount
if (orderAmount >= 1000) {
    let discount = orderAmount * 0.10;
    let finalAmount = orderAmount - discount;

    console.log("Discount applied :", discount);
    console.log("Final amount :", finalAmount);
}
else {
```

```

    console.log("No discount for order below ₹1000");
    console.log("Add more items to unlock discount");
}

// =====
// ♦ Dry Run
// =====

// orderAmount = 800
// 800 >= 1000 → false

// Output:
// No discount for order below ₹1000
// Add more items to unlock discount

// =====
// ♦ Important Interview Points
// =====

// ✓ if block runs only when condition is TRUE
// ✓ else block runs only when condition is FALSE
// ✓ Only ONE block executes at a time
// ✓ Use === for safer comparison
// ✓ Curly braces {} define block scope

// =====
// ♦ Real-Time Example
// =====

let isLoggedIn = false;

// Check login status
if (isLoggedIn === true) {
    console.log("Welcome User");
}
else {
    console.log("Please Login First");
}

// =====
// ♦ Output
// =====

// Please Login First

```

Program 3

```

// =====

```

```
// ♦ if...else if...else Ladder in JavaScript
// =====

// Used when you have MORE THAN TWO possibilities.

// JavaScript checks each condition from TOP TO BOTTOM.

// The FIRST matching condition block executes.

// Once a match is found,
// remaining conditions are skipped completely.

/*
Syntax:

if(condition1){
    // Runs if condition1 is true
}
else if(condition2){
    // Runs when condition1 is false
    // AND condition2 is true
}
else if(condition3){
    // Runs when condition1 and condition2 are false
    // AND condition3 is true
}
else{
    // Runs when NONE of the above conditions are true
}
*/

// =====
// ♦ Example : Student Grade System
// =====

let marks = 30;

// Check marks and assign grade
if (marks >= 90) { // 30 >= 90 → false
    console.log("Grade : A+");
}
else if (marks >= 80) { // 30 >= 80 → false
    console.log("Grade : A");
}
else if (marks >= 70) { // 30 >= 70 → false
    console.log("Grade : B");
}
else if (marks >= 60) { // 30 >= 60 → false
    console.log("Grade : C");
}
else if (marks >= 50) { // 30 >= 50 → false
```

```

        console.log("Grade : D");
    }
    else {
        // Executes because all above conditions are false
        console.log("Grade : F - Failed");
    }

    // =====
    // ♦ Dry Run
    // =====

    // marks = 30

    // 30 >= 90 → false
    // 30 >= 80 → false
    // 30 >= 70 → false
    // 30 >= 60 → false
    // 30 >= 50 → false

    // None of the conditions matched

    // Output:
    // Grade : F - Failed

    // =====
    // ♦ Example 2 : Grade B
    // =====

    marks = 75;

    // Check marks and assign grade
    if (marks >= 90) { // false
        console.log("Grade : A+");
    }
    else if (marks >= 80) { // false
        console.log("Grade : A");
    }
    else if (marks >= 70) { // true
        console.log("Grade : B");
    }
    else if (marks >= 60) {
        console.log("Grade : C");
    }
    else if (marks >= 50) {
        console.log("Grade : D");
    }
    else {
        console.log("Grade : F - Failed");
    }

    // =====

```



```
// ♦ Dry Run
// =====

// marks = 75

// 75 >= 90 → false
// 75 >= 80 → false
// 75 >= 70 → true

// Once true is found,
// remaining conditions are skipped

// Output:
// Grade : B

// =====
// ♦ Important Interview Points
// =====

// ✓ Used for multiple conditions
// ✓ Conditions are checked from top to bottom
// ✓ Only FIRST matching block executes
// ✓ Remaining conditions are skipped
// ✓ else block is optional
// ✓ Order of conditions is very important
```

Program 1

//Conditional Statements in JavaScript

//1. if statement

```
let test = 50;

if (test >= 50) { //true
    console.log("Good morning");
}

console.log("Hi");

test = 40;

if (test >= 50) { //false
    console.log("Good morning");
}
```

```
console.log("Hi");
```

```
//2. if else statement
```

```
if (test >= 50) { //false
    console.log("GM");
}
else {
    console.log("GE");
}
```

```
console.log("Hi");
```

```
//3. if else if else ladder
```

```
/*
Syntax
```

```
if(condition1){
    //Runs if condition1 is true
}
else if(condition2){
    //Runs when condition1 is false
    //AND condition2 is true
}
else if(condition3){
    //Runs when condition1 and condition2 are false
    //AND condition3 is true
}
else{
    //Runs when none of the above conditions are true
}
*/
```

```
//BMI Calculator
```

```
let bmi = 27.5;

if (bmi < 18.5) {
    console.log("Category : Underweight");
}
else if (bmi < 25) {
    console.log("Category : Normal weight");
}
else if (bmi < 30) {
    console.log("Category : Overweight");
}
else {
    console.log("Category : Obese");
}
```

```
}
```

```
bmi = 17.5;
```

```
if (bmi < 18.5) {  
    console.log("Category : Underweight");  
}  
else if (bmi < 25) {  
    console.log("Category : Normal weight");  
}  
else if (bmi < 30) {  
    console.log("Category : Overweight");  
}  
else {  
    console.log("Category : Obese");  
}
```

```
//Internet Speed Example
```

```
let speed = 40;
```

```
if (speed >= 100) {  
    console.log("Speed : Ultra Fast");  
}  
else if (speed >= 50) {  
    console.log("Speed : Fast");  
}  
else if (speed >= 20) {  
    console.log("Speed : Moderate");  
}  
else if (speed >= 5) {  
    console.log("Speed : Slow");  
}  
else {  
    console.log("Speed : Very slow – check connection");  
}
```

```
speed = 3;
```

```
if (speed >= 100) {  
    console.log("Speed : Ultra Fast");  
}  
else if (speed >= 50) {  
    console.log("Speed : Fast");  
}  
else if (speed >= 20) {  
    console.log("Speed : Moderate");  
}  
else if (speed >= 5) {
```

```
    console.log("Speed : Slow");
}
else {
    console.log("Speed : Very slow – check connection");
}
```

//Time of Day Greeting

```
let hour = 22; //24-hour format

if (hour >= 5 && hour < 12) {
    console.log("Good morning");
}
else if (hour >= 12 && hour < 17) {
    console.log("Good afternoon");
}
else if (hour >= 17 && hour < 21) {
    console.log("Good evening");
}
else {
    console.log("Good night");
}
```

//E-commerce Shipping Example

```
let orderValue = 750;
let isPremium = false;

if (isPremium) {
    console.log("Shipping : Free (Premium member)");
}
else if (orderValue >= 1000) {
    console.log("Shipping : Free (Order above ₹1000)");
}
else if (orderValue >= 500) {
    console.log("Shipping : ₹99 (Standard)");
}
else {
    console.log("Shipping : Express only");
}
```

//Nested if statement
//An if inside another if

/*
Syntax

```
if(outerCondition){
    if(innerCondition){
```

```
        //Inner block
    }
}
*/
```

//Voting Eligibility Example

```
let age = 22;
let isCitizen = true;

if (age >= 18) {
    console.log("Age requirement met");

    if (isCitizen) {
        console.log("You are eligible to vote");
    }
    else {
        console.log("Only citizens can vote");
    }
}
else {
    console.log("You must be 18 or older to vote");
}
```

```
age = 22;
isCitizen = false;

if (age >= 18) {
    console.log("Age requirement met");

    if (isCitizen) {
        console.log("You are eligible to vote");
    }
    else {
        console.log("Only citizens can vote");
    }
}
else {
    console.log("You must be 18 or older to vote");
}
```

```
age = 16;
isCitizen = true;

if (age >= 18) {
    console.log("Age requirement met");

    if (isCitizen) {
        console.log("You are eligible to vote");
    }
}
```

```
    }  
    else {  
        console.log("Only citizens can vote");  
    }  
}  
else {  
    console.log("You must be 18 or older to vote");  
}
```

//Login + Role Check Example

```
let isLoggedIn = true;  
let userRole = "admin";  
  
if (isLoggedIn) {  
    console.log("User is logged in");  
  
    if (userRole === "admin") {  
        console.log("Access : Full Admin Panel");  
    }  
    else if (userRole === "editor") {  
        console.log("Access : Edit content only");  
    }  
    else {  
        console.log("Access : View only");  
    }  
}  
else {  
    console.log("Please login to continue");  
}
```

```
isLoggedIn = true;  
userRole = "editor";  
  
if (isLoggedIn) {  
    console.log("User is logged in");  
  
    if (userRole === "admin") {  
        console.log("Access : Full Admin Panel");  
    }  
    else if (userRole === "editor") {  
        console.log("Access : Edit content only");  
    }  
    else {  
        console.log("Access : View only");  
    }  
}  
else {  
    console.log("Please login to continue");  
}
```

```
isLoggedIn = false;
userRole = "editor";

if (isLoggedIn) {
    console.log("User is logged in");

    if (userRole === "admin") {
        console.log("Access : Full Admin Panel");
    }
    else if (userRole === "editor") {
        console.log("Access : Edit content only");
    }
    else {
        console.log("Access : View only");
    }
}
else {
    console.log("Please login to continue");
}

//Bank ATM Example

let hasCard = true;
let pinCorrect = true;
let balance = 5000;
let withdrawal = 10000;

if (hasCard) {
    if (pinCorrect) {
        if (withdrawal <= balance) {
            let newBalance = balance - withdrawal;
            console.log("Dispensing ₹ : " + withdrawal);
            console.log("Remaining Balance : " + newBalance);
        }
        else {
            console.log("Insufficient Balance");
        }
    }
    else {
        console.log("Incorrect PIN : Try Again");
    }
}
else {
    console.log("Please insert your card");
}
```

```
}
```

```
//Important Points
```

```
//if → Executes block when condition is true  
//if else → Executes one block from two options  
//if else if → Used for multiple conditions  
//Nested if → if inside another if
```

```
//&& → AND operator  
//|| → OR operator
```

```
//true && true → true  
//true && false → false
```

```
//true || false → true  
//false || false → false
```


Program 2

//Switch Statement in JavaScript

//Switch compares one value against multiple fixed options (cases)

/*

Syntax

```
switch(expression){  
    case value1:  
        //Code  
        break;  
  
    case value2:  
        //Code  
        break;  
  
    default:  
        //Runs if no case matches  
}  
*/
```

//Example 1 – Day of Week

```
let day = 5;
```

```
switch (day) {  
    case 1:  
        console.log("Monday");  
        break;  
  
    case 2:  
        console.log("Tuesday");  
        break;  
  
    case 3:  
        console.log("Wednesday");  
        break;  
  
    case 4:  
        console.log("Thursday");  
        break;  
  
    case 5:  
        console.log("Friday");  
        break;  
  
    case 6:
```

```
        console.log("Saturday");
        break;

    case 7:
        console.log("Sunday");
        break;

    default:
        console.log("Invalid day number");
}
```

//Invalid day example

```
day = 15;

switch (day) {

    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    case 4:
        console.log("Thursday");
        break;

    case 5:
        console.log("Friday");
        break;

    case 6:
        console.log("Saturday");
        break;

    case 7:
        console.log("Sunday");
        break;

    default:
        console.log("Invalid day number");
}
```

//Role-based Access Example

```
let role = "admin";

switch (role) {

  case "admin":
    console.log("Full access to all features");
    break;

  case "Editor":
    console.log("Can create and edit content");
    break;

  case "viewer":
    console.log("Read only access");
    break;

  default:
    console.log("Unknown Role");
}
```

//Case-sensitive example

```
role = "Editor";

switch (role) {

  case "Admin":
    console.log("Full access to all features");
    //No break

  case "Editor":
    console.log("Can create and edit content");
    break;

  case "viewer":
    console.log("Read only access");
    break;

  default:
    console.log("Unknown Role");
}
```

//Calculator Example

```
let num1 = 20;
let num2 = 5;
let operation = "*";

switch (operation) {
```

```
case "+":
    console.log("Result :", num1 + num2);
    break;

case "-":
    console.log("Result :", num1 - num2);
    break;

case "*":
    console.log("Result :", num1 * num2);
    break;

case "/":
    console.log("Result :", num1 / num2);
    break;

case "%":
    console.log("Result :", num1 % num2);
    break;

default:
    console.log("Unknown operation");
}
```

//Invalid operation example

```
num1 = 20;
num2 = 5;
operation = "^";

switch (operation) {

    case "+":
        console.log("Result :", num1 + num2);
        break;

    case "-":
        console.log("Result :", num1 - num2);
        break;

    case "*":
        console.log("Result :", num1 * num2);
        break;

    case "/":
        console.log("Result :", num1 / num2);
        break;

    default:
        console.log("Unknown operation");
}
```

```
}
```

```
//Important Points
```

```
//Switch uses strict equality (===)  
//Used for fixed values only  
//Cannot use >, <, && inside switch cases
```

```
/*  
switch(value){  
    case 10:  
}  
*/
```

```
//For complex conditions use if else
```

```
//Ternary Operator
```

```
//Compact way to write simple if else
```

```
//Syntax  
//condition ? trueValue : falseValue
```

```
let age = 20;  
let access;  
  
if (age >= 18) {  
    access = "Allowed";  
}  
else {  
    access = "Denied";  
}
```

```
console.log(access);
```

```
//Using ternary operator
```

```
let result = age >= 18 ? "Allowed" : "Denied";  
console.log(result);
```

```
//PASS / FAIL Example
```

```
let marks = 72;  
  
let result1 = marks >= 35 ? "PASS" : "FAIL";
```

```
console.log(result1);

marks = 32;

let result2 = marks >= 35 ? "PASS" : "FAIL";

console.log(result2);
```

//Chained Ternary Operator

```
let score = 68;

let grade = score >= 90 ? "A+"
  : score >= 80 ? "A"
    : score >= 70 ? "B"
      : score >= 60 ? "C"
        : score >= 50 ? "D"
          : "F";

console.log("Grade :", grade);
```

```
score = 5;

grade = score >= 90 ? "A+"
  : score >= 80 ? "A"
    : score >= 70 ? "B"
      : score >= 60 ? "C"
        : score >= 50 ? "D"
          : "F";

console.log("Grade :", grade);
```

//Truthy and Falsy Example

```
let x = ""; //Falsy value

if (x) {
  console.log("True");
}
else {
  console.log("False");
}
```

//Find Largest of Two Numbers

```
let a = 45;
let b = 72;
```

```
if (a > b) {  
    console.log("Largest number is :", a);  
}  
else if (b > a) {  
    console.log("Largest number is :", b);  
}  
else {  
    console.log("Both numbers are equal");  
}
```

//Find Largest of Three Numbers

```
x = 30;
```

```
let y = 85;  
let z = 600;
```

```
if (x >= y && x >= z) {  
    console.log("Largest is :", x);  
}  
else if (y >= x && y >= z) {  
    console.log("Largest is :", y);  
}  
else {  
    console.log("Largest is :", z);  
}
```

//Important Interview Points

```
//break prevents fall-through  
//default runs when no case matches  
//switch is case-sensitive
```

```
//Truthy values  
//"Hello", 1, [], {}, true
```

```
//Falsy values  
//false, 0, "", null, undefined, NaN
```

8. Control Statements in JavaScript - Looping Statements

What are Loops?

- A loop is a control statement that *repeats a block of code* multiple times until a condition becomes false.
- Instead of writing the same code 100 times, we write it once inside a loop.

Why Loops are Important?

- Reduce code duplication → write once, execute many times
- Automate repetitive tasks → iterate test data, retry failed tests
- Process collections → loop through arrays, tables, dropdowns
- Essential in Automation → iterating test cases, data-driven testing, retrying

Advantages of Loops

Advantage	Explanation
Code Reusability	Same code runs multiple times
Less Code	No need to repeat statements
Easy Maintenance	Change logic in one place
Dynamic Execution	Loop count can change at runtime
Data Processing	Process arrays, objects, API responses

Real-Time Examples

Scenario	Loop Usage
ATM PIN retry	Loop 3 times for wrong PIN
Automation	Loop through test data rows
API Testing	Loop through response array

Types of Loops in JavaScript

1. for loop → when count is known
2. while loop → when count is unknown, condition-based
3. do...while loop → execute at least once, then check condition
4. for...of → iterate over iterable values (arrays, strings)
5. for...in → iterate over object keys
6. forEach() → array method for iteration

1. for Loop

Syntax

```
``javascript
for(initialization; condition; increment/decrement){
    // code to execute repeatedly
}
```

Program1

//for Loop in JavaScript

```
const { promptLoop } = require("readline-sync");
```

```
//Without loop
//Same statement repeated multiple times
```

```
console.log("Hi");
console.log("Hi");
console.log("Hi");
console.log("Hi");
console.log("Hi");
```

```
/*
for loop
```

Syntax

```
for(initialization; condition; increment/decrement){
    //Code to execute repeatedly
}
```

Flow of execution

Step 1 → Initialization runs once

Step 2 → Condition check
 true → execute loop body
 false → exit loop

Step 3 → Increment/decrement

Step 4 → Again check condition

*/

```
//Print numbers from 1 to 5
```

```
//Starting number = 1
```

```
//Ending number = 5
```

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

```
/*
```

```
Iteration Table
```

Iteration	i value	Condition	Output	After i++
1st	1	1<=5 true	1	2
2nd	2	2<=5 true	2	3
3rd	3	3<=5 true	3	4
4th	4	4<=5 true	4	5
5th	5	5<=5 true	5	6
6th	6	6<=5 false	EXIT	

```
*/
```

```
//Print numbers from 10 to 20
```

```
for (let i = 10; i <= 20; i++) {
    console.log(i);
}
```

```
//Print numbers from 50 to 25 in reverse order
```

```
for (let i = 50; i >= 25; i--) {
    console.log(i);
}
```

```
console.log("Loop completed");
```

```
//Print square of numbers
```

```
/*
```

```
Number 1 : Square 1
```

```
Number 2 : Square 4
```

```
Number 3 : Square 9
```

```
*/
```

```
for (let i = 1; i <= 3; i++) {
    console.log(`Number : ${i} : Square : ${i * i}`);
}
```

```
//Print table of 2
```

```
/*  
2 x 1 = 2  
2 x 2 = 4  
2 x 10 = 20  
*/  
  
for (let i = 1; i <= 10; i++) {  
    console.log(`2 x ${i} = ${2 * i}`);  
}
```

```
//Print table of 5
```

```
for (let i = 1; i <= 10; i++) {  
    console.log(`5 x ${i} = ${5 * i}`);  
}
```

```
//Print numbers divisible by 3 from 3 to 30
```

```
/*  
3  
6  
9  
12  
...  
30  
*/  
  
for (let i = 3; i <= 30; i += 3) {  
    console.log(i);  
}
```

```
//Important Interview Points
```

```
//Initialization runs only once  
//Condition is checked before every iteration  
//Increment/decrement changes loop variable  
//for loop is best when iteration count is known
```

```
//Infinite loop example
```

```
/*  
for(;;){  
    console.log("Infinite Loop");  
}  
*/
```

```
//Real-time Automation Examples

//Loop through dropdown values
//Loop through test data
//Retry failed tests
//Iterate product list
//Validate multiple rows in web table
```

Program 2

```
// =====
// JavaScript For Loop Examples
// =====

// for loop is used to execute a block of code multiple times.

// Syntax:
// for(initialization ; condition ; increment/decrement){
//     code
// }

// =====
// Example 1 : Addition of First 5 Numbers
// =====

// 1 + 2 + 3 + 4 + 5 = 15

let total = 0;
for (let i = 1; i <= 5; i++) {
    total = total + i;

    // total += i;

    // console.log(`After adding ${i} : total = ${total}`);
}

console.log("Addition of first 5 numbers is : " + total);

// =====
// Example 2 : String Index Access
// =====

let word = "Hello";

console.log(word[0]);
```

```

for (let i = 0; i < word.length; i++) {
    console.log(`Index ${i} : ${word[i]}`);

    console.log("Index " + i + " : " + word[i]);
}

```

```

// =====
// Example 3 : Print Numbers with Step of 5
// =====

```

```

// 5 to 50 with step of 5

```

```

for (let i = 5; i <= 50; i += 5) {
    // i = i + 5

    console.log(i);
}

```

```

// =====
// Example 4 : Print Even Numbers
// =====

```

```

// 2 4 6 8 10 ... 20

```

```

for (let i = 2; i <= 20; i += 2) {
    console.log(i);
}

```

```

// =====
// Example 5 : Print Odd Numbers
// =====

```

```

// 1 3 5 7 9 ... 19

```

```

for (let i = 1; i <= 20; i += 2) {
    console.log(i);
}

```

```

// =====
// Example 6 : Factorial of a Number
// =====

```

```

// 5! = 5 * 4 * 3 * 2 * 1 = 120

```

```

let n = 6;

```

```

let factorial = 1;

```

```

for (let i = 1; i <= n; i++) {
    factorial = factorial * i;

    // factorial *= i;
}

console.log(`Factorial of ${n} is ${factorial}`);

// =====
// Example 7 : Count Vowels in String
// =====

// Automation Testing

let text = "Automation Testing";
console.log(text.length);

let vowels = 'aeiouAEIOU';
let count = 0;

for (let i = 0; i < text.length; i++) {
    if (vowels.includes(text[i])) {
        count++;

        // console.log(text[i]);
    }
}

console.log(`Vowels in ${text} : ${count}`);

// =====
// Example 8 : Find Largest Number in Array
// =====

const prices = [9999, 14199];

// Assume first element is largest

let largest = prices[0];

for (let i = 1; i < prices.length; i++) {
    // Compare each element

    if (prices[i] > largest) {

        // Update largest number
    }
}

```

```

        largest = prices[i];
    }
}

console.log("Largest price : " + largest);

// =====
// Important Points About for Loop
// =====

/*

initialization
Executed only once

condition
Loop runs until condition becomes false

increment/decrement
Updates loop variable

i++
Increase by 1

i--
Decrease by 1

i += 5
Increase by 5

*/

// =====
// Interview Questions
// =====

/*

Q1. What is for loop?
Ans:
Used to execute code repeatedly.

Q2. Syntax of for loop?
Ans:
for(initialization ; condition ; increment/decrement)

Q3. Difference between i++ and i+=2 ?
Ans:
i++ increases by 1
i+=2 increases by 2

```

```
*/
```

Program 3

//break and continue statements in JavaScript

//Infinite loop examples

```
/*  
for (let i = 1; ; i++) { //No condition  
    console.log(i);  
}
```

```
for (let i = 1; i > 0; i++) {  
    console.log(i);  
}
```

```
for (let i = 1; true; i++) {  
    console.log(i);  
}  
*/
```

//break statement

//Immediately exits the current loop or switch block

```
/*  
Syntax
```

```
for(...){  
    if(condition){  
        break;  
    }  
}  
*/
```

//Example 1

//Stop loop when number 5 is found

```
for (let i = 1; i <= 10; i++) {  
    if (i === 5) { //true  
        console.log(`Found ${i} : Breaking loop`);  
        break;  
    }  
  
    console.log(i);  
}
```



```
//Example 2
//Stop execution when first FAIL is found

let result = ["PASS", "PASS", "FAIL", "PASS", "PASS"];

for (let i = 0; i < result.length; i++) {
  console.log(`TC${i + 1}: ${result[i]}`);

  if (result[i] === "FAIL") {
    console.log(`First failure found at TC${i + 1}. Stopping
execution`);
    break;
  }
}
```

```
//continue statement
//Skips current iteration and moves to next iteration
```

```
/*
Syntax
```

```
for(...){
  if(condition){
    continue;
  }

  //Skipped when continue runs
}
*/
```

```
//Example 1
//Print only odd numbers from 1 to 10

for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) { //even number
    continue;
  }

  console.log(i); //only odd numbers reach here
}
```

```
console.log("=====");
```

```
//Example 2
//Skip failed test cases
```

```
let result1 = ["PASS", "FAIL", "FAIL", "PASS", "PASS"];
```

```
let passCount = 0;

for (let i = 0; i < result1.length; i++) {
  console.log(`TC${i + 1}: ${result1[i]}`);

  if (result1[i] === "FAIL") {
    console.log(`TC${i + 1}: Skipped failed test`);
    continue;
  }

  //Only PASS test reaches here

  console.log(`TC${i + 1}: Processing passed test`);

  passCount++;
}

console.log("Total passed: " + passCount);

//Important Interview Points

//break → completely exits loop
//continue → skips current iteration only

//break stops execution immediately
//continue jumps to next iteration

//Used heavily in automation
//break → stop execution on critical failure
//continue → skip invalid data/test case

//Real-time Automation Examples

//Retry mechanism
//Skip failed API responses
//Stop test execution on blocker issue
//Process only PASS test cases
```

2. While Loop

Program 4

//while Loop in JavaScript

//Syntax

```
/*
initialization

while(condition){
    //Code to execute while condition is true
    increment/decrement
}
```

Flow

Step 1 → Check condition

true → execute body → increment/decrement → again check condition

false → exit loop

*/

//Print 1 to 5 using while loop

```
let i = 1; //Initialization outside loop
```

```
while (i <= 5) { //condition check
    console.log(i);
    i++; //increment inside loop
}
```

```
console.log("While loop completed");
```

//Print 5 to 10

```
let num = 5;
```

```
while (num <= 10) {
    console.log(num);
    num++;
}
```

//Print even numbers from 2 to 20

```
let even = 2;
```

```
while (even <= 20) {
    console.log(even);
    even += 2; //increase by 2
}

//Print table of 5

/*
5 X 1 = 5
5 X 2 = 10
*/

let t = 1;

console.log("----- Table of 5 -----");

while (t <= 10) {
    console.log(`5 X ${t} = ${5 * t}`);
    t++;
}

//Print table of 9

let w = 1;

console.log("----- Table of 9 -----");

while (w <= 10) {
    console.log(`9 X ${w} = ${9 * w}`);
    w++;
}

//Sum of numbers from 1 to 10

let idx = 1;
let sum = 0;

while (idx <= 10) {
    sum += idx; //sum = sum + idx
    idx++;
}

console.log("Sum of 1 to 10 :", sum);

//Print numbers from 10 to 1

let countdown = 10;
```

```
while (countdown >= 1) {
  console.log(countdown);
  countdown--;
}
```

//Reverse a number

//12345 → 54321

```
let number = 12345;
let reverse = 0;
```

```
while (number > 0) {
  let digit = number % 10;
  reverse = reverse * 10 + digit;
  number = Math.floor(number / 10);
}
```

```
console.log("Reversed Number :", reverse);
```

//Important Interview Points

//while loop is used when iteration count is unknown

//Initialization happens before loop

//Increment/decrement happens inside loop

//Condition is checked before execution

//If condition is false initially → loop will not run

//Infinite while loop example

```
/*
while(true){
  console.log("Infinite Loop");
}
*/
```

//Difference between for and while

```
/*
for loop
-----
```

Used when iteration count is known

Initialization, condition, increment in one line

```
while loop
-----
Used when iteration count is unknown
Initialization outside loop
Increment inside loop
*/
```

```
//Real-time Automation Examples
```

```
//Retry login until success
//Wait until element visible
//Read dynamic API response
//Pagination handling
//Scroll until element found
```

While Loop

```
// =====
// WHILE LOOP
// =====
```

```
/*
SYNTAX:
```

```
while(condition){
    // code
}
```

FLOW:

1. Check condition
2. If condition is true → execute loop body
3. Again check condition
4. Repeat until condition becomes false

```
*/
```

```
// =====
// PROGRAM 1 : SUM OF DIGITS OF A NUMBER
// =====
```

```
// Example:
// 123 = 1 + 2 + 3 = 6
```

```
let num = 123;
```

```
// Variable to store final sum
let digitSum = 0;
```

```
// Store original number in temp variable
// because original value will change inside loop
```

```

let temp = num;

/*
LOGIC:

Step 1:
Get last digit using % 10

123 % 10 = 3
12 % 10 = 2
1 % 10 = 1

Step 2:
Add digit to digitSum

Step 3:
Remove last digit using Math.floor(num / 10)

Math.floor(123 / 10) = 12
Math.floor(12 / 10) = 1
Math.floor(1 / 10) = 0
*/

while (temp > 0) {

    // Extract last digit
    let digit = temp % 10;

    // Add digit to sum
    digitSum = digitSum + digit;

    // Remove last digit from number
    temp = Math.floor(temp / 10);
}

// Print final result
console.log(`Sum of digits of ${num} is ${digitSum}`);

// =====
// PROGRAM 2 : REVERSE A NUMBER
// =====

// Example:
// 5678 → 8765

let original = 5678;

// Variable to store reversed number

```

```
let revnum = 0;

// Copy original number into temp variable
let temp2 = original;

/*
LOGIC:

Step 1:
Get last digit using % 10

5678 % 10 = 8
567 % 10 = 7
56 % 10 = 6
5 % 10 = 5

Step 2:
Build reverse number

FORMULA:
revnum = revnum * 10 + digit

WHY revnum * 10 ?

To shift existing digits to left side

Example:

0 * 10 + 8 = 8
8 * 10 + 7 = 87
87 * 10 + 6 = 876
876 * 10 + 5 = 8765

Step 3:
Remove last digit from original number
*/

while (temp2 > 0) {

    // Extract last digit
    let digit = temp2 % 10;

    // Build reverse number
    revnum = revnum * 10 + digit;

    // Remove last digit
    temp2 = Math.floor(temp2 / 10);
}

// Print reversed number
console.log(`Reverse of ${original} is ${revnum}`);
```


3. Do While loop

```
// =====  
// DO WHILE LOOP  
// =====
```

```
/*  
SYNTAX:
```

```
do{  
    // code  
}  
while(condition);
```

```
IMPORTANT POINT:
```

```
-----  
In do while loop,  
the code block executes AT LEAST ONCE  
even if the condition is false.
```

```
FLOW:
```

1. Execute code
2. Check condition
3. If true → repeat
4. If false → exit loop

```
*/
```

```
// =====  
// PROGRAM 1 : PRINT 1 TO 5  
// =====
```

```
let j = 1;  
do {  
    console.log(j);  
    j++;  
}  
while (j <= 5);
```

```
/*  
OUTPUT:  
1
```

```
2
3
4
5
*
console.log("-----");
```

```
// =====
// PROGRAM 2 : PRINT 5 TO 20
// =====
```

```
let d1 = 5;
do {
    console.log(d1);
    d1++;
}
while (d1 <= 20);
/*
OUTPUT:
5
6
7
...
20
*/
console.log("-----");
```

```
// =====
// PROGRAM 3 : PRINT EVEN NUMBERS FROM 20 TO 40
// =====
```

```
let d2 = 20;
do {
    console.log(d2);
    // Increment by 2 to get only even numbers
    d2 += 2;
}
while (d2 <= 40);

/*
OUTPUT:
20
22
24
...
40
*/
```

```
// =====
// PROGRAM 4 : TABLE OF 4
// =====

/*
4 X 1 = 4
4 X 2 = 8
...
4 X 10 = 40
*/

let d3 = 1;
console.log("----- Table of 4 -----");

do {
    console.log(`4 X ${d3} = ${4 * d3}`);
    d3++;
}
while (d3 <= 10);

// =====
// PROGRAM 5 : CONDITION FALSE BUT STILL EXECUTES
// =====

let d4 = 11;

do {
    console.log(d4);
    d4++;
}
while (d4 <= 10);

/*
IMPORTANT:
-----
Condition is FALSE initially
11 <= 10 → false

BUT still loop executes ONCE
because do while first executes code
then checks condition.

OUTPUT:
11
*/

// =====
```

```
// PATTERN PROGRAMS
// =====

/*
PATTERN 1
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
*/

let rows = 5;

/*
OUTER LOOP:
Controls number of rows

INNER LOOP:
Controls number of columns/stars
*/

for (let i = 1; i <= rows; i++) {
  let pattern = "";
  for (let j = 1; j <= rows; j++) {
    pattern += "* ";
  }
  console.log(pattern);
}

// =====
// PATTERN 2 : INCREASING STAR PATTERN
// =====

/*
*
* *
* * *
* * * *
* * * * *
*/

for (let i = 1; i <= rows; i++) {
  let pattern = "";

  /*
  Inner loop runs based on row number
  Row 1 → 1 star
  Row 2 → 2 stars
  Row 3 → 3 stars
  */
}
```

```

    */
    for (let j = 1; j <= i; j++) {
        pattern += "* ";
    }
    console.log(pattern);
}
console.log("PC");

```

```

// =====
// PATTERN 3 : DECREASING STAR PATTERN
// =====

```

```

/*
* * * * *
* * * *
* * *
* *
*
*/

for (let i = 5; i >= 1; i--) {
    let pattern = "";
    for (let j = 1; j <= i; j++) {
        pattern += "* ";
    }
    console.log(pattern);
}

```

```

// =====
// PATTERN 4 : NUMBER PATTERN
// =====

```

```

/*
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
*/

for (let i = 1; i <= rows; i++) {
    let pattern = "";
    for (let j = 1; j <= i; j++) {
        pattern += j + " ";
    }
    console.log(pattern);
}

```

```
// =====
// STRING + NUMBER CONCATENATION
// =====

console.log("" + 2);

/*
OUTPUT:
2

WHY?
-----
"" is a string

When string + number happens,
JavaScript converts number into string.
*/

// =====
// NEW LINE EXAMPLES
// =====

console.log("Hello");
console.log("World");

/*
OUTPUT:
Hello
World
*/
console.log("Hello\nWorld");

/*
\n = NEW LINE CHARACTER

OUTPUT:
Hello
World
*/

// =====
// process.stdout.write()
// =====

/*
console.log():
-----
Prints output WITH new line automatically
```

```
process.stdout.write():  
-----  
Prints output WITHOUT new line  
*/  
  
process.stdout.write("Hello ");  
process.stdout.write("World");  
  
/*  
OUTPUT:  
Hello World  
*/
```

FUNCTIONS IN JAVASCRIPT

1. What is a Function?

A **function** is a self-contained, reusable block of code that performs a specific task. Functions are the **fundamental building blocks** of JavaScript programs.

Key Points:

- A function is defined once but can be executed (called/invoked) **multiple times**
- Functions help in code organization, reusability, and maintainability
- In JavaScript, functions are **first-class citizens** — they can be assigned to variables, passed as arguments, and returned from other functions
- Functions create their own **scope** — variables declared inside a function are not accessible outside
- Every function in JavaScript **implicitly returns undefined** if no return statement is provided

Interview Definition:

"A function is a reusable, self-contained unit of code that accepts input (parameters), processes it, and optionally returns output (return value). In JavaScript, functions are first-class objects."

2. Function Declaration and Function Calling

Function Declaration

- Creating a function using the function keyword
- Function is stored in memory but does NOT execute immediately
- Function body executes only when called

Function Calling / Invocation

- Executing the function using its name followed by ()
- Function can be called multiple times

Important Points:

- Function declaration is hoisted
- Can call function before declaration
- Improves code reusability

3. Types of Functions in JavaScript

A. Function Declaration (Named Function)

- Defined using the function keyword followed by a name
- **Hoisted** — can be called BEFORE the declaration in code
- Has its own this context

- Creates a named reference in current scope

Characteristics:

- Hoisting:  Yes (entire function is hoisted)
- Naming: Required
- this binding: Dynamic (depends on how it's called)
- arguments object:  Available
- Can be used as constructor:  Yes (new keyword)

B. Function Expression (Anonymous/Named Expression)

- A function assigned to a variable
- **NOT hoisted** — the variable is hoisted but its value (function) is NOT
- Can be anonymous (no name) or named
- Often used as callbacks

Characteristics:

- Hoisting:  No (only variable declaration is hoisted, not the function body)
- Naming: Optional
- this binding: Dynamic
- arguments object:  Available
- Can be used as constructor:  Yes

C. Arrow Function (ES6+)

- Shorter syntax using =>
- Does **NOT** have its own this — inherits this from the enclosing lexical scope
- Does **NOT** have arguments object
- Cannot be used as constructor (new keyword will throw error)
- Best for callbacks and short functions

Characteristics:

- Hoisting:  No
- Naming: Assigned to variable
- this binding: **Lexical** (inherits from parent scope)
- arguments object:  Not available (use rest parameters instead)

D. IIFE (Immediately Invoked Function Expression)

- A function that **runs immediately** after being defined
- Creates a **private scope** — variables inside are not accessible outside
- Used to avoid polluting global scope
- Pattern: (function() { ... })()

Use Cases:

- Module pattern (before ES6 modules)
- Initialization code that only runs once
- Creating private variables
- Avoiding global namespace pollution

 4. Function Parameters vs Arguments**Parameters**

- Variables listed in the function **definition**
- Act as **placeholders** for values
- Defined at function creation time

Arguments

- Actual **values** passed to the function when it is **called**
- Provided at function invocation time

Key Differences:

Aspect	Parameters	Arguments
When	At function definition	At function call
What	Variable names (placeholders)	Actual values
Where	In function signature	In function invocation
Example	<code>function add(a, b)</code> — a, b are parameters	<code>add(5, 3)</code> — 5, 3 are arguments

Important Interview Points:

- JavaScript does NOT throw error for mismatched parameter/argument count
- Extra arguments are ignored (but accessible via arguments object)
- Missing arguments become undefined
- Default parameters (ES6) handle undefined arguments

 5. Default Parameters (ES6)

- Allow parameters to have **predefined values** if no argument is passed
- Only triggers when argument is undefined (NOT for null or false)
- Can use expressions, function calls, or previous parameters as defaults
- Evaluated **left to right** at call time

Important Points:

- Default params are evaluated at **call time**, not at definition time
- Each call creates a fresh default value

- Later parameters can reference earlier parameters in defaults
- arguments object does NOT reflect default values (in non-strict mode)

6. Rest Parameters (...args)

- Collects **all remaining arguments** into a real Array
- Must be the **last** parameter in function signature
- Only ONE rest parameter is allowed per function
- Replaces the old arguments object approach

Rest vs Arguments Object:

Feature	Rest Parameters	arguments Object
Type	Real Array	Array-like object
Arrow functions	✓ Works	✗ Not available
Array methods	✓ map, filter, reduce	✗ Need to convert first
Named parameters	✓ Works alongside	Contains all arguments
ES version	ES6+	ES1 (legacy)

7. Return Statement

- **Stops** function execution immediately
- **Sends back** a value to the caller
- Only ONE value can be returned (use object/array for multiple values)
- If no return statement → function returns undefined
- Code after return is **unreachable** (dead code)

Important Points:

- return without value returns undefined
- Arrow functions with single expression have **implicit return** (no return keyword needed)
- Arrow functions with {} braces require **explicit return**
- Can return any data type: primitive, object, array, function, promise

8. Hoisting

What is Hoisting?

Hoisting is JavaScript's behavior of moving **declarations** (not initializations) to the top of their scope before code execution.

Function Hoisting Rules:

Type	Declaration Hoisted?	Value/Body Hoisted?	Can Call Before Definition?
Function Declaration	✅ Yes	✅ Yes (entire function)	✅ Yes
Function Expression	✅ Variable only	❌ No	❌ No (TypeError)
Arrow Function	✅ Variable only	❌ No	❌ No (TypeError)
var variable	✅ Yes	❌ No (undefined)	Accessible but undefined
let/const	✅ Yes (TDZ)	❌ No	❌ No (ReferenceError)

Temporal Dead Zone (TDZ):

- The period between entering a scope and the variable's declaration being reached
- let and const exist in TDZ — accessing them throws ReferenceError
- var does NOT have TDZ — returns undefined before initialization

📌 9. Scope in Functions**Types of Scope:****1. Global Scope**

- Variables declared outside any function/block
- Accessible from anywhere in the program
- In browser: attached to window object

2. Function Scope (Local Scope)

- Variables declared inside a function
- Accessible ONLY within that function
- Created when function is called, destroyed when function returns
- var creates function-scoped variables

3. Block Scope

- Variables declared with let or const inside {}
- Accessible only within that block
- var does NOT respect block scope

Memory Implications:

- Closures prevent garbage collection of outer variables
- Can cause memory leaks if not handled properly
- Only the variables actually referenced by the inner function are retained

10. Callback Functions

Definition:

A callback is a function **passed as an argument** to another function, which is then **invoked** inside the outer function to complete some action.

Key Points:

- Callbacks are NOT called immediately — they are "called back" at a later time
- Foundation of JavaScript's asynchronous programming
- Used in event handlers, timers, array methods, API calls
- Can be synchronous (map, filter) or asynchronous (setTimeout, fetch)

Callback Hell (Pyramid of Doom):

- Nested callbacks become deeply indented and hard to read
- Solution: Promises, async/await, or modular functions

Higher-Order Function:

- A function that takes a function as argument OR returns a function
- map(), filter(), reduce(), forEach() are all higher-order functions

11. Arrow Functions

Key Differences from Regular Functions:

Feature	Regular Function	Arrow Function
<code>this</code> binding	Dynamic (depends on caller)	Lexical (inherits from parent)
<code>arguments</code> object	✅ Available	❌ Not available
<code>new</code> keyword	✅ Can be constructor	❌ TypeError
<code>prototype</code> property	✅ Has it	❌ Doesn't have
Hoisting	✅ (declarations)	❌ No
Implicit return	❌ Need <code>return</code> keyword	✅ For single expressions
<code>super</code> keyword	✅ Works	❌ Not own <code>super</code>
Method definition	✅ Use for object methods	❌ Avoid (wrong <code>this</code>)

When to Use Arrow Functions:

- ✅ Callbacks (map, filter, setTimeout)

-  Short single-expression functions
-  When you need lexical this
-  Array method callbacks
-  Promise chains (.then())

When NOT to Use Arrow Functions:

-  Object methods (needs own this)
-  Event handlers in DOM (needs dynamic this)
-  Constructor functions
-  When you need arguments object
-  Prototype methods

12. IIFE (Immediately Invoked Function Expression)

Definition:

A function that is defined and **executed immediately** without being stored or called separately.

Syntax Patterns:

- (function() { ... })()
- (function() { ... }())
- (() => { ... })()
- !function() { ... }()
- +function() { ... }()
- void function() { ... }()

Why IIFE Exists:

- JavaScript (before ES6) only had **function scope**
- No block scope meant var leaked everywhere
- IIFE created a **private scope** to contain variables
- Prevented global namespace pollution

Use Cases:

- Module pattern (before ES6 modules)
- One-time initialization
- Avoiding naming conflicts
- Creating private variables
- Running async code at top level

Modern Alternatives:

- ES6 modules

- Block scope with let and const

13. Interview FAQs — Functions Theory

Q1: What is the difference between function declaration and function expression?

A: Function declarations are hoisted completely (can be called before definition). Function expressions are NOT hoisted (variable is hoisted but value is not). Declarations use function name() syntax while expressions assign anonymous/named functions to variables.

Q2: What are first-class functions?

A: In JavaScript, functions are first-class citizens meaning they can be:

- Assigned to variables
- Passed as arguments to other functions
- Returned from functions
- Stored in data structures (arrays, objects)
- Have properties and methods

Q3: What is a callback function?

A: A function passed as argument to another function and executed later.

Q5: What is hoisting?

A: Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope during the compilation phase. Function declarations are fully hoisted. var declarations are hoisted but initialized as undefined. let/const are hoisted but remain in the Temporal Dead Zone until their declaration is reached.

Q6: What is the difference between arrow functions and regular functions?

A: Arrow functions:

- Don't have own this
- Don't have arguments
- Cannot be constructors
- Use lexical scope for this

Q11: What is recursion?

A: Function calling itself until base condition is met.

Q12: Difference between undefined and not defined?

A:

- undefined → declared but no value assigned
- not defined → variable does not exist

Q13: What happens if you call a function with fewer arguments than parameters?

A: Missing parameters become undefined.

Q14: What is the arguments object?

A: Array-like object containing all passed arguments in normal functions.

Q16: What is an IIFE and why is it used?

A: Immediately Invoked Function Expression used for private scope and avoiding global pollution.

Q19: What is the difference between var, let, and const in function scope?

A:

- var → function scoped
- let → block scoped
- const → block scoped and cannot reassign

Program 1

```
//Function
//A function is a block of code that performs a specific task
//and can be reused multiple times in a program.

//Functions help us organize code into smaller reusable blocks.
//Instead of writing the same code again and again,
//we can write it once inside a function and call it whenever
needed.

//Functions run only when they are called/invoked.

//Why functions are important
//1. Code Reusability -> Write once, use many times
//2. Code Organization -> Divide large programs into smaller parts
//3. Easy Maintenance -> Fix code in one place, changes apply
everywhere
//4. Reduce Code Duplication -> Avoid writing same code repeatedly
//5. Better Readability -> Code becomes clean and easy to
understand
//6. Easier Debugging -> Errors can be identified quickly

//-----
//Without Function
//-----

//Same code repeated multiple times
console.log("Welcome user 1");
```



```
console.log("Welcome user 2");
console.log("Welcome user 3");

//-----
//Using Function
//-----

//Function Declaration
//greetUser is function name
//name is parameter
function greetUser(name) {
    console.log("Welcome " + name);
}

//Function Calls
//"User 1", "User 2" are arguments
greetUser("User 1");
greetUser("User 2");
greetUser("User 3");

console.log();

//-----
//Login Function Example
//-----

//Function to simulate login process
function login(username, password) {

    //Printing username
    console.log("Entering username : " + username);

    //Printing password
    console.log("Entering password : " + password);

    //Login action
    console.log("Clicking login button");
}

//Calling login function
login("Admin", "admin123");

console.log();

login("user", "user123");

//-----
//Syntax of Function
//-----

/*
function functionName(parameter1, parameter2){
```

```

    //Code to execute

    return value; //optional
}
*/

//function      -> Keyword used to create function
//functionName  -> Name of the function
//parameters    -> Inputs passed to function
//return        -> Sends result back (optional)

//-----
//Simple Function Example
//-----

//Function without parameters
function sayHello() {

    //Code inside function runs only when function is called
    console.log("Hello World !");
}

//Calling function
sayHello();

//-----
//Function with Parameters
//-----

//a and b are parameters
function add(a, b) {

    //Adding two numbers
    console.log(a + b);
}

//Passing values while calling function
add(10, 5);
add(40, 20);

//-----
//Function Execution Flow
//-----

//JavaScript executes code line by line

console.log("Step 1");

//Function definition
function myFunction() {

```

```

    //This executes only when function is called
    console.log("Step 3 - Inside function");
}

console.log("Step 2");

//Calling function
myFunction();

console.log("Step 4");

/*
Output:
Step 1
Step 2
Step 3 - Inside function
Step 4
*/

//-----
//Types of Functions in JavaScript
//-----

//1. Function Declaration
//2. Function Expression
//3. Anonymous Function
//4. Arrow Function
//5. Callback Function
//6. IIFE (Immediately Invoked Function Expression)

//-----
//1. Function Declaration
//-----

//Example 1
function greet() {

    console.log("Good morning");
}

//Calling function
greet();

//-----
//Multiply Function
//-----

function multiply(x, y) {

    //Multiplying two numbers
    console.log(x * y);
}

```

```
multiply(4, 5);
multiply(2, 4);

//-----
//Browser Launch Example
//-----

//Reusable function for browser launching
function launchBrowser(browserName) {

    console.log("Launching " + browserName + " browser");
}

launchBrowser("Chrome");
launchBrowser("FireFox");
launchBrowser("Edge");

//-----
//Check Even or Odd
//-----

//Function to check whether number is even or odd
function checkEvenOdd(num) {

    //If number divided by 2 gives remainder 0
    if (num % 2 === 0) {

        console.log(num + " is Even");
    }
    else {

        console.log(num + " is Odd");
    }
}

//Function calls
checkEvenOdd(10);
checkEvenOdd(13);
checkEvenOdd(123);

//-----
//Print Table Function
//-----

//Function to print multiplication table
function printTable(num) {

    //Loop from 1 to 10
    for (let i = 1; i <= 10; i++) {

        //Template literals used for formatting
```

```
        console.log(`${num} x ${i} = ${num * i}`);
    }
}

//Printing tables
printTable(5);

console.log();

printTable(7);

console.log();

printTable(10);

//-----
//Important Theory Points
//-----

//1. Function Definition
//    Writing the function code is called function definition.

//2. Function Call / Invocation
//    Executing a function is called function call.

//3. Parameters
//    Variables defined in function definition are called
parameters.

//4. Arguments
//    Values passed during function call are called arguments.

//5. Return Keyword
//    return is used to send value back from function.

//6. Reusable Code
//    Functions allow us to reuse same logic multiple times.

//7. Local Variables
//    Variables declared inside function are accessible only inside
function.

//8. Function Naming Convention
//    Function names should be meaningful and usually written in
camelCase.
//    Example:
//    calculateTotal()
//    printTable()
//    launchBrowser()
```

Program 2

```
//-----  
//Function with Parameters  
//-----  
  
//Function to fill form details  
function fillForm(name, email, phone) {  
  
    //Printing user details  
    console.log("Name : " + name);  
    console.log("Email : " + email);  
    console.log("Phone : " + phone);  
}  
  
//Calling function  
fillForm("Rahul", "rahul@gmail.com", "9484747444");  
  
//-----  
//Theory Point  
//-----  
  
//Parameters  
//Variables declared inside function definition.  
  
//Arguments  
//Actual values passed during function call.  
  
//Example:  
//name, email, phone -> Parameters  
//"Rahul", "rahul@gmail.com", "9484747444" -> Arguments  
  
//-----  
//Default Parameters  
//-----  
  
//Default parameter is used when  
//no argument is passed during function call.  
  
function greetUser(name = "Guest") {  
  
    console.log("Welcome " + name);  
}  
  
//No value passed  
//Default value "Guest" will be used  
greetUser();  
  
//Value passed  
//Default value replaced by "Akshay"  
greetUser("Akshay");
```

```
//-----  
//Multiple Default Parameters  
//-----  
  
//Function with multiple default values  
function createUser(name, role = "Tester", level = 1) {  
    console.log(`${name} - ${role} - Level ${level}`);  
}  
  
//Only name passed  
//role = Tester  
//level = 1  
createUser("Amit");  
  
//All arguments passed  
createUser("Raj", "Lead", 5);  
  
//-----  
//Theory Point  
//-----  
  
//Default parameters help avoid undefined values.  
//They make functions more flexible and reusable.  
  
//-----  
//Missing Arguments  
//-----  
  
//If arguments are missing,  
//JavaScript assigns undefined to missing parameters.  
  
function add(a, b) {  
    console.log(a + b);  
}  
  
//Only one argument passed  
//b becomes undefined  
add(5); //5 + undefined = NaN  
  
//Both arguments passed  
add(5, 10);  
  
//-----  
//Theory Point  
//-----  
  
//NaN means "Not a Number".  
  
//It occurs when JavaScript performs
```

```
//invalid mathematical operations.

//Example:
//5 + undefined = NaN

//-----
//Extra Arguments
//-----

//If extra arguments are passed,
//JavaScript ignores additional values by default.

function sum(a, b) {
    console.log(a + b);
}

//Normal function call
sum(1, 2);

//Extra arguments 3 and 4 are ignored
sum(10, 22, 3, 4);

//-----
//Theory Point
//-----

//JavaScript allows extra arguments,
//but only declared parameters are used normally.

//-----
//Return Statement
//-----

//What is return?

//return sends a value back
//to where the function was called.

//Without return,
//functions automatically return undefined.

//-----
//Function with Return Statement
//-----

function add1(a, b) {
    //Returning result
    return a + b;
}
```



```
//Storing returned value
let result = add1(3, 7);

//Printing returned value
console.log(result);

//-----
//Function without Return Statement
//-----

function add2(a, b) {

    //Calculation performed
    //but value is not returned
    a + b;
}

//Function returns undefined
let result2 = add2(3, 7);

console.log(result2); //undefined

//-----
//Difference Between console.log() and return
//-----

//console.log()
//-> Displays value on console
//-> Cannot store result directly

//return
//-> Sends value back
//-> Result can be stored in variable

//-----
//Example
//-----

//Using console.log()
function demo1() {

    console.log("Hello");
}

let output1 = demo1();

console.log(output1); //undefined

//-----

//Using return
function demo2() {
```

```

    return "Hello";
}

let output2 = demo2();

console.log(output2); //hello

//-----
//Important Interview Theory Points
//-----

//1. Parameters
//   Variables inside function definition.

//2. Arguments
//   Values passed during function call.

//3. Default Parameter
//   Used when no argument is passed.

//4. Missing Argument
//   Missing parameters become undefined.

//5. Extra Argument
//   Extra arguments are ignored by default.

//6. return Keyword
//   Sends value back from function.

//7. Function without return
//   Automatically returns undefined.

//-----
//Real-Time Example
//-----

//Login Function
function login(username = "Guest", password = "12345") {
    console.log("Username : " + username);
    console.log("Password : " + password);
}

//Using default values
login();

console.log();

//Passing custom values
login("Admin", "admin123");

```

Program 3

```
// =====  
// JavaScript Functions - Theory + Examples  
// =====  
  
// =====  
// 1. Function Declaration  
// =====  
// A function declaration is created using the function keyword.  
// These functions are HOISTED in JavaScript.  
// Meaning: We can call them before declaration.  
  
function getUser() {  
    // Returning Object from Function  
    return {  
        name: "Ketan",  
        age: 30,  
        role: "Tester"  
    };  
}  
  
// Function Call  
let user = getUser();  
  
console.log(user);           // Complete object  
console.log(user.name);     // Access specific property  
  
// =====  
// Returning Array from Function  
// =====  
// Functions can return arrays also.  
  
function getColors() {  
    return ["Red", "Green", "Blue"];  
}  
  
let colors1 = getColors();  
  
console.log(colors1);        // Complete array  
console.log(colors1[0]);     // First element  
  
// =====  
// Code After Return is Unreachable  
// =====  
// Once return executes, function execution stops immediately.  
  
function test() {  
    // This line executes  
    console.log("This runs");  
}
```

```

    return "Hello";

    // Unreachable Code
    // This line never executes
    console.log("Hi");
}

let test1 = test();

console.log(test1);

// =====
// Function with Parameters and Return Value
// =====
// Parameters are variables passed into functions.

function addition(a, b) {
    // Returning addition result
    return a + b;
}

// Passing arguments
let add = addition(30, 10);

console.log(add);

// =====
// 2. Function Expression
// =====
// Function stored inside a variable.
//
// Syntax:
//
// let/const variableName = function(parameters){
//
// }

const greet = function () {
    console.log("Hello from expression !");
};

// Function Call
greet();

// Function Expression with Parameters
const multiply = function (a, b) {
    console.log("Addition :", a + b);

    // Return statement is optional
    return a * b;
};

```

```

console.log(multiply(3, 4));

// =====
// Difference Between Function Declaration
// and Function Expression
// =====

/*

Function Declaration:
-----
1. Created using function keyword directly
2. Hoisted
3. Can call before declaration

Function Expression:
-----
1. Function stored inside variable
2. Not fully hoisted
3. Cannot call before declaration

*/

// =====
// Hoisting in JavaScript
// =====

// Hoisting means JavaScript moves declarations
// to the top of their scope before execution.

// =====
// Function Declaration Hoisting
// =====

// Calling before declaration
sayHi();

function sayHi() {
    console.log("Hi");
}

// =====
// Function Expression Hoisting
// =====

// Function expressions are NOT accessible before initialization.

// sayBye();
// Error:
// Cannot access 'sayBye' before initialization

const sayBye = function () {

```

```

    console.log("Bye !!");
};

sayBye();

// =====
// Variable Hoisting with var
// =====

// var variables are hoisted
// but initialized with undefined.

console.log(a1); // undefined

var a1 = 10;

// JavaScript internally behaves like this:

var a1;

console.log(a1); // undefined

a1 = 10;

// =====
// let and Temporal Dead Zone (TDZ)
// =====

// let and const are hoisted
// BUT they stay inside Temporal Dead Zone
// until initialization.

// console.log(y); // ReferenceError

let y = 20;

// =====
// Important Interview Theory Points
// =====

/*

1. Function Declaration
-----
- Hoisted
- Accessible before declaration
- Reusable block of code

2. Function Expression
-----
- Stored in variable
- Not accessible before initialization

```

- Often used in callbacks

3. Anonymous Function

- Function without name
- Mostly used in expressions

4. Return Keyword

- Stops function execution
- Sends value back to caller
- Code after return never executes

5. Hoisting

- JavaScript moves declarations to top
- Only declarations are hoisted, not values

6. var vs let

var:

- Hoisted
- Initialized as undefined
- Function scoped

let:

- Hoisted but in TDZ
- Cannot access before declaration
- Block scoped

7. Temporal Dead Zone (TDZ)

- Time between hoisting and initialization
- Accessing variable gives ReferenceError

*/

```
// =====
// Quick Comparison Table
// =====
```

/*

Function Declaration

Hoisted -> Yes
 Accessible Before Decl -> Yes

Function Expression

Hoisted -> No

Accessible Before Decl -> No

var

```
-----
Hoisted          -> Yes
Default Value    -> undefined
```

let

```
-----
Hoisted          -> Yes
Accessible Before Decl -> No
Error Type       -> ReferenceError
```

*/

```
// =====
// Random Internal JavaScript Function Calls
// =====
```

```
getDefaultsForExtension();
Text();
```

Program 4

```
// =====
// Anonymous Functions & Arrow Functions
// =====

// =====
// Anonymous Function
// =====
// Anonymous function means function without a name.
//
// Mostly used:
// 1. As callback functions
// 2. As arguments
// 3. In setTimeout / setInterval
// 4. Immediately Invoked Function Expressions (IIFE)

// =====
// setTimeout Example
// =====
// setTimeout executes code after given time.

// Syntax:
// setTimeout(function, time_in_milliseconds)

// 5000 ms = 5 seconds
```



```
// setTimeout(function () {  
//     console.log("Runs after 5 seconds");  
// }, 5000);  
  
// =====  
// forEach with Anonymous Function  
// =====  
// forEach loop executes function for every array element.  
  
let fruits = ["Apple", "Banana", "Mango", "Test"];  
  
fruits.forEach(function (fruit) {  
    console.log(fruit);  
});  
  
// =====  
// Another forEach Example  
// =====  
  
let number = [1, 2, 3, 4, 5];  
  
number.forEach(function (num) {  
    console.log(num);  
});  
  
// =====  
// Arrow Functions  
// =====  
// Arrow functions introduced in ES6.  
//  
// Short syntax for writing functions.  
//  
// Syntax:  
//  
// const functionName = (parameters) => {  
//     body  
// }  
  
// =====  
// Arrow Function without Parameters  
// =====  
  
const hello = () => {  
    console.log("Hello Arrow");  
};
```

```
hello();
```

```
// =====  
// Arrow Function with One Parameter  
// =====
```

```
const square = (num) => {  
    return num * num;  
};
```

```
let result = square(5);
```

```
console.log(result);  
console.log(square(5));  
console.log(result + 5);
```

```
// =====  
// Arrow Function with Multiple Parameters  
// =====  
// More than one parameter requires () brackets.
```

```
const add = (a, b) => {  
    return a + b;  
};
```

```
console.log(add(3, 5));
```

```
// =====  
// Short Form Arrow Function  
// =====  
// If function has single statement,  
// return keyword and {} can be removed.
```

```
const multiply = (a, b) => a * b;
```

```
console.log(multiply(2, 4));
```

```
// =====  
// Important Theory Points  
// =====
```

```
/*
```

1. Anonymous Function

-
- Function without name
 - Mostly used as callback

Example:

```
function () {  
    console.log("Hello");  
}
```

2. Callback Function

- Function passed as argument to another function

Example:

```
forEach(function () {})
```

3. Arrow Function

- Short syntax of function
- Introduced in ES6
- Uses => symbol

4. Advantages of Arrow Function

- Short and clean syntax
- Easy to write
- Mostly used in modern JavaScript

5. Difference Between Normal Function and Arrow Function

Normal Function:

- Uses function keyword
- Has its own this keyword

Arrow Function:

- Uses =>
- Does not have its own this

6. forEach Loop

- Used for arrays
- Executes callback for every element

7. setTimeout

```
-----
- Executes function after specific delay
- Time is in milliseconds
```

```
1000 ms = 1 second
```

```
*/
```

Program 5

```
// =====
// ARROW FUNCTIONS, CALLBACK FUNCTIONS & SCOPE IN JAVASCRIPT
// =====

// =====
// FILTER METHOD + ARROW FUNCTION
// =====

// filter() is used to return elements that match a condition

const testResult = [85, 92, 45, 78, 33, 99];

// Arrow Function Syntax
// (parameter) => {
//     code
// }

// Filter scores greater than or equal to 60
const passed = testResult.filter(score => score >= 60);
console.log(passed); // [85, 92, 78, 99]

// =====
// SIMPLE ARROW FUNCTION
// =====
// Arrow function without parameters
let Test = () => {
    console.log("Hello good morning");
};
Test();

// =====
// ARROW FUNCTION WITH PARAMETER
// =====

// Arrow function with one parameter
let addition = (num) => {
    console.log(num + num);
};
```

```
addition(5); // 10

// =====
// CALLBACK FUNCTION
// =====

// Callback Function:
// A function passed as an argument to another function
// and executed later

// -----
// BASIC CALLBACK EXAMPLE
// -----

function greet(name) {
    console.log("Hello " + name);
}

function processUser(callback) {
    // Calling callback function
    callback("Rahul");
}

// Passing greet function as argument
processUser(greet);

// =====
// CALLBACK FUNCTION WITH DATA
// =====

function processData(data, callback) {
    console.log("Processing : " + data);
    // Execute callback function
    callback();
}

function done() {
    console.log("Processing completed");
}
processData("Test Data", done);

// =====
// CALLBACK FUNCTION – AUTOMATION REAL TIME EXAMPLE
// =====

// Example:
// Retry mechanism in automation testing

function retryAction(action, retries, callback) {
    // Retry loop
```

```

    for (let i = 1; i <= retries; i++) {
        console.log(`Attempt ${i} : ${action}`);
    }
    // Execute callback after retries
    callback("Success after " + retries + " attempts");
}

// Anonymous callback function
retryAction("Click Button", 2, function (result) {
    console.log(result);
});

// =====
// CALLBACK FUNCTION WITH PARAMETER
// =====

function display(name) {
    console.log("Welcome " + name);
}

function getUser(callback) {
    let username = "Ram";
    // Passing username to callback
    callback(username);
}

// Passing display function as callback
getUser(display);

// =====
// FUNCTION SCOPE (LOCAL SCOPE)
// =====

// Variables declared inside a function
// are accessible only inside that function

function myFunction() {
    let localVar = "I am local";
    console.log(localVar);
}
myFunction();

// console.log(localVar); ❌ Error
// localVar cannot be accessed outside function

// =====
// BLOCK SCOPE
// =====

```

```
// let and const are block scoped
// var is function scoped

function testScope() {
  if (true) {
    // Accessible only inside block
    let blockVar = "Block Scope Variable";
    // Accessible only inside block
    const constVar = "Constant Variable";
    // Accessible in entire function
    var funcVar = "Function Scope Variable";
    console.log(blockVar);
    console.log(constVar);
  }
  // console.log(blockVar); ❌ Error
  // blockVar is not accessible outside block

  // Accessible because var is function scoped
  console.log(funcVar);
}
testScope();

// =====
// IMPORTANT INTERVIEW POINTS
// =====

// let → Block Scoped
// const → Block Scoped
// var → Function Scoped

// Callback functions are heavily used in:
// ✓ Automation Testing
// ✓ API Handling
// ✓ setTimeout()
// ✓ Event Handling
// ✓ Async Programming
```

Hello Everyone,

Weekly External Mock Interviews will start from tomorrow. The detailed schedule will be shared shortly in the group.

Please note that these mock interviews are extremely important and are being conducted to prepare you for real-time company interviews. All students are expected to maintain professionalism and seriousness throughout the session.

 Important Guidelines (Mandatory for Everyone):

1. Mock interviews are compulsory for all students.
No one should skip the session without prior notice or a valid reason.
2. Attendance is mandatory.
Students who remain absent without informing will be marked accordingly.
3. Your video camera must remain ON throughout the entire mock interview.
This is mandatory to simulate real interview conditions followed by companies.
4. Proper introduction is compulsory.
You must confidently introduce yourself, including your background, skills, project experience, and current learning.
5. Keep your Jupyter Notebook / IDE / required setup ready before joining the session.
You should be prepared to share your screen immediately if the interviewer asks for coding, debugging, or explanation.
6. Maintain professional interview etiquette:
 - * Sit in a quiet environment
 - * Avoid background noise and distractions
 - * Join with proper internet connectivity
 - * Keep your microphone muted unless asked to speak
 - * Use your proper name while joining the meeting
7. Each mock interview will be conducted individually and will last approximately 20 minutes.
Only one student will be interviewed at a time.
8. Do NOT continuously disturb or interrupt during another student's interview.
Wait for your turn patiently.
9. You should join the meeting only when your Name or Roll Number is called by the interviewer in the WhatsApp group.
10. Students are expected to answer confidently and seriously, exactly as in real company interviews.
Casual behavior, lack of preparation, or negligence will not be appreciated.

11. Repeated absence, late joining, or unprofessional behavior may affect your participation in future activities and recommendations.

These mocks are designed to improve your confidence, communication, technical knowledge, and interview performance. Treat every mock interview as a real interview opportunity.

Be prepared and give your best.

Thank You.

Good morning sir/mam.

First of all, thank you for giving me this opportunity to introduce myself.

My name is Rahul Sharma, and I have around 4 years of experience in Software Testing, including both Manual Testing and Automation Testing.

Currently, I am working with XYZ Company on an E-commerce domain project as a QA Automation Engineer. In my current project, I am responsible for understanding requirements, preparing test scenarios, executing test cases, reporting defects, and automating regression test cases.

In Manual Testing, I have experience in Functional Testing, Regression Testing, Smoke Testing, Sanity Testing, Integration Testing, and User Acceptance Testing. I also have good knowledge of SDLC and STLC processes, and I regularly use Jira for defect tracking and test management activities.

For Automation Testing, I have worked extensively on Selenium with Java and currently I am working on Playwright with JavaScript. I have experience in developing and maintaining automation frameworks using the Page Object Model design pattern. I have worked on reusable utilities, locators, assertions, test data handling, and generating reports.

In Playwright, I have worked on cross-browser testing, API testing, parallel execution, handling dynamic elements, and CI/CD integration using Jenkins and GitHub.

Apart from this, I am also familiar with API testing using Postman and basic SQL queries for database validation.

I am currently working in an Agile environment where I actively participate in sprint planning, daily stand-up meetings, retrospective meetings, and defect discussions with developers and business analysts.

That's all about me.

Thank you.

ARRAYS IN JAVASCRIPT

1. INTRODUCTION TO ARRAYS

--- What is an Array? ---

// An array is an ordered collection of values stored in a single variable.
// It allows you to store multiple values of any data type under one name.
// Think of it as a list/container that holds items in a specific order.

// --- Why Arrays are Important ---

// In automation testing, we deal with:
// - Multiple test data values
// - Lists of web elements
// - API response arrays
// - Test results collection
// - Browser tabs, URLs, locators etc.

// --- Advantages of Arrays ---

// 1. Store multiple values in one variable
// 2. Ordered - maintains insertion order
// 3. Dynamic - can grow/shrink at runtime
// 4. Built-in methods for searching, sorting, filtering
// 5. Works with loops for repetitive tasks
// 6. Can hold any data type (string, number, object, etc.)

// --- Real-Time Use Cases in Automation ---

// - Store list of test URLs
// - Store locators for elements
// - Store test data for data-driven testing
// - Collect all error messages from a page
// - Store API response items

// =====
// 🔥 ARRAYS IN JAVASCRIPT
// =====

//Array
// Array stores multiple values inside single variable

// let no = 1,2,3; ❌ Invalid
// let name = "Rahul";
// name = "Ketan";

// =====
// 💎 Difference Between Variables and Arrays
// =====

```

// Variable stores single value
let singleUrl = "https://www.facebook.com/";

// Array stores multiple values
// index =      0                      1
2
let urls = ["https://www.facebook.com/", "https://
www.flipkart.com/", "https://www.amazon.in/"];

console.log(typeof urls); // object

// =====
// ♦ Ordered Collection
// =====

// Arrays maintain insertion order

// index =      0                      1                      2
3
let steps = ["Open browser", "Navigate to URL", "Click login",
"Enter Credentials"];

// neg index = -4                      -3                      -2
-1

console.log(steps[0]);
console.log(steps[1]);

// Array index out of bound
// JavaScript returns undefined
console.log(steps[5]);

// =====
// ♦ Dynamic Nature of Arrays
// =====

// Arrays are dynamic
// We can add/remove elements anytime

// index =      0                      1
let testCases = ["TC001", "TC002"];

console.log(testCases);

// push() -> Adds element at end
testCases.push("TC003");

```

```
console.log(testCases);

// index =      0      1      2

testCases.push("TC004");

console.log(testCases);

// index =      0      1      2      3

// pop() -> Removes last element
testCases.pop();

console.log(testCases);

// =====
// ♦ Array Syntax
// =====

// let/const arrayName = [element1, element2, element3]

// let numbers = [10,20,30];

// =====
// ♦ Array Declaration
// =====

// index =      0      1      2
let browsers = ["Chrome", "Firefox", "Safari"];

console.log(browsers);

// length -> total number of elements
console.log(browsers.length);

// Accessing element
console.log(browsers[2]);

// =====
// ♦ Mixed Datatype Arrays
// =====

// JavaScript allows different datatypes in same array
```

```

// index =          0          1      2      3      4
5
let mixedData = ["TestCase001", 200, true, null, undefined,
{ status: "pass" }];

console.log(mixedData);

// Accessing object
console.log(mixedData[5]);

// =====
// ♦ Creating Arrays
// =====

// Method 1 -> Using Square Brackets [] ★ Most Common

// index =      0          1          2
let locators = ["#username", "#password", "#login"];

console.log(locators);

// =====
// Method 2 -> Using new Array()
// =====

// index =      0      1      2      3
let ports = new Array(300, 200, 400, 500);

console.log(ports);

console.log(ports[1]);
console.log(ports[3]);

// =====
// ♦ Accessing Array Elements
// =====

// Arrays are zero-indexed

// index =      0          1          2          3
4          5
let pages = ["Home", "DashBoard", "Profile", "Settings",
"Accounts", "Register"];

```

```

// neg index =  -6          -5          -4          -3
-2          -1

// Positive indexing
console.log(pages[0]);
console.log(pages[1]);
console.log(pages[2]);
console.log(pages[3]);
console.log(pages[4]);
console.log(pages[5]);

console.log();

// =====
// ♦ Looping Through Arrays
// =====

for (let i = 0; i < pages.length; i++) {
    console.log(pages[i]);
}

// Printing array with index
for (let i = 0; i < pages.length; i++) {
    console.log(`Page ${i} : ${pages[i]}`);
}

// =====
// ♦ Negative Indexing
// =====

// at(-1) -> last element

console.log(pages.at(-1));
console.log(pages.at(-2));
console.log(pages.at(-3));
console.log(pages.at(-4));
console.log(pages.at(-5));
console.log(pages.at(-6));

// =====
// ♦ Modifying Values
// =====

```

```

// index =      0          1
let modules = ["Login", "DashBoard"];

console.log(modules);

// Modifying existing value
modules[1] = "Reports";

console.log(modules);

// Adding new values dynamically
modules[2] = "Settings";

console.log(modules);

// index =      0          1          2

modules[3] = "Register";
console.log(modules);

// index =      0          1          2          3

// =====
// ♦ Arrays are Mutable
// =====

// Arrays allow value modification

// index =      0      1      2      3
let number = [10, 20, 30, 40];

console.log(number);

// Changing datatype also possible
number[0] = "Test";

console.log(number);

// index =      0          1      2      3

// =====
// 🔥 IMPORTANT POINTS
// =====

// ✓ Arrays are dynamic
// ✓ Arrays are zero-indexed

```

```
// ✓ Arrays can store mixed datatypes
// ✓ typeof array => object
// ✓ length gives total elements
// ✓ JavaScript arrays support dynamic size
// ✓ Arrays are mutable
```

Program 2

```
//length property
//Use → Returns number of elements present in array

let tools = ["Selenium", "Playwright", "Cypress", "Puppeteer",
"Qualitia"];

console.log(tools.length); //o/p - 5

console.log();

//Use length property for iteration
for (let i = 0; i < tools.length; i++) {
    console.log(tools[i]);
}

//length changes dynamically

let items = [1, 2, 3];

console.log(items.length);

items.push(4, 5);

console.log(items.length);

items.pop();

console.log(items.length);

//Array Iterations

//1. for loop
//Use → Used when index is required
//Most commonly used in automation

let testSceanrios = ["Login Test", "Search Test", "Cart Test",
"Checkout Test", "Logout Test"];

console.log("--- For loop ---");
```



```
for (let i = 0; i < testSceanrios.length; i++) {
  console.log(`Sceanrio ${i + 1} : ${testSceanrios[i]}`);
}
```

```
//2. While loop
//Use → Used when iteration count is unknown
```

```
console.log("--- While Loop ---");

let j = 0;

while (j < testSceanrios.length) {
  console.log(`Running: ${testSceanrios[j]}`);
  j++;
}
```

```
//3. do while loop
//Use → Executes at least once even if condition is false
```

```
console.log("--- do While Loop ---");

let k = 0;

do {
  console.log(`Executing :${testSceanrios[k]}`);
  k++;
} while (k < testSceanrios.length);
```

```
//4. for of loop
//Use → Used to directly access values
//Works with arrays, strings, sets, iterables
```

```
//Syntax
//for(let/const variableName of arrayName){
//    //code
//}
```

```
console.log("--- for of loop ---");

for (let sceanrio of testSceanrios) {
  console.log(`Test : ${sceanrio}`);
}

console.log("LC");
```

```
//5. for in loop
//Use → Used to access indexes/keys
```

```
console.log("--- for in loop ---");  
  
for (let i in testSceanrios) {  
    console.log(`Index ${i} : ${testSceanrios[i]}`);  
}
```

//Array Methods

```
//1. push()  
//Use → Add element at END of array  
  
let queue = ["Test1", "Test2"];  
queue.push("Test3");  
console.log(queue);  
queue.push("Test4", "Test5", "Test4");  
console.log(queue);
```

```
//Real-time use  
//Adding failed test cases dynamically  
  
let failedTC = [];  
failedTC.push("Login Test");  
failedTC.push("Payment Test");  
console.log(failedTC);
```

```
//2. pop()  
//Use → Remove LAST element from array  
  
let stack = ["Step1", 'Step2', "Step3"];  
console.log(stack);  
stack.pop();  
console.log(stack);
```

```
//Real-time use  
//Removing latest executed test  
  
let execution = ["TC1", "TC2", "TC3"];
```

```
execution.pop();

console.log(execution);

//3. shift()
//Use → Remove FIRST element from array

stack.shift();

console.log(stack);

//Real-time use
//Removing first request from queue

let apiQueue = ["Request1", "Request2", "Request3"];

apiQueue.shift();

console.log(apiQueue);

//4. unshift()
//Use → Add element at BEGINNING of array

stack.unshift("Step 1");

console.log(stack);

//Real-time use
//Adding high priority test at beginning

let priorityTests = ["Search Test", "Cart Test"];

priorityTests.unshift("Login Test");

console.log(priorityTests);

//5. splice()
//Use → Add/remove/replace elements at any position

//Syntax
//array.splice(startIndex, deleteCount, item1, item2)

let feature = ["Login", "Search", "Cart", "Checkout", "Logout"];

//Remove 1 element at index 2
```

```
let spliced = feature.splice(2, 1); //removed element
console.log(spliced);
console.log(feature);

//Add without removing
feature.splice(2, 0, "Wishlist", "Cart");
console.log(feature);

//Replace : remove 1 add 1
feature.splice(1, 1, "SSO Login");
console.log(feature);

//add, remove, or replace element
//Array.splice(start, deleteCount, item1, item2);
//start → index position
//deleteCount → number of elements to remove
//item1 → optional item to add

//Real-time use
//Updating application modules dynamically
let modules = ["Login", "Payment", "Reports"];
modules.splice(1, 1, "Dashboard");
console.log(modules);

//6. slice()
//Use → Extract portion of array

//Syntax
//array.slice(startIndex, endIndex)

//startIndex → included
//endIndex → excluded
```

```
let allTest = ["T1", "T2", "T3", "T4", "T5"];
let subset = allTest.slice(1, 3);
console.log(subset);
console.log(allTest.slice(1));
console.log(allTest.slice(0, 5));
console.log(allTest.slice());

//Real-time use
//Running smoke suite from large suite
let smokeSuite = allTest.slice(0, 2);
console.log(smokeSuite);

//splice and slice
//splice
//Purpose → Add/remove/replace element
//slice
//Purpose → Extract element

//Can add element
//splice → Yes
//slice → No

let test = [10, 20, 30, 40, 50];
//for (let i = 1; i < 3; i++) {
//    console.log(test[i]);
//}
console.log(test.slice(1, 3));
console.log(test.slice());
console.log(test.slice(0));

//Interview Points
```

```
//length → returns total elements  
//push/pop → end operations  
//shift/unshift → beginning operations  
//splice → add/remove/replace  
//slice → extract elements  
//for loop → most common in automation  
//for of → values  
//for in → indexes/keys
```

Program 3

```
//concat()  
//Use → Merge multiple arrays into single array  
  
let smoke = ["Login", "DashBoard"];  
let Regression = ["Reports", "Setting", "Profile"];  
let allSuites = smoke.concat(Regression);  
console.log(allSuites);  
console.log(allSuites.length);  
  
//Merge multiple arrays  
let Sanity = ["QuickCheck"];  
let fullSuites = smoke.concat(Regression, Sanity);  
console.log(fullSuites);  
console.log(fullSuites.length);  
  
//Real-time use  
//Combine smoke, regression and sanity suite  
let smokeSuite = ["Login", "Cart"];  
let regressionSuite = ["Orders", "Payment"];
```

```
let sanitySuite = ["Profile"];

let completeSuite = smokeSuite.concat(regressionSuite,
sanitySuite);

console.log(completeSuite);


//join()
//Use → Convert array into String

//Syntax
//array.join(separator)

let pathParts = ["Home", "User", "Documents"];

let path = pathParts.join("/");

console.log(path);


let csv = ["Name", "Age", "City"];

console.log(csv);

let path1 = csv.join(',');

console.log(path1);


//Real-time use
//Generate CSV report

let failedTC = ["TC101", "TC102", "TC103"];

console.log(failedTC.join(","));


//reverse()
//Use → Reverse array order

let order = [1, 2, 3, 4, 5];

order.reverse();

console.log(order);


let exectionLog = ["Step1", "Step2", "Step3"];

console.log(exectionLog.reverse());
```

```
//Real-time use
//Reverse execution flow

let priority = ["Low", "Medium", "High"];
priority.reverse();
console.log(priority);

//sort()
//Use → Sort array elements alphabetically

let names = ["Zebra", "Apple", "Mango", "Banana"];
console.log(names);
names.sort();
console.log(names);

let names1 = ["AZ", "AA", "AD", "AB"];
console.log(names1);
names1.sort();
console.log(names1);

//Real-time use
//Sort usernames alphabetically

let users = ["Virat", "Akash", "Rohit", "Aman"];
users.sort();
console.log(users);

//includes()
//Use → Check element exists or not

let supportBrowser = ["Chrome", "FireFox", "Safari", "Edge"];
console.log(supportBrowser.includes("Chrome")); //true
console.log(supportBrowser.includes("IF")); //false
```



```
let currentBrowser = "Chrome";

if (supportBrowser.includes(currentBrowser)) {
    console.log(`Running test on ${currentBrowser}`);
}

//Real-time use
//Validate supported browser

let supportedEnv = ["QA", "STG", "PROD"];

console.log(supportedEnv.includes("QA"));

//indexOf()
//Use → Find first index of matching element

let supportBrowser1 = ["Chrome", "FireFox", "Chrome", "Safari",
    "Edge", "FireFox"];

console.log(supportBrowser1.indexOf("FireFox"));
console.log(supportBrowser1.indexOf("Chrome"));
console.log(supportBrowser1.indexOf("Chrome1"));

//Real-time use
//Find failed test index

let result = ["PASS", "PASS", "FAIL", "PASS"];

console.log(result.indexOf("FAIL"));

//at()
//Use → Access element using positive or negative index

let data = [10, 20, 30, 40, 50];

//      0   1   2   3   4
//     -5  -4  -3  -2  -1

console.log(data.at(0)); //10
console.log(data.at(-1)); //50
console.log(data.at(-3)); //30
```

```
//Real-time use
//Get latest build

let builds = ["Build1", "Build2", "Build3"];

console.log(builds.at(-1));

//Interview Points

//concat → merge arrays

//join → convert array to string

//reverse → reverse array order

//sort → sort array alphabetically

//includes → check value exists or not

//indexOf → returns first matching index

//at → access element using negative index
```

Program 4

```
//MULTIDIMENSIONAL ARRAY / 2D Array

//Use → Store data in rows and columns format

let table = [
  ["Name", "Age", "City"], //Row 0
  ["Rohit", 39, "Mumbai"], //Row 1
  ["Rahul", 30, "Pune"], //Row 2
  ["Virat", 45, "Nashik"] //Row 3
];

//Access complete row

console.log(table[0]);

//Access specific element

console.log(table[2][0]); //Rahul
console.log(table[3][2]); //Nashik
console.log();
```

```
//Nested loop for multidimensional array
for (let row = 0; row < table.length; row++) {
    for (let col = 0; col < table[row].length; col++) {
        process.stdout.write(table[row][col] + " ");
    }
    console.log();
}

//Real-time use
//Store student records

let students = [
    ["Rohit", 80],
    ["Virat", 90],
    ["Rahul", 70]
];

console.log(students[1][0]); //Virat
console.log(students[1][1]); //90

//Matrix example

let Matrix = [
    [1, 2, 3], //R0
    [4, 5, 6], //R1
    [7, 8, 9] //R2
];

//Access specific element
console.log(Matrix[1][2]); //6

//length of row
console.log(Matrix[1].length);

//Total rows
console.log(Matrix.length);
console.log();
```

```
//Print matrix using nested loop
for (let row = 0; row < Matrix.length; row++) {
    for (let col = 0; col < Matrix[row].length; col++) {
        process.stdout.write(Matrix[row][col] + " ");
    }
    console.log();
}

//Real-time use
//Store execution status
let execution = [
    ["TC01", "PASS"],
    ["TC02", "FAIL"],
    ["TC03", "PASS"]
];

console.log(execution[1][1]); //FAIL

//Convert String Array to Number Array using map()
//map()
//Use → Transform each element and return new array
let arr = ["10", "20", "30"];
console.log(arr);

//Convert string values to number
let number = arr.map(Number);
console.log(number);

//Real-time use
//Convert API response string IDs into numbers
let apiIds = ["101", "102", "103"];
let convertedIds = apiIds.map(Number);
console.log(convertedIds);
```

//Interview Points

//2D Array → Array inside another array

//Nested loops used for multidimensional arrays

//table[row][col] → Access specific value

//map() → Transform array elements

//map(Number) → Convert string array into number array

STRINGS IN JAVASCRIPT

1. INTRODUCTION TO STRINGS

What is a String?

A string is a sequence of characters enclosed in quotes.
Characters can be letters, numbers, symbols, or spaces.
Strings represent TEXT data in JavaScript.

```
let greeting = "Hello World";  
let testId = "TC_001";  
let url = "https://example.com/login";  
  
console.log(greeting); // Output: Hello World  
console.log(typeof greeting); // Output: string
```

Why Strings are Important

In automation testing, almost everything is a string:

- URLs, page titles, error messages
- Locators (CSS selectors, XPath)
- API response bodies
- Test data (usernames, passwords, emails)
- Assertions (expected vs actual text)

Real-Time Use Cases [in](#) Automation

1. Validate page title matches expected string
2. Extract text from web elements
3. Build dynamic locators [using](#) string concatenation
4. Parse API response strings (JSON)
5. Validate error messages on form submission
6. Generate dynamic test data (emails, usernames)
7. URL manipulation for different environments

Immutable means **cannot be changed after creation**.

When a string is created in JavaScript, its individual characters cannot be modified, added, or removed directly.

```
let name = "Hello";  
name[0] = "J";  
console.log(name);  
//Hello
```

Q. What does immutable mean?

Answer:

Immutable means the value cannot be changed after creation. In JavaScript, strings are immutable, so individual characters cannot be modified. Any operation like `replace()`, `slice()`, `trim()`, or `toUpperCase()` creates and returns a new string instead of modifying the original one.

Program 1

```
//String Basics in JavaScript
```

```
//JavaScript is dynamically typed
```

```
//A variable can hold different data types at different times
```

```
let strNum = "100"; //String
```

```
let num = 100; //Number
```

```
console.log(typeof strNum); //string
```

```
console.log(typeof num); //number
```

```
//String + Number = String Concatenation
```

```
console.log(strNum + num); //100100
```

```
console.log(strNum + 10); //10010
```

```
console.log(num + 10); //110
```

```
//Strings are Immutable
```

```
//Theory:
```

```
//Immutable means once a string is created, its characters cannot be changed
```

```
//Any modification creates a NEW string
```

```
let names = "Hello";
```

```
names[0] = "J"; //This does nothing  
console.log(names); //Hello
```

```
//Correct way to modify string
```

```
let newName = "J" + names.slice(1);  
console.log(newName); //Jello
```

```
//String Syntax
```

```
//1. Single Quotes
```

```
//Use → Create string using single quotes
```

```
let single = 'Hello World';  
console.log(single);
```

```
//2. Double Quotes
```

```
//Use → Create string using double quotes
```

```
let double = "Hello World";  
console.log(double);
```

```
//3. Backticks (Template Literals)
```

```
//Use → Multi-line strings and interpolation
```

```
let backticks = `Hello World`;  
console.log(backticks);
```


//String Interpolation

//Use → Insert variables inside string using \${}

```
let user = "Admin";
```

```
let message = `Welcome, ${user}`;
```

```
console.log(message);
```

//Interpolation with expressions

```
let price = 100;
```

```
let tax = 18;
```

```
let total = `Total : ${price + (price * tax / 100)}`;
```

```
console.log(total);
```

//Real-time Example

//Environment-based URL generation

```
let env = "QA";
```

```
let baseUrl = `https://opensource/${env}.orangehrmlive.com/web/index.php/auth/login`;
```

```
console.log(baseUrl);
```

//Automation Testing Examples

//SIT , UAT , ITF , STAGE , MTF , PERF

//QA , DEV , STG , PROD

//https://QA.myapp.com/api/v1

//https://STG.myapp.com/api/v1

```
//https://DEV.myapp.com/api/v1
```

```
//https://PROD.myapp.com/api/v1
```

```
let QATest = "QA";
```

```
let DEVTest = "DEV";
```

```
let STGTest = "STG";
```

```
let PRODTest = "PROD";
```

```
let baseUrlTest = `https://${QATest}.myapp.com/api/v1`;
```

```
console.log(baseUrlTest);
```

```
//Empty Strings
```

```
//Use → String with no visible characters
```

```
let empty1 = "";
```

```
let empty2 = " ";
```

```
let empty3 = ` `;
```

```
console.log(empty1.length); //0
```

```
console.log(empty2.length); //1
```

```
console.log(empty3.length); //2
```

```
//Theory
```

```
//Empty string and string containing spaces are different
```

```
//"" → Length = 0
```

```
/" " → Length = 1
```

```
//" " → Length = 2
```

```
//String Properties
```

```
//length property
```

```
//Use → Returns total number of characters
```

```
let text = "Hello world";
```

```
console.log(text.length); //11
```

```
//Calculation
```

```
//H e l l o _ w o r l d
```

```
//1 2 3 4 5 6 7 8 9 10 11
```

```
//String Indexing
```

```
//Each character has an index
```

```
// H e l l o
```

```
// 0 1 2 3 4
```

```
let word = "Hello";
```

```
console.log(word[0]); //H
```

```
console.log(word[1]); //e
```

```
console.log(word[2]); //l
```

```
console.log(word[3]); //l
```

```
console.log(word[4]); //o
```

```
//Get Last Character
```

```
console.log(word[word.length - 1]); //o
```

```
//Theory Points
```

```
//Strings are zero-indexed
```

```
//First character index = 0
```

```
//Last character index = length - 1
```

```
//Accessing invalid index returns undefined
```

```
console.log(word[10]); //undefined
```

```
//Interview Points
```

```
//String is a primitive datatype
```

```
//Strings are immutable
```

```
//String indexing starts from 0
```

```
//length returns total characters including spaces
```

```
//Template literals use backticks (`)
```

```
//Interpolation uses ${}
```

```
//typeof string returns "string"
```

```
//String + Number = Concatenation
```

```
//Number + Number = Arithmetic Addition
```

Program 2

```
// =====  
// TEMPLATE LITERALS (Multiline Strings)  
// =====  
  
// Modern way to create multiline strings using backticks ( ` ` )  
let multiline = `Line1  
Line2  
Line3`;  
  
console.log(multiline);  
  
// Old way using \n (newline character)  
console.log();  
  
let multiline2 = "Line1\nLine2\nLine3";  
console.log(multiline2);  
  
// Another old way using string concatenation  
console.log();  
  
let multiline3 = "Line1\n" +  
  "Line2\n" +  
  "Line3";  
  
console.log(multiline3);  
  
// =====  
// STRING PROPERTY  
// =====  
  
// length → Returns total number of characters in a string  
  
let text = "Hello World";  
console.log(text.length); // 11  
  
let emptyStr = "";  
console.log(emptyStr.length); // 0  
  
let spaces = "  ";  
console.log(spaces.length); // 3
```

```
// =====  
// ACCESSING CHARACTERS  
// =====
```

```
let word = "Hello";
```

```
// Access characters using index
```

```
console.log(word[0]); // H
```

```
console.log(word[1]); // e
```

```
console.log(word[2]); // l
```

```
console.log(word[3]); // l
```

```
console.log(word[4]); // o
```

```
// Index does not exist
```

```
console.log(word[5]); // undefined
```

```
// Last character
```

```
console.log(word[word.length - 1]); // o
```

```
// =====  
// STRING METHODS  
// =====
```

```
// 1. toUpperCase()
```

```
// Use: Convert text into uppercase.
```

```
// Real-time: Compare usernames, emails, etc.
```

```
let lower = "hello world";
```

```
let upper = lower.toUpperCase();
```

```
console.log(lower);
```

```
console.log(upper);
```

```
// 2. toLowerCase()
```

```
// Use: Convert text into lowercase.
```

```
// Real-time: Case-insensitive comparisons.
```

```
let upperStr = "HELLO WORLD";
```

```
let lowerStr = upperStr.toLowerCase();
```

```
console.log(upperStr);
```

```
console.log(lowerStr);
```

```
// 3. trim()
```

```
// Use: Remove spaces from both beginning and end.
```

```
// Real-time: Clean user input before validation.
```

```
let padded = " Hello World ";
```

```
console.log(padded);  
console.log(padded.trim());
```

```
// 4. trimStart()
```

```
// Use: Remove spaces only from beginning.
```

```
let startPadded = " Hello ";
```

```
console.log(startPadded.trimStart());
```

```
// 5. trimEnd()
```

```
// Use: Remove spaces only from end.
```

```
let endPadded = " Hello ";
```

```
console.log(endPadded.trimEnd());
```

```
// =====  
// SEARCHING METHODS  
// =====
```

```
// 6. includes()
```

```
// Use: Check if a substring exists.
```

```
// Returns true or false.
```

```
// Real-time: Validate messages, URLs, titles, etc.
```

```
let sentence = "The quick brown fox jumps over the lazy dog";
```

```
console.log(sentence.includes("fox")); // true  
console.log(sentence.includes("cat")); // false  
console.log(sentence.includes("Fox")); // false (case-sensitive)
```

```
let errorMessage =
```

```
    "Error: Invalid credentials. Please try again.";
```

```
if (errorMessage.includes("Invalid credentials")) {  
    console.log("Login error message displayed");  
}
```

```
// 7. indexOf()
```

```
// Use: Find FIRST occurrence position.  
// Returns -1 if not found.  
// Real-time: Search text.
```

```
let text2 = "Hello World Hello";
```

```
console.log(text2.indexOf("Hello")); // 0  
console.log(text2.indexOf("World")); // 6  
console.log(text2.indexOf("ABC")); // -1
```

```
// 8. lastIndexOf()  
// Use: Find LAST occurrence position.
```

```
let repeated = "abc xyz abc test abc";
```

```
console.log(repeated.lastIndexOf("abc")); // Last abc position  
console.log(repeated.lastIndexOf("xyz"));
```

```
// 9. startsWith()  
// Use: Check if string starts with specific text.  
// Real-time: URL validation.
```

```
let urlCheck = "https://www.facebook.com/";
```

```
console.log(urlCheck.startsWith("https")); // true  
console.log(urlCheck.startsWith("http://")); // false
```

```
// 10. endsWith()  
// Use: Check if string ends with specific text.  
// Real-time: File type validation.
```

```
let fileName = "report.html";
```

```
console.log(fileName.endsWith(".html")); // true  
console.log(fileName.endsWith(".pdf")); // false
```

```
// =====  
// EXTRACTING METHODS  
// =====
```

```
// 11. slice(start, end)  
// Use: Extract part of string.  
// Start = inclusive  
// End = exclusive
```



```
// Supports negative indexes.
```

```
let fullStr = "Hello World";  
//      01234567890
```

```
console.log(fullStr.slice(0, 5)); // Hello  
console.log(fullStr.slice(6));   // World  
console.log(fullStr.slice(-5));  // World  
console.log(fullStr.slice(-5, -1)); // Worl  
console.log(fullStr.slice(0, -6)); // Hello  
console.log(fullStr.slice(0));   // Entire string
```

```
// 12. substring(start, end)  
// Use: Similar to slice()  
// Does NOT support negative indexes.
```

```
let str8 = "Hello World";
```

```
console.log(str8.substring(0, 5)); // Hello  
console.log(str8.substring(-5));  // Hello World
```

```
// =====  
// CHARACTER METHODS  
// =====
```

```
// 13. charAt(index)  
// Use: Get character at specific position.
```

```
let str9 = "JavaScript";
```

```
console.log(str9.charAt(0)); // J  
console.log(str9.charAt(15)); // Empty string
```

```
// 14. at(index)  
// Modern version of charAt()  
// Supports negative indexes.
```

```
let str10 = "JavaScript";
```

```
console.log(str10.at(0)); // J  
console.log(str10.at(-1)); // t  
console.log(str10.at(-4)); // c
```

```
// =====
```

```
// REPLACE METHODS
// =====
```

```
// 15. replace()
// Use: Replace FIRST occurrence only.
```

```
let msg = "Hello World, Hello JavaScript";
```

```
let newStr = msg.replace("Hello", "Hi");
```

```
console.log(newStr);
// Hi World, Hello JavaScript
```

```
// 16. replaceAll()
// Use: Replace ALL occurrences.
```

```
let repeatedMsg = "foo bar foo baz foo";
```

```
let replaced = repeatedMsg.replaceAll("foo", "ABC");
```

```
console.log(replaced);
// ABC bar ABC baz ABC
```

Program 3.

```
// =====
// 17. split()
// =====
// Use: Convert a STRING into an ARRAY.
// Real-time: Read CSV data, split dates, parse API responses.
```

```
let csv1 = "apple,banana,cherry,date";
```

```
console.log(csv1.length);
```

```
let fruits = csv1.split(",");
```

```
console.log(fruits);
console.log(fruits.length);
```

```
// Split sentence into words
let sentence = "hello world javascript";
```

```
let words = sentence.split(" ");
```

```
console.log(words);
```

```
// Split string into characters
let chars = "hello".split("");

console.log(chars);

// Split date
let date = "02/06/2026";

let days = date.split("/");

console.log(days);
console.log(days[0]); // Day
console.log(days[1]); // Month
console.log(days[2]); // Year

// =====
// 18. join()
// =====
// Use: Convert ARRAY into STRING.
// Real-time: Create CSV files, build dynamic strings.

let fruitArr = ["apple", "banana", "cherry"];

console.log(fruitArr);

console.log(fruitArr.join(", "));
console.log(fruitArr.join("-"));
console.log(fruitArr.join(""));
console.log(fruitArr.join()); // Default separator = comma

// =====
// STRING CONCATENATION
// =====

let part1 = "Hello";
let part2 = "World";

// Using + operator
let combined = part1 + " " + part2;

console.log(combined);

// String + Number
console.log("Age: " + 25);
```

```
console.log("Score: " + 100 + 25);  
// Output: Score: 10025
```

```
console.log("Score: " + (100 + 25));  
// Output: Score: 125
```

```
// =====  
// 19. concat()  
// =====  
// Use: Join multiple strings together.
```

```
let s1 = "Hello";  
let s2 = " ";  
let s3 = "World";  
let s4 = "!";
```

```
let concatResult = s1.concat(s2, s3, s4);
```

```
console.log(concatResult);
```

```
// =====  
// TEMPLATE LITERALS (Best Practice)  
// =====  
// Use: Dynamic string creation.  
// Real-time: Build URLs, logs, messages.
```

```
let method = "GET";  
let endPoint = "users";
```

```
let fullUrl = `https://reqres.in/api/${endPoint}/2`;
```

```
console.log(`${method} ${fullUrl}`);
```

```
// =====  
// 20. STRING COMPARISON  
// =====
```

```
// == compares values only  
console.log("hello" == "hello");
```

```
// === compares value + datatype  
console.log("hello" === "hello");
```

```
console.log();
```

```
console.log("5" == 5); // true
console.log("5" === 5); // false
```

```
console.log("Hello" === "hello"); // false
console.log("Hello" === "Hello"); // true
```

```
// =====
// STRING ITERATION
// =====
```

```
let str = "Hello";
```

```
// 1. For Loop
// Use: When index is needed.
```

```
for (let i = 0; i < str.length; i++) {
  console.log(str[i]);
}
```

```
// =====
// 2. For...of Loop
// =====
// Use: Direct access to characters.
```

```
console.log();
```

```
let str2 = "Hello";
```

```
for (let char of str2) {
  console.log(char);
}
```

```
// =====
// 3. While Loop
// =====
```

```
console.log();
```

```
let str3 = "Hello";
```

```
let i = 0;
```

```
while (i < str3.length) {
  console.log(str3[i]);
  i++;
}
```

```
// =====  
// 4. For...in Loop  
// =====  
// Use: Access indexes.
```

```
let str4 = "Hello";
```

```
for (let index in str4) {  
    console.log(index + " " + str4[index]);  
}
```

```
// =====  
// REVERSE STRING - METHOD 1  
// =====  
// split() + reverse() + join()  
// Most popular interview solution.
```

```
function reverseString(str) {  
    return str.split("").reverse().join("");  
}
```

```
console.log(reverseString("Hello"));  
console.log(reverseString("JavaScript"));
```

```
// =====  
// REVERSE STRING - METHOD 2  
// FOR LOOP  
// =====
```

```
let str5 = "Hello";
```

```
let rev = "";
```

```
for (let i = str5.length - 1; i >= 0; i--) {  
    rev = rev + str5[i];  
}
```

```
console.log(rev);
```

```
// =====  
// REVERSE STRING - METHOD 3  
// FOR...OF LOOP  
// =====
```

```
let str6 = "Hello";
let rev1 = "";

for (let ch of str6) {
  rev1 = ch + rev1;
}

console.log(rev1);

// =====
// REVERSE STRING FUNCTION
// =====

function revString(str) {

  let reverse = "";

  for (let i = str.length - 1; i >= 0; i--) {
    reverse += str[i];
  }

  return reverse;
}

console.log(revString("JavaScript"));
console.log(revString("Playwright"));

// =====
// COUNT VOWELS
// =====
// Interview Question
// Real-time: Text analysis, validation.

function countVowels(str) {

  let vowels = "aeiouAEIOU";

  let count = 0;

  for (let char of str) {

    if (vowels.includes(char)) {
      count++;
    }
  }
}
```

```

    return count; // Correct return
}

```

```

console.log(countVowels("Automation Testing"));
console.log(countVowels("JavaScript"));

```

```

// =====
// OUTPUT
// =====
// Automation Testing -> 8 vowels
// JavaScript -> 3 vowels

```

```

/*
| Method      | Use in Automation Testing |
| ----- | ----- |
| `length`    | Validate text length, Order ID length, Phone Number length |
| `toUpperCase()` | Convert text to uppercase before comparison |
| `toLowerCase()` | Perform case-insensitive validation |
| `trim()`     | Remove extra spaces from UI text before validation |
| `trimStart()` | Remove leading spaces from input data |
| `trimEnd()`  | Remove trailing spaces from input data |
| `includes()` | Verify success/error messages contain expected text |
| `indexOf()`  | Check whether expected text exists in a string |
| `lastIndexOf()` | Find last occurrence of repeated text |
| `startsWith()` | Validate application URLs start with `https` |
| `endsWith()` | Validate downloaded file extensions (`.pdf`, `.xlsx`, `.csv`) |
| `slice()`    | Extract Order ID, PO Number, Invoice Number |
| `substring()` | Extract a portion of text from UI/API response |
| `charAt()`   | Validate character at a specific position |
| `at()`       | Access characters from beginning or end of string |
| `replace()`  | Replace part of dynamic test data |
| `replaceAll()` | Remove or replace unwanted characters from data |
| `split()`    | Convert dates, CSV values, or API data into arrays |
| `join()`     | Convert array data into a string for validation |
| `concat()`   | Combine strings to build URLs or messages |
*/

```


OBJECTS IN JAVASCRIPT

/What is an Object?

An object is a collection of KEY-VALUE pairs (properties).

It stores related data and functionality together.

Think of it as a real-world entity with attributes and behaviors.

```
let car = {  
  brand: "Toyota",    // key: "brand", value: "Toyota"  
  model: "Camry",     // key: "model", value: "Camry"  
  year: 2024,         // key: "year", value: 2024  
  isElectric: false   // key: "isElectric", value: false  
};  
  
console.log(car);  
// Output: { brand: "Toyota", model: "Camry", year: 2024, isElectric: false }
```

Why Objects are Important

In automation testing, objects are used EVERYWHERE:

```
// 1. Store test configurations (browser, env, credentials)  
// 2. Represent API request/response bodies (JSON)  
// 3. Page Object Model (POM) pattern  
// 4. Store locators for elements  
// 5. Handle test data (user profiles, products)  
// 6. Environment configurations
```

```
// Real-Time Use Cases
```

```
// API Response:  
let apiResponse = {  
  status: 200,  
  message: "Success",  
  data: {  
    id: 1,  
    name: "User1"  
  }  
};
```

```
// Test Configuration:  
let testConfig = {  
  browser: "chrome",  
  headless: false,  
  timeout: 30000
```

```
};

// User Credentials:
let credentials = {
  username: "admin",
  password: "Pass@123"
};
```

Program 1

```
// =====
// OBJECTS IN JAVASCRIPT
// =====

// Object = Collection of key-value pairs
// Key = Property Name
// Value = Property Value

let car = {
  brand: "Seltos", // Key : Value pair
  model: "kia",
  year: 2020
}

console.log(car);

// =====
// DIFFERENCE BETWEEN VARIABLE, ARRAY & OBJECT
// =====

// Variable → Stores ONE value
// Array → Stores multiple values in order (access by index)
// Object → Stores multiple values with names (access by key)

// Key (Property Name) → Always string or symbol
// Value → Can be any datatype

let mixed = {
  name: "Test",      // String
  count: 5,          // Number
  active: true,      // Boolean
  tags: ["Smoke"],   // Array
  config: {},        // Object
  run: function () { } // Function
}

console.log(mixed);
```

```
// =====  
// OBJECT CREATION  
// =====  
  
// Syntax:  
// let/const objName = {  
//   key1 : value1,  
//   key2 : value2  
// }  
  
let user = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 30  
};  
  
// Accessing values using Dot Notation  
// objectName.keyName  
  
console.log(user);  
console.log(user.firstName);  
console.log(user.age);  
console.log(user.lastName);  
  
// =====  
// EMPTY OBJECT  
// =====  
  
let emptyObj = {};  
  
console.log(emptyObj);  
  
// Object.keys() returns all keys as array  
// .length returns number of properties  
  
console.log(Object.keys(emptyObj).length); // 0  
  
// Adding properties dynamically  
  
emptyObj.firstname = "Added FirstName Later";  
emptyObj.LastName = "Added LastName Later";  
  
console.log(emptyObj);  
  
// Now object contains 2 properties
```

```
console.log(Object.keys(emptyObj).length);
```

```
// =====  
// NESTED OBJECTS  
// =====
```

```
// Object inside another object
```

```
let employee = {  
  name: "Rohit",  
  department: "QA",  
  
  address: {  
    city: "Pune",  
    state: "Maha",  
    zip: "404287"  
  },  
  
  skills: {  
    primary: "Javascript",  
    secondary: "Playwright"  
  }  
}
```

```
// Accessing nested object properties
```

```
console.log(employee.name);  
console.log(employee.department);  
console.log(employee.address.city);  
console.log(employee.address.state);  
console.log(employee.skills.primary);
```

```
// =====  
// OBJECT WITH MULTIPLE DATA TYPES  
// =====
```

```
let fullObject = {  
  name: "Test Suite",           // String  
  totalTests: 50,               // Number  
  isActive: true,              // Boolean  
  tags: ["Smoke", "Regression"], // Array  
  config: { timeout: 5000 },    // Object  
  
  execute: function () {       // Function  
    return "Running.."  
  }
```

```
    },  
  
    lastRun: null,           // Null  
    owner: undefined        // Undefined  
  }  
  
  // typeof → Returns datatype  
  
  console.log(typeof fullObject.name);  
  console.log(typeof fullObject.isActive);  
  
  // =====  
  // CREATING OBJECTS  
  // =====  
  
  // Method 1 : Object Literal (Most Common)  
  
  let literal = {  
    name: "chrome",  
    version: "120"  
  }  
  
  console.log(literal);  
  
  // Method 2 : Using new Object()  
  
  let constructor = new Object();  
  
  constructor.name = "Chrome";  
  constructor.version = 119;  
  
  console.log(constructor);  
  
  // =====  
  // ACCESSING OBJECT PROPERTIES  
  // =====  
  
  // Dot Notation (Most Common)  
  
  let product = {  
    name: "Laptop",  
    price: 99999,  
    inStock: true,  
  }
```

```
console.log(product.name);  
console.log(product.price);  
console.log(product.inStock);
```

```
// Bracket Notation  
// Useful when property name is dynamic
```

```
console.log(product["name"]);  
console.log(product["price"]);  
console.log(product["inStock"]);
```

```
// =====  
// NESTED OBJECT EXAMPLE  
// =====
```

```
let response = {  
  status: 200,  
  
  data: {  
    user: {  
      id: 1,  
      name: "Rohit",  
  
      address: {  
        city: "Mumbai",  
        pin: "448484"  
      }  
    }  
  }  
};
```

```
// Adding new property to nested object
```

```
response.data.user.id1 = "Added FirstName Later";  
  
console.log(response);
```

```
// =====  
// ADD NEW PROPERTY  
// =====
```

```
let testResult = {  
  name: "Login Test"  
};
```

```
// Adding properties dynamically
```

```
testResult.status = "Pass";  
testResult.Duration = 2.5;  
testResult.timeStamp = new Date().toISOString();
```

```
console.log(testResult);
```

```
// =====  
// UPDATE EXISTING PROPERTY  
// =====
```

```
let userProfile = {  
  name: "Virat",  
  
  preference: {  
    notifications: true  
  }  
};
```

```
// Updating nested property value
```

```
userProfile.preference.notifications = false;
```

```
console.log(userProfile);
```

```
// =====  
// DELETE PROPERTY  
// =====
```

```
let temp = {  
  a: 1,  
  b: 2,  
  c: 3,  
  d: 4  
};
```

```
console.log(temp);
```

```
// Delete property c
```

```
delete temp.c;
```

```
console.log(temp);
```

```
// Delete property d
```

```

console.log(delete temp['d']);

console.log(temp);

// Delete non-existing property

console.log(delete temp['x']);

// If property does not exist,
// delete still returns true

// delete only removes object properties
// It does not delete variables declared with let/const

delete temp;

console.log(temp);

```

Program 2

```

// =====
// OBJECT PROPERTIES AND METHODS
// =====

// Property = Key-Value pair where value is DATA

let laptop = {
  brand: "Dell",    // Property (String Data)
  model: "XPS i7",  // Property (String Data)
  price: 50000      // Property (Number Data)
}

// =====
// METHOD
// =====

// Method = A key-value pair where value is a FUNCTION

let calculator = {
  add: function (a, b) {
    return a + b;
  },
  subtract: function (a, b) {
    return a - b;
  },
  multiply: function (a, b) {
    return a * b;
  }
}

```



```

    }
  }
}

```

// Calling object methods

```

console.log(calculator.add(5, 3));    // 8
console.log(calculator.subtract(5, 3)); // 2
console.log(calculator.multiply(5, 3)); // 15

```

```

// =====
// THIS KEYWORD
// =====

```

// this refers to the current object that owns the method

```

let userAccount = {
  name: "RAM",
  balance: 1000,

  deposit(amount) {
    // this.balance refers to balance property
    // of current object

    this.balance += amount;

    return `Deposited ${amount}. New Balance : ${this.balance}`;
  },

  getInfo() {
    return `${this.name} has ${this.balance}`;
  }
}

```

```

console.log(userAccount.deposit(5000));
console.log(userAccount.getInfo());

```

```

// =====
// OBJECT ITERATION
// =====

```

```

let student = {
  name: "Ram",
  age: 25,
  grade: "A",
  city: "Pune"
}

```

// for...in loop

```
// Used to iterate through object keys

console.log("--- for...in loop ---");

for (let Test in student) {
  // student[Test] gives corresponding value

  console.log(`${Test} : ${student[Test]}`);
}

// =====
// Object.keys()
// =====

// Returns an array of all object keys

console.log("--- Object.keys() ---");

let keys = Object.keys(student);

console.log(keys);

// =====
// Object.values()
// =====

// Returns an array of all object values

console.log("--- Object.values() ---");

let values = Object.values(student);

console.log(values);

// =====
// Object.entries()
// =====

// Returns array of [key,value] pairs

console.log("--- Object.entries() ---");

let entries = Object.entries(student);
console.log(entries);

// Destructuring key and value from entries

for (let [Demo, Test] of Object.entries(student)) {
```

```
    console.log(`${Demo} = ${Test}`);
  }

// =====
// NESTED OBJECT
// =====

// Object inside object inside object

let userApiResponse = {
  status: 200,
  message: "Success",
  data: {
    user: {
      id: 101,
      profile: {
        firstName: "Rohit",
        LastName: "Sharma",
        email: "Rohit@gmail.com"
      },
      permission: {
        canRead: true,
        canWrite: true,
        canDelete: false
      },
      orders: [
        {
          orderId: "ORDER01",
          amount: 500
        },
        {
          orderId: "ORDER02",
          amount: 700
        }
      ]
    }
  }
}

// =====
// ACCESSING NESTED OBJECT PROPERTIES
// =====

// Access email from nested profile object

console.log(userApiResponse.data.user.profile.email);

// Access second order ID
```

```
console.log(userApiResponse.data.user.orders[1].orderId);
```

```
// Access second order amount
```

```
console.log(userApiResponse.data.user.orders[1].amount);
```

```
// Access complete second order object
```

```
console.log(userApiResponse.data.user.orders[1]);
```

```
// =====
```

```
// REAL-TIME AUTOMATION USE CASES
```

```
// =====
```

```
// Object Properties
```

```
// → Store test data
```

```
// Object Methods
```

```
// → Create reusable utility functions
```

```
// this Keyword
```

```
// → Access current object properties
```

```
// Object.keys()
```

```
// → Validate API response fields
```

```
// Object.values()
```

```
// → Validate returned data
```

```
// Object.entries()
```

```
// → Iterate key-value pairs
```

```
// Nested Objects
```

```
// → Handle API responses
```

```
// Arrays inside Objects
```

```
// → Handle Orders, Products, Users etc.
```

```
// for...in Loop
```

```
// → Read all properties dynamically
```

1. Introduction to OOPs

What is OOPs?

OOPS (Object-Oriented Programming System) is a programming paradigm that organizes software design around **objects** rather than functions and logic. It uses the concept of **objects** which contain **data (properties)** and **code (methods)**.

Definition: OOPS is a methodology to design a program using classes and objects. It simplifies software development and maintenance by providing concepts like encapsulation, inheritance, polymorphism, and abstraction.

Why OOPS is Important

- **Code Reusability** – Write once, use many times through inheritance
- **Modularity** – Code is organized into separate classes
- **Security** – Data hiding through encapsulation
- **Easy Maintenance** – Changes in one class don't affect others
- **Real-World Modeling** – Maps directly to real-world entities
- **Scalability** – Easy to extend and add new features
- **Team Collaboration** – Different developers can work on different classes

Features of OOPs

Feature	Description
Class	Blueprint/template for creating objects
Object	Instance of a class
Encapsulation	Wrapping data and methods together
Inheritance	Acquiring properties from parent class
Polymorphism	Same method behaving differently
Abstraction	Hiding implementation details

Advantages of OOPs

1. Code reusability through inheritance
2. Data security through encapsulation
3. Easy to manage and maintain complex code
4. Supports modular programming
5. Faster development with existing code
6. Better problem-solving approach
7. Maps directly to real-world problems

Real-Time Usage in Software Companies

- **Automation Testing** – Page Object Model (POM) uses classes and objects
- **Web Development** – React components are class-based
- **API Development** – Models/Controllers are classes
- **Mobile Apps** – Every screen is a class
- **Framework Design** – Selenium, Playwright use OOPs heavily

Real-World Examples of OOPs

- **Car:** Class = Car Blueprint, Object = Your specific Honda City
- **Human:** Class = Human, Object = "Rahul" with properties (name, age)
- **Bank:** Class = Account, Object = Your savings account
- **Phone:** Class = Mobile, Object = Your iPhone 15

2. Class in Detail

Definition of Class

A **class** is a **blueprint** or **template** that defines the structure and behavior of objects. It describes what properties and methods an object will have.

Simple Definition: A class is like an architectural plan – it tells you WHAT to build but is not the building itself.

Syntax of Class in JavaScript

```
class ClassName {  
  
    // Constructor - initializes object properties  
  
    constructor(parameter1, parameter2) {  
        this.property1 = parameter1;  
        this.property2 = parameter2;  
    }  
  
    // Method - defines object behavior  
    methodName() {  
        // code  
    }  
}
```

Characteristics of Class

1. **Blueprint** – Template for objects

2. **Logical Entity** – Does not occupy memory until an object is created
3. **Contains** – Properties (variables) + Methods (functions)
4. **Reusable** – Can create multiple objects from one class
5. **Extensible** – Can be inherited by child classes

Why Classes are Used

- To organize code logically
- To create reusable templates
- To maintain clean architecture

Program 1

```
/*
# Object-Oriented Programming (OOPs) in JavaScript

## 1. What is OOPs?
**OOPs (Object-Oriented Programming System)** is a programming paradigm that organizes
software
design around **objects** rather than functions and logic.
```

Objects contain:

```
* **Properties (Data/Variables)** → Characteristics of an object
* **Methods (Functions)** → Actions performed by an object
```

Definition

OOPs is a methodology used to design programs using **Classes** and **Objects**. It helps in creating reusable, maintainable, and scalable applications.

Why OOPs?

```
* Code Reusability
* Easy Maintenance
* Better Code Organization
* Real-world Modeling
* Improved Security
* Faster Development
```

Real-Time Examples

Real World Object	Properties	Methods
Student	Name, Roll No, Grade	Attend Class, Give Exam
Car	Brand, Model, Color	Start(), Stop()
Bank Account	Account No, Balance	Deposit(), Withdraw()
Employee	Name, ID, Salary	Work(), ApplyLeave()

2. Class in Detail

Definition of Class

A ****Class**** is a blueprint or template used to create objects.

A class defines:

- * What data an object will contain
- * What actions an object can perform

Simple Definition

A class is like an architectural blueprint of a house.

- * Blueprint = Class
- * Actual House = Object

The blueprint describes the house, but the actual house is built from that blueprint.

Syntax of Class in JavaScript

```
``javascript
class ClassName {
  // Constructor
  constructor(parameter1, parameter2) {
    this.property1 = parameter1;
    this.property2 = parameter2;
  }
  // Method
  methodName() {

  }
}
``
*/
// Class = Blueprint for creating Student objects
class Student {
  // Constructor
  // Automatically called when object is created
  // Used to initialize object properties

  constructor(name, rollNo, grade) {
```



```
// Instance Variables / Properties
this.Test = name;
this.rollNo = rollNo;
this.grade = grade;
this.attendance = 0;
}
```

```
// Method
// Used to increase attendance
```

```
markAttendance() {
  this.attendance++;
  console.log(
    `${this.Test} attendance marked, Total: ${this.attendance}`
  );
}
```

```
// Method
// Display student details
```

```
getResult() {
  console.log(
    `${this.Test} (Roll No: ${this.rollNo}) - Grade ${this.grade}`
  );
}
```

```
// Method
// Check eligibility for exam
```

```
isEligibleForExam() {
  return this.attendance >= 75;
}
}
```

```
// Creating Objects
```

```
// Syntax
```

```
// let/const variableName = new ClassName();
```

```
const student1 = new Student("Rahul", 101, "A");
```

```
const student2 = new Student("Rohit", 102, "B+");
...
```

```
### What Happens Internally?
```

```
const student1 = new Student("Rahul", 101, "A");
```

Steps:

1. Memory allocated for object
2. Constructor automatically called
3. Properties initialized
4. Object returned and assigned to student1

```
...
```

```
// Calling Methods
```

```
student1.markAttendance();
```

```
student2.getResult();
```

```
// Output:
```

```
// Rahul attendance marked, Total: 1
```

```
// Rohit (Roll No: 102) - Grade B+
```

Program 3

```
// =====  
// BANK ACCOUNT MANAGEMENT SYSTEM  
// =====
```

```
// Class = Blueprint for creating Bank Account objects
```

```
class BankAccount {
```

```
    // Constructor
```

```
    // Automatically called when object is created
```

```
    // Used to initialize object properties
```

```
    constructor(accountHolder, accountNumber, balance) {
```

```
        this.accountHolder = accountHolder;
```

```
        this.accountNumber = accountNumber;
```

```
        this.balance = balance;
```

```
        // Array to store transaction history
```

```
        this.transactions = [];
```

```
    }
```

```
    // Deposit Method
```

```
    // Adds money to account balance
```

```
    // Records transaction in statement
```

```

deposit(amount) {
  this.balance = this.balance + amount;

  this.transactions.push({
    type: 'credit',
    amount,
    date: new Date()
  });

  console.log(`₹${amount} deposited. New Balance : ₹${this.balance}`);
}

// Withdraw Method
// Withdraw money from account
// Checks sufficient balance before withdrawal

withdraw(amount) {

  if (amount > this.balance) {
    console.log("Insufficient balance!");
    return;
  }

  this.balance = this.balance - amount;

  this.transactions.push({
    type: 'debit',
    amount,
    date: new Date()
  });

  console.log(`₹${amount} withdrawn. New Balance : ₹${this.balance}`);
}

// Get Balance Method
// Displays current account balance

getBalance() {
  console.log(`Account : ${this.accountNumber} | Balance : ₹${this.balance}`);
}

// Get Statement Method
// Displays complete transaction history

getStatement() {
  console.log(`---- Statement for ${this.accountHolder} ----`);
}

```

```

    for (const transaction of this.transactions) {
        console.log(`${transaction.type} : ₹${transaction.amount}`);
    }
}
}

```

```

// =====
// OBJECT CREATION
// =====

```

```

// Syntax:
// const objectName = new ClassName(parameters);

```

```

const account1 = new BankAccount(
    "Rahul Sharma",
    "SBI0001234",
    50000
);

```

```

// =====
// METHOD CALLING
// =====

```

```

// Syntax:
// objectName.methodName(parameters);

```

```

account1.deposit(10000);
account1.withdraw(8000);
account1.getBalance();
account1.withdraw(80000);
account1.getBalance();
account1.withdraw(6000);

```

```

console.log();

```

```

// Create another account object

```

```

const account2 = new BankAccount(
    "Virat",
    "SBI00004567",
    30000
);

```

```

account2.deposit(20000);

```

```

// Print statement for account1

```

```

account1.getStatement();

```

```
// =====  
// OOPS CONCEPTS USED  
// =====  
  
// Class  
// → BankAccount  
  
// Object  
// → account1, account2  
  
// Constructor  
// → Initializes account details  
  
// Properties  
// → accountHolder  
// → accountNumber  
// → balance  
// → transactions  
  
// Methods  
// → deposit()  
// → withdraw()  
// → getBalance()  
// → getStatement()  
  
// this Keyword  
// → Refers to current object  
  
// Encapsulation  
// → Data and methods are grouped together  
  
// Array Inside Object  
// → transactions stores transaction history  
  
// Object Inside Array  
// → Each transaction is stored as an object  
  
// =====  
// REAL-TIME USE CASES  
// =====  
  
// Banking Applications  
// ATM Systems  
// E-Commerce Applications  
// Playwright Framework Classes  
// Page Object Model (POM)  
// API Response Handling
```

Program 3

```
// =====  
// PRODUCT MANAGEMENT SYSTEM  
// =====  
  
// What is a Class?  
// A Class is a blueprint or template used to create objects.  
// It defines what properties (data) and methods (functions)  
// an object will have.  
  
// Example:  
// Product = Class  
// laptop = Object  
  
class Product {  
  
    // Constructor  
    // Automatically called when an object is created.  
    // Used to initialize object properties.  
  
    constructor(productId, name, price, category, stock) {  
  
        this.productId = productId;  
        this.name = name;  
        this.price = price;  
        this.stock = stock;  
        this.category = category;  
  
        // Empty array to store product ratings  
  
        this.ratings = [];  
    }  
  
    // Apply Discount Method  
    // Calculates discounted price  
  
    applyDiscount(percent) {  
  
        const discountedPrice =  
            this.price - (this.price * percent / 100);  
  
        console.log(  
            ` ${this.name} : ₹${this.price} --> ₹${discountedPrice} (${percent}% off)`  
        );  
    }  
}
```

```
    return discountedPrice;
}

// Add Rating Method
// Adds rating to ratings array

addRating(rating) {

    this.ratings.push(rating);

    console.log(rating + "*");

    // Average Rating Logic

    // const avg =
    // this.ratings.reduce((a, b) => a + b, 0)
    // / this.ratings.length;

    // console.log(
    // `${this.name} - New Rating : ${rating}*
    // | Average : ${avg.toFixed(1)}*`
    // );
}

// Check Stock Method

isInstock() {

    console.log("Stock 10");
}

// Purchase Method
// Reduces stock after purchase

purchase(quantity) {

    if (quantity > this.stock) {

        console.log("Not enough stock");

        return false;
    }

    this.stock -= quantity;

    console.log(
        `${quantity} x ${this.name} purchased. Remaining : ${this.stock}`
    );
}
```

```
    );  
  }  
}  
  
// =====  
// OBJECT CREATION  
// =====  
  
// Syntax:  
// const objectName = new ClassName();  
  
const laptop = new Product(  
  "P001",  
  "MacBook Pro",  
  150000,  
  "Electronics",  
  50  
);  
  
// =====  
// METHOD CALLING  
// =====  
  
// Apply 15% discount  
  
laptop.applyDiscount(15);  
  
// Add product rating  
  
laptop.addRating(4);  
  
// Purchase 10 quantities  
  
laptop.purchase(10);  
  
// Check stock  
  
laptop.iInstock();  
  
// =====  
// OOPS CONCEPTS USED  
// =====  
  
// Class  
// → Product  
  
// Object  
// → laptop
```



```
// Constructor
// → Initializes product details

// Properties
// → productId
// → name
// → price
// → category
// → stock
// → ratings

// Methods
// → applyDiscount()
// → addRating()
// → purchase()
// → iStock()

// this Keyword
// → Refers to current object

// Encapsulation
// → Data and methods are grouped together

// Array Inside Object
// → ratings stores all ratings

// Object Creation
// → new Product()

// =====
// REAL-TIME USE CASES
// =====

// E-Commerce Applications
// Amazon
// Flipkart
// Myntra
// Inventory Management Systems
// Product Catalog Systems
// Shopping Cart Applications

// Similar concepts are used in
// Playwright Framework Classes
// Page Object Model (POM)
// API Automation Frameworks
```

Program 4

```
// =====
// STUDENT MANAGEMENT SYSTEM
// =====

// Class = Blueprint
// Student = Class

class Student {

    // Constructor
    // Used to initialize object properties

    constructor(name, rollNo, course) {
        this.name = name;
        this.rollNo = rollNo;
        this.course = course;
    }

    // Method to display student details

    displayStudentInfo() {
        console.log("----- Student Details -----");
        console.log(`Name : ${this.name}`);
        console.log(`Roll No : ${this.rollNo}`);
        console.log(`Course : ${this.course}`);
    }

    // Method to simulate attending class

    attendClass() {
        console.log(`${this.name} is attending JavaScript class.`);
    }
}

// =====
// OBJECT CREATION
// =====

const student1 = new Student("Rahul Sharma", 101, "Playwright Automation");

// =====
// METHOD CALLING
// =====

student1.displayStudentInfo();
```

```
student1.attendClass();
```

```
// =====
// OUTPUT
// =====
```

```
// ----- Student Details -----
```

```
// Name : Rahul Sharma
```

```
// Roll No : 101
```

```
// Course : Playwright Automation
```

```
// Rahul Sharma is attending JavaScript class.
```

Constructor

```
// =====
// CONSTRUCTOR IN JAVASCRIPT
// =====
```

```
// Constructor
```

```
// A constructor is a special method that is
// automatically called when a new object is
// created from a class.
```

```
// Purpose:
```

```
// Used to initialize object properties.
```

```
// =====
// 1. DEFAULT CONSTRUCTOR (No Parameters)
// =====
```

```
class Animal {
  constructor() {
    this.type = "Unknown";
    this.sound = "...";

    console.log("Animal created with default values");
  }
}
```

```
// Creating Object
```

```
const dog = new Animal();
```

```
// Accessing Properties
```

```

console.log(dog.type);
console.log(dog.sound);

const animal = new Animal();

console.log(animal.type);

// =====
// PARAMETERIZED CONSTRUCTOR
// =====

// Constructor with parameters

class Animal1 {
    constructor(type, name, sound) {
        this.type = type;
        this.name = name;
        this.sound = sound;

        console.log(`${this.sound} the ${type} created!`);
    }

    makeSound() {
        console.log(`${this.name} says : ${this.sound}`);
    }
}

// Creating Object

const dog1 = new Animal1(
    "Dog",
    "Brown",
    "Woof"
);

dog1.makeSound();

console.log();

// =====
// CONSTRUCTOR EXECUTION FLOW
// =====

class Example {
    constructor() {
        console.log("Step 1 : Constructor Called");

        // Property initialization

        console.log("Step 2 : Property Initialized");
    }
}

```

```
console.log("Before object creation");

const obj = new Example();

console.log("After object creation");

// What happens internally?

// Step 1:
// A new empty object {} is created

// Step 2:
// this points to the new object

// Step 3:
// Constructor code executes

// Step 4:
// Object is returned

// =====
// OBJECT CREATION SYNTAX
// =====

// let/const variableName = new ClassName()

// =====
// THIS KEYWORD
// =====

// this refers to the current object

class Calculator {
  constructor(initialValue = 0) {
    // this refers to newly created object
    this.result = initialValue;
  }

  add(num) {
    return num + num;
  }
}

const c2 = new Calculator();

let result = c2.add(10);

console.log(result);
```

```

// =====
// CLASS, OBJECT & METHODS
// =====

class D1 {
    constructor() {
        console.log("JavaScript");
    }

    name(params) {

    }

    Demo() {
        console.log("Hi");
    }

    Demo1() {
        console.log("Hello");
    }
}

// Object Creation

const d1 = new D1();

// Method Calling

d1.Demo1();

d1.Demo();

// =====
// IMPORTANT INTERVIEW QUESTIONS
// =====

// Can we have multiple constructors
// in JavaScript?

// No.
// JavaScript allows only ONE constructor
// inside a class.

// What happens if constructor is not defined?

// JavaScript automatically provides
// a default constructor.

// constructor() {}

// =====
// FUNCTION VS METHOD

```

```
// =====

// Function
// A reusable block of code that performs
// a specific task and can be called
// independently.

// Example

function add(a, b) {
    return a + b;
}

// Method
// A function that belongs to a class
// or object.

// Example

class Test {
    display() {
        console.log("Hello");
    }
}

// Difference

// Function
// ✓ Does not require class
// ✓ Called directly

// Method
// ✓ Belongs to class/object
// ✓ Called using object reference

// =====
// OOPS PRINCIPLES
// =====

// 1. Inheritance
// Child class acquires properties
// and methods of parent class.

// 2. Encapsulation
// Data and methods are wrapped
// together into a single unit.

// 3. Abstraction
// Hides implementation details
// and shows only essential features.

// 4. Polymorphism
```

```
// Same method behaves differently
// in different situations.

// =====
// SUMMARY
// =====

// Class
// → Blueprint

// Object
// → Instance of class

// Constructor
// → Initializes object properties

// this
// → Refers to current object

// Function
// → Independent block of code

// Method
// → Function inside class/object

// OOPS
// → Inheritance
// → Encapsulation
// → Abstraction
// → Polymorphism
```


5. OOPs Principles in Detail

📌 A. Encapsulation

◆ Definition ****Encapsulation**** is the bundling of data (properties) and methods that operate on that data into a single unit (class), while ****restricting direct access**** to some of the object's components.

Simple Definition: Encapsulation = Data Hiding + Data Binding. Like a capsule that wraps medicine inside, encapsulation wraps data inside a class.

```
// =====
// 1. ENCAPSULATION
// =====

// Definition:
// Encapsulation is the process of wrapping data (properties)
// and methods (functions) into a single unit called a Class.
//
// It also helps in restricting direct access to internal data,
// which prevents accidental modification of important values.
//
// Benefits:
// ✓ Data Hiding
// ✓ Better Security
// ✓ Better Control over Data
// ✓ Easier Maintenance
// ✓ Improves Code Quality

// =====
// WITHOUT ENCAPSULATION
// =====

class BankAccount {
    constructor(balance) {
        this.balance = balance;    // Public property
    }
}

const acc = new BankAccount(5000);

console.log("Initial Balance :", acc.balance);

// Anyone can modify the balance directly
acc.balance = -10000;    // Invalid value
acc.balance = 400;      // Value changed

console.log("Updated Balance :", acc.balance);
```

```

// Problem:
// Since balance is public,
// anyone can access and modify it directly.
// There is no validation or protection.

// =====
// WITH ENCAPSULATION
// =====

class BankAccount1 {

    // Private Property
    // Private members can only be accessed
    // inside the same class.
    #balance;

    constructor(balance) {

        // Local variable
        let test = 0;

        this.#balance = balance;
    }

    // Getter Method
    // Used to read private data
    getBalance() {
        return this.#balance;
    }
}

const acc1 = new BankAccount1(5000);

// Direct access is not allowed

// acc1.#balance = 3000;          // SyntaxError
// console.log(acc1.#balance);    // SyntaxError

console.log("Current Balance :", acc1.getBalance());

// =====
// IMPORTANT INTERVIEW POINTS
// =====

// 1. What is Encapsulation?
//     Wrapping data and methods into a single unit (Class)
//     and restricting direct access to data.

// 2. Why do we use Encapsulation?

```

```
// To protect data from unauthorized access
// and maintain data integrity.

// 3. How is Encapsulation achieved in JavaScript?
// Using private fields (#),
// getters, and setters.

// 4. What is a Private Property?
// A property declared using #
// that cannot be accessed outside the class.

// Example:
// #balance;

// 5. Can we access private properties directly?
// No.
// It will throw a SyntaxError.

// Example:
// acc1.#balance; ❌

// 6. How can we access private properties?
// Through public methods like getters and setters.

// Example:
// getBalance()

// Real-Life Example:
// ATM Machine
//
// You cannot directly access or modify the money
// stored inside a bank account.
// You can only interact through controlled operations
// such as Deposit, Withdraw, and Check Balance.
//
// This is a perfect example of Encapsulation.
```

Program 2

```
// =====
// BANK ACCOUNT MANAGEMENT SYSTEM
// USING ENCAPSULATION
// =====

// Encapsulation:
// Encapsulation is the process of wrapping data (properties)
// and methods (functions) into a single unit called a Class.
//
// It also restricts direct access to sensitive data and
```

```

// allows controlled access through methods.
//
// Benefits:
// ✓ Data Hiding
// ✓ Security
// ✓ Better Control Over Data
// ✓ Code Maintainability
// ✓ Prevents Unauthorized Access

class BankAccount {

    // =====
    // PRIVATE PROPERTIES
    // =====

    // Private properties can only be accessed
    // inside the same class.

    #balance;
    #pin;

    // =====
    // CONSTRUCTOR
    // =====

    // Constructor is a special method that is
    // automatically called when an object is created.

    constructor(holderName, initialBalance, pin) {

        // Public Property
        // Can be accessed directly outside the class
        this	holderName = holderName;

        // Private Properties
        this.#balance = initialBalance;
        this.#pin = pin;
    }

    // =====
    // CHECK BALANCE
    // =====

    // This method returns account balance
    // only if the correct PIN is provided.

    getBalance(pin) {

        // Validate PIN
        if (pin !== this.#pin) {
            return "Invalid PIN";
        }
    }
}

```

```

        return this.#balance;
    }

    // =====
    // DEPOSIT MONEY
    // =====

    // Used to add money to the account

    deposit(amount) {

        // Deposit amount must be greater than 0
        if (amount <= 0) {
            console.log("Invalid Amount");
            return;
        }

        // Add amount to balance
        this.#balance += amount;

        console.log(
            `Deposited: ₹${amount} | New Balance: ₹$
{this.#balance}`
        );
    }

    // =====
    // WITHDRAW MONEY
    // =====

    // Used to withdraw money from account

    withdraw(amount, pin) {

        // Verify PIN before withdrawal
        if (pin !== this.#pin) {
            console.log("Wrong PIN");
            return;
        }

        // Check available balance
        if (amount > this.#balance) {
            console.log("Insufficient Balance");
            return;
        }

        // Deduct amount
        this.#balance -= amount;

        console.log(

```

```

        `Withdrawn: ₹${amount} | New Balance: ₹$
{this.#balance}`
    );
}

// =====
// CHANGE PIN
// =====

// Extra method demonstrating encapsulation

changePin(oldPin, newPin) {
    if (oldPin !== this.#pin) {
        console.log("Incorrect Old PIN");
        return;
    }

    this.#pin = newPin;

    console.log("PIN changed successfully");
}

// =====
// ACCOUNT DETAILS
// =====

showAccountInfo() {
    console.log("Account Holder :", this.holderName);
}

}

// =====
// OBJECT CREATION
// =====

const account1 = new BankAccount(
    "Rohit",
    50000,
    1234
);

// =====
// PRIVATE FIELDS CANNOT BE ACCESSED DIRECTLY
// =====

// account1.#balance; // SyntaxError
// account1.#pin;      // SyntaxError

```

```
// Private fields are only accessible inside the class.

// =====
// ACCOUNT DETAILS
// =====

account1.showAccountInfo();

// =====
// DEPOSIT MONEY
// =====

account1.deposit(5000);

// =====
// WITHDRAW MONEY
// =====

account1.withdraw(8000, 1234);

// =====
// CHECK BALANCE
// =====

console.log(account1.getBalance(2344)); // Invalid PIN
console.log(account1.getBalance(1234)); // Correct Balance

// =====
// CHANGE PIN
// =====

account1.changePin(1234, 5678);

// =====
// IMPORTANT INTERVIEW QUESTIONS
// =====

// 1. What is Encapsulation?
//
// Encapsulation is the process of wrapping data and
// methods into a single unit called a class and
// restricting direct access to internal data.

// 2. How do we achieve Encapsulation in JavaScript?
```

```
//
// Using:
// ✓ Private Fields (#)
// ✓ Getters
// ✓ Setters
// ✓ Controlled Public Methods

// 3. What is a Private Field?
//
// A property declared using # which cannot be
// accessed outside the class.
//
// Example:
// #balance;
// #pin;

// 4. Why do we use Encapsulation?
//
// ✓ Protects sensitive data
// ✓ Prevents direct modification
// ✓ Improves security
// ✓ Improves maintainability

// 5. Real-Life Example
//
// ATM Machine
//
// You cannot directly access or modify your
// bank balance.
//
// You must use controlled operations such as:
// - Deposit
// - Withdraw
// - Check Balance
//
// This is a perfect example of Encapsulation.
```

Program 3

```
// =====
// ENCAPSULATION USING PRIVATE FIELDS, GETTERS & SETTERS
// =====

// Encapsulation:
// Encapsulation is the process of wrapping data (properties)
// and methods (functions) into a single unit called a class
// and restricting direct access to internal data.
```



```

// We achieve encapsulation in JavaScript using:
// 1. Private Fields (#)
// 2. Getters (get)
// 3. Setters (set)

class Employee {

    // Private Properties
    // Cannot be accessed directly outside the class
    #salary;
    #name;

    // Constructor
    // Automatically called when an object is created
    constructor(name, salary) {

        this.#name = name;
        this.#salary = salary;
    }

    // =====
    // GETTERS
    // =====

    // Getter for Name
    // Used to read/access private data
    get name() {
        return this.#name;
    }

    // Getter for Salary
    get salary() {
        return `₹${this.#salary}`;
    }

    // =====
    // SETTERS
    // =====

    // Setter for Salary
    // Used to update private data with validation
    set salary(newSalary) {

        // Salary cannot be negative
        if (newSalary < 0) {
            console.log("Salary cannot be negative");
            return;
        }

        // Salary should not be reduced
        if (newSalary < this.#salary) {

```

```

        console.log("Salary cannot be reduced");
        return;
    }

    this.#salary = newSalary;
}

// Setter for Name
set name(newName) {

    // Name should contain at least 2 characters
    if (newName.length < 2) {
        console.log("Name is too short");
        return;
    }

    this.#name = newName;
}
}

// =====
// OBJECT CREATION
// =====

const emp = new Employee("Rahul", 80000);

// =====
// PRIVATE FIELDS CANNOT BE ACCESSED DIRECTLY
// =====

// console.log(emp.#salary); // SyntaxError
// console.log(emp.#name);    // SyntaxError

// =====
// CALLING GETTERS
// =====

console.log(emp.name);    // Rahul
console.log(emp.salary);  // ₹80000

// =====
// CALLING SETTERS
// =====

emp.salary = 90000;

console.log(emp.salary);  // ₹90000

```

```
emp.salary = -1000;           // Salary cannot be negative
emp.salary = 50000;           // Salary cannot be reduced
emp.name = "A";
emp.name = "Rohit";
console.log(emp.name);

// =====
// IMPORTANT NOTES
// =====

// Getter:
// Used to read/get the value of private properties.

// Example:
// get salary() {
//     return this.#salary;
// }

// Usage:
// console.log(emp.salary);

// Setter:
// Used to update/set the value of private properties
// with validation.

// Example:
// set salary(value) {
//     this.#salary = value;
// }

// Usage:
// emp.salary = 90000;

// =====
// INTERVIEW QUESTIONS
// =====

// 1. What is Encapsulation?
//
// Encapsulation is the process of wrapping data and methods
// into a single unit called a class and restricting direct
// access to internal data.

// 2. How do you achieve Encapsulation in JavaScript?
```

```
//  
// Using:  
// ✓ Private Fields (#)  
// ✓ Getters (get)  
// ✓ Setters (set)  
  
// 3. What is a Getter?  
//  
// A getter is a special method used to read/access  
// private properties.  
  
// 4. What is a Setter?  
//  
// A setter is a special method used to update private  
// properties in a controlled manner.  
  
// 5. Difference Between Getter and Setter  
//  
// Getter:  
// - Reads data  
// - Uses get keyword  
// - Returns value  
//  
// Setter:  
// - Updates data  
// - Uses set keyword  
// - Accepts one parameter  
  
// Real-Life Example:  
//  
// ATM Machine  
//  
// You cannot directly access your account balance.  
// You can only check balance, deposit, or withdraw  
// using controlled operations.  
//  
// This is a perfect example of Encapsulation.
```

What is Inheritance in JavaScript?

Definition

Inheritance is an OOP (Object-Oriented Programming) concept where a **child class acquires all properties and methods of a parent class**.

It allows code reuse and helps avoid writing the same code multiple times.

Simple Definition

Inheritance is a mechanism where one class (Child Class) can use the properties and methods of another class (Parent Class).

Program 1:

```
//Inheritance
//Inheritance is a mechanism where a child class acquires all
properties and methods of a parent class

//Parent Class
class Parent {

    constructor() {
        //Property of Parent Class
        this.parentProp = "I am from Parent";
        console.log(this.parentProp);
    }

    //Parent Method
    parentMethod() {
        console.log("Parent method called");
    }
}

//Child Class
//extends keyword is used to inherit Parent class
class Child extends Parent {

    constructor() {
        //super() calls Parent class constructor
        //Mandatory before using 'this' inside child constructor
        super();

        //Child Property
        this.childProp = "I am from Child";
        console.log(this.childProp);
    }
}
```

```

    //Child Method
    childMethod() {
        console.log("Child method called");
    }
}

```

```

//Creating Object of Child Class
const child = new Child();

```

```

//Calling Child Method
child.childMethod();

```

```

//Calling Parent Method
//Inherited from Parent class
child.parentMethod();

```

```

//Real-Time Understanding
/*
Parent
  |
  Child
*/

```

```

//Important Points

```

```

//Inheritance is achieved using extends keyword

```

```

//Child class acquires properties and methods from parent class

```

```

//super() is used to call parent constructor

```

```

//super() is mandatory before using this inside child constructor

```

```

//Inheritance promotes code reusability

```

```

//Avoids code duplication

```

```

//Child class can:

```

```

//1. Use parent methods

```

```

//2. Add new methods

```

```

//3. Override parent methods

```

```

//Interview Questions

```

```

//Q. What is Inheritance?

```

```

//Inheritance is an OOP concept where a child class acquires
properties and methods of a parent class.

```

```

//Q. Why is super() required?

```

```

//super() calls the parent class constructor.

```

```

//Q. What happens if super() is not called?

```

```
//Error: Must call super constructor in derived class before  
accessing 'this'
```

```
//Q. Advantages of Inheritance  
//Code Reusability  
//Maintainability  
//Reduced Duplication  
//Method Overriding Support
```

Program 2:

```
//2. Types of Inheritance
```

```
//1. Single Inheritance  
//One child class inherits from one parent class
```

```
/*  
Parent Class  
    |  
Child Class  
*/
```

```
//Parent Class  
class Animal {  
  
    constructor(name) {  
        this.name = name;  
    }  
  
    //Parent Method  
    eat() {  
        console.log(`${this.name} is eating`);  
    }  
}  
  
//Child Class  
//Dog inherits Animal  
  
class Dog extends Animal {  
  
    //Child Specific Method  
    bark() {  
        console.log(`${this.name} says : Woof!`);  
    }  
}  
  
//Creating Object  
  
const dog = new Dog("Tommy");
```

```
//Inherited Method from Parent Class
```

```
dog.eat();
```

```
//Child Class Method
```

```
dog.bark();
```

```
//Real-Time Understanding
```

```
/*  
Animal  
  |  
  Dog  
*/
```

```
//Dog gets access to:
```

```
//name property
```

```
//eat() method
```

```
//bark() method
```

```
//Output
```

```
/*  
Tommy is eating  
Tommy says : Woof!  
*/
```

```
//Important Points
```

```
//Single Inheritance means one child inherits from one parent
```

```
//Achieved using extends keyword
```

```
//Child class can access parent properties and methods
```

```
//Code reusability is the main advantage
```

```
//super() is required if child class has its own constructor
```

```
//Child class can add its own methods
```

```
//Child class can override parent methods
```

```
//Interview Question
```


//Q. What is Single Inheritance?

//Single Inheritance is a type of inheritance where one child class inherits properties and methods from one parent class.

//Example:

//Animal → Dog

Program 3

//2. Multilevel Inheritance

//A child class inherits from a parent class

//and another child class inherits from that child class

```
/*
GrandParent
  |
  Parent
  |
  Child
  |
  GrandChild
*/
```

//Example

```
class LivingBeing {
    //GrandParent Method
    breath() {
        console.log("Breathing...");
    }
}
```

```
class Animal extends LivingBeing {
    //Inherited:
    //breath()

    //Parent Method
    eat() {
        console.log("Eating...");
    }
}
```

```
class Dog extends Animal {
    //Inherited:
    //breath()
```

```

    //eat()
    //Own Method

    bark() {
        console.log("Barking...");
    }
}

class Demo extends Dog {

    //Inherited:
    //breath()
    //eat()
    //bark()

    //Own Method
    test() {
        console.log("Test method");
    }
}

//Creating Dog Object

const dog = new Dog();
dog.breath(); //GrandParent Method
dog.eat(); //Parent Method
dog.bark(); //Own Method

console.log();

//Creating Demo Object

const demo = new Demo();
demo.breath(); //From LivingBeing
demo.eat(); //From Animal
demo.bark(); //From Dog
demo.test(); //Own Method

console.log();

//Creating LivingBeing Object

const living = new LivingBeing();
living.breath();
//living.test(); //Error

//Real-Time Understanding

/*
LivingBeing
|

```

```

    Animal
    |
    Dog
    |
    Demo
*/

//Methods Available

/*
LivingBeing
-----
breath()

Animal
-----
breath()
eat()

Dog
---
breath()
eat()
bark()

Demo
----
breath()
eat()
bark()
test()
*/

//Output

/*
Breathing...
Eating...
Barking...

Breathing...
Eating...
Barking...
Test method

Breathing...
*/

//Important Points

```

```
//Multilevel Inheritance forms a chain of inheritance

//Child class gets access to all properties and methods of its
parent classes

//Code reusability increases

//Avoids code duplication

//Methods can be accessed through inheritance chain

//A class can act as both:
//Parent Class
//Child Class

//Animal is:
//Child of LivingBeing
//Parent of Dog

//Dog is:
//Child of Animal
//Parent of Demo


//Interview Question

//Q. What is Multilevel Inheritance?

//Multilevel Inheritance is a type of inheritance where one class
inherits from another class, and a third class inherits from the
second class.

//Example:
//LivingBeing → Animal → Dog → Demo


//Q. Is Multilevel Inheritance supported in JavaScript?

//Yes

//JavaScript supports Multilevel Inheritance using the extends
keyword.
```

Program 4

```
//3. Hierarchical Inheritance

//One Parent Class has multiple Child Classes
```

```

/*
    Vehicle
    /   |   \
  Car  Bike Truck
*/

class Vehicle {
  constructor(brand,speed) {
    this.brand = brand;
    this.speed = speed;
  }

  //Common Method
  move() {
    console.log(`${this.brand} is moving at ${this.speed} km/h`);
  }
}

class Car extends Vehicle {
  //Car Specific Method
  honk() {
    console.log("Beep Beep!");
  }
}

class Bike extends Vehicle {
  //Bike Specific Method
  wheelie() {
    console.log("Doing a Wheelie!");
  }
}

class Truck extends Vehicle {
  //Truck Specific Method
  loadCargo(weight) {
    console.log(`Loading ${weight} tons`);
  }
}

const car = new Car("Honda",120);
car.honk();
car.move();

console.log();

const bike = new Bike("Yamaha",80);
bike.wheelie();
bike.move();

```

```

console.log();

const truck = new Truck("Tata",60);
truck.loadCargo(10);
truck.move();

//Important Points

//Hierarchical Inheritance = One Parent → Multiple Child Classes

//All child classes inherit common methods from parent

//Each child class can have its own methods

//Achieved using extends keyword

//Promotes code reusability

//Example:
//Vehicle → Car
//Vehicle → Bike
//Vehicle → Truck

```

Program 5

```

//Multiple Inheritance

//Multiple Inheritance means one child class inherits from
multiple parent classes

/*
  Parent1    Parent2
   \        /
    \      /
     \    /
      \  /
       Child
*/

//JavaScript does NOT support Multiple Inheritance directly using
classes

//Reason:
//If both parent classes have same method,
//JavaScript cannot decide which method child should inherit

//Example (Not Supported)

/*
class D1 {
  T1() {

```

```

        console.log("T1");
    }
}

class D2 {
    T1() {
        console.log("T1");
    }
}

```

//Not Allowed

```

class D3 extends D1,D2 {
}
*/

```

//Solution → Mixins

//Mixin
 //A technique used to add properties and methods from one object/
 class
 //to another class without using inheritance

//Mixin Objects

```

const canFly = {
    fly() {
        console.log("Flying...");
    }
};

const canSwim = {
    swim() {
        console.log("Swimming...");
    }
};

```

//Class

```

class Duck {
    constructor(name) {
        this.name = name;
    }
}

```

//Adding Mixin Methods to Duck Class

```
Object.assign(Duck.prototype, canFly, canSwim);
```

```
//Creating Object
```

```
const duck = new Duck("Donald");
```

```
duck.fly();
```

```
duck.swim();
```

```
//Another Example
```

```
const canEat = {
  eat() {
    console.log("Eating");
  }
};
```

```
const canWalk = {
  walk() {
    console.log("Walking");
  }
};
```

```
class Human {
}
```

```
//Adding Multiple Behaviors
```

```
Object.assign(Human.prototype, canEat, canWalk);
```

```
//Creating Object
```

```
const person = new Human();
```

```
person.eat();
```

```
person.walk();
```

```
//Real-Time Understanding
```

```
/*
Human
|
|----- canEat
```



```
|  
|----- canWalk
```

```
Duck  
|  
|----- canFly  
|  
|----- canSwim  
*/
```

```
//Output
```

```
/*  
Flying...  
Swimming...
```

```
Eating  
Walking  
*/
```

```
//Important Points
```

```
//JavaScript does NOT support Multiple Inheritance using classes
```

```
//A class can extend only ONE parent class
```

```
//Mixins are used to achieve Multiple Inheritance behavior
```

```
//Object.assign() copies methods from source objects
```

```
//Mixins promote code reuse
```

```
//No inheritance relationship is created
```

```
//Methods are simply copied to prototype
```

```
//Interview Questions
```

```
//Q. Does JavaScript support Multiple Inheritance?
```

```
//No
```

```
//JavaScript supports only Single Inheritance using classes.
```

```
//Q. How can Multiple Inheritance behavior be achieved?
```

```
//Using Mixins.
```

//Q. What is a Mixin?

//A Mixin is a technique used to add properties and methods from one object to another class without using inheritance.

//Q. Which method is commonly used for Mixins?

//Object.assign()

//Interview Answer

//JavaScript does not support Multiple Inheritance directly because a class can extend only one parent class.

//To achieve Multiple Inheritance-like behavior, JavaScript uses Mixins, where methods from multiple objects are copied into a class using Object.assign().

Polymorphism

//3. Polymorphism in JavaScript

//Polymorphism means "Many Forms"

//One method can behave differently depending on the object that calls it

/*

Real-Life Example

Person

At Home → Son

At Office → Employee

On Ground → Player

At School → Student

*/

//Types of Polymorphism

//1. Compile Time Polymorphism (Method Overloading)

//JavaScript does NOT support true Method Overloading like Java or C#

```
class Demo {
```

```
    //Workaround for Method Overloading
```

```
    add(a, b, c) {  
        if (c !== undefined) {  
            return a + b + c;  
        }  
        return a + b;  
    }  
}
```

```
const demo = new Demo();
```

```
console.log(demo.add(10, 20)); //30
```

```
console.log(demo.add(10, 20, 30)); //60
```

//2. Runtime Polymorphism (Method Overriding)

```
class Animal {
```

```
    makeSound() {  
        console.log("Animal makes sound");  
    }  
}
```

```
}  
}
```

```
class Dog extends Animal {  
  
    makeSound() {  
        console.log("Dog says Woof");  
    }  
}
```

```
class Test extends Animal {  
  
    makeSound() {  
        console.log("Good Morning");  
    }  
}
```

```
const c1 = new Animal();  
c1.makeSound();
```

```
const c2 = new Dog();  
c2.makeSound();
```

```
const c3 = new Test();  
c3.makeSound();
```

```
//Output
```

```
/*  
Animal makes sound  
Dog says Woof  
Good Morning  
*/
```

```
//Real-Time Playwright Example
```

```
class BasePage {  
  
    navigate() {  
        console.log("Navigate to Base Page");  
    }  
}
```

```
class LoginPage extends BasePage {  
  
    navigate() {  
        console.log("Navigate to Login Page");  
    }  
}
```

```

}

class DashboardPage extends BasePage {

  navigate() {
    super.navigate();
    console.log("Navigate to Dashboard Page");
  }
}

```

```

const d1 = new LoginPage();
d1.navigate();

```

```

console.log();

```

```

const d2 = new DashboardPage();
d2.navigate();

```

//Advantages of Polymorphism

//Code Reusability

//Flexibility

//Easy Maintenance

//Supports Method Overriding

//Reduces Conditional Logic

//Improves Extensibility

//Interview Questions

//Q. What is Polymorphism?

//Polymorphism means "Many Forms".

//The same method can behave differently depending on the object calling it.

//Q. Types of Polymorphism in JavaScript?

//1. Compile Time Polymorphism (Simulated Method Overloading)

//2. Runtime Polymorphism (Method Overriding)

//Q. Does JavaScript support Method Overloading?

//No

//JavaScript does not support true Method Overloading like Java or C#.

//Q. What is Method Overriding?

//When a child class provides its own implementation of a parent class method.

//One-Line Interview Answer

//Polymorphism is an OOP concept where the same method name can perform different actions depending

// on the object that invokes it.

Program 2

//Abstraction

//Abstraction is the process of hiding complex implementation details
//and showing only the essential features to the user

//Real-Life Examples

//ATM → User only sees Withdraw, Deposit, Balance Check

//Car → User only starts the car

//Mobile App → User clicks button without knowing backend logic

//Example 1 : Car

```
class Car {  
  
    //Private Method  
    //Hidden from user  
  
    #checkFuel() {  
        console.log("Checking Fuel");  
    }  
  
    //Public Method  
    //Visible to user  
  
    start() {  
        this.#checkFuel();  
        console.log("Car Started");  
    }  
}
```

```
const car = new Car();

car.start();

//Not Allowed

//car.#checkFuel();

//Example 2 : ATM

class ATM {

  //Private Method

  #validatePin() {
    console.log("PIN Verified");
  }

  //Private Method

  #checkBalance() {
    console.log("Balance Checking");
  }

  //Public Method

  withdraw(amount) {

    this.#validatePin();

    this.#checkBalance();

    console.log(`${amount} withdrawn successfully`);
  }
}

const atm = new ATM();

atm.withdraw(1000);

//Output

/*
Checking Fuel
Car Started
```

```
PIN Verified  
Balance Checking  
1000 withdrawn successfully  
*/
```

```
//Real-Time Understanding
```

```
/*  
User  
|  
|  
start()  
|  
|  
Car Internal Logic  
|  
checkFuel()  
startEngine()  
checkBattery()  
*/
```

```
//Important Points
```

```
//Abstraction hides internal implementation details
```

```
//User interacts only with public methods
```

```
//Private methods remain hidden
```

```
//Reduces complexity
```

```
//Improves security
```

```
//Improves maintainability
```

```
//Achieved using:
```

```
//Private Fields (#)
```

```
//Private Methods (#)
```

```
//Abstract Classes (TypeScript)
```

```
//Difference Between Abstraction and Encapsulation
```

```
/*  
Encapsulation
```

Hide Data

Focus:
Data Protection

Example:
private balance

Abstraction

Hide Implementation

Focus:
Show only necessary features

Example:
ATM withdraw()
Car start()
*/

//OOP Pillars

//1. Encapsulation
//Data Hiding

//2. Inheritance
//Code Reusability

//3. Polymorphism
//One Method, Multiple Behaviors

//4. Abstraction
//Hide Complexity

//Interview Questions

//Q. What is Abstraction?

//Abstraction is the process of hiding implementation details
//and exposing only essential functionality to users.

//Q. How is Abstraction achieved in JavaScript?

```
//Using:  
//Private Fields (#)  
//Private Methods (#)
```

//Q. Real-Time Example of Abstraction?

```
//ATM Machine  
//Car Start Button  
//Mobile Apps  
//Playwright Framework Methods
```

//Playwright Example

```
class LoginPage {  
  
    #enterUsername() {  
        console.log("Entering Username");  
    }  
  
    #enterPassword() {  
        console.log("Entering Password");  
    }  
  
    #clickLogin() {  
        console.log("Clicking Login Button");  
    }  
  
    login() {  
        this.#enterUsername();  
        this.#enterPassword();  
        this.#clickLogin();  
        console.log("Login Successful");  
    }  
}  
  
const loginPage = new LoginPage();  
  
loginPage.login();
```

//One-Line Interview Answer

```
//Abstraction is an OOP concept that hides implementation details and exposes only the essential  
// functionality to the user.
```

Program 3

```
//Static Keyword

//Static members belong to the class itself
//They are not associated with individual objects

//Access static members using:
//ClassName.memberName()

class MathUtils {

    //Static Property
    //Use → Store common constant value

    static PI = 3.14159;

    //Static Method
    //Use → Add two numbers

    static add(a, b) {
        return a + b;
    }

    //Static Method
    //Use → Calculate circle area

    static circleArea(radius) {
        return MathUtils.PI * radius ** 2;
    }

    //Static Method
    //Use → Generate unique ID

    static generateID() {
        return `ID_${Date.now()}`;
    }

    //Non-Static Method
    //Use → Object specific method

    demo() {
        console.log("Demo Method");
    }
}

//Calling Static Members
```

```
console.log(MathUtils.PI);  
  
console.log(MathUtils.add(10, 20));  
  
console.log(MathUtils.circleArea(7));  
  
console.log(MathUtils.generateID());
```

//Creating Object

```
const c1 = new MathUtils();  
  
c1.demo();
```

//Not Allowed

```
//console.log(c1.add(10,20));
```

//Error:

//c1.add is not a function

//Static methods belong to class, not object

//Calling Static Members Correctly

```
console.log(MathUtils.add(50, 20));  
  
console.log(MathUtils.circleArea(5));
```

//Real-Time Examples

```
class Config {  
  
    static BASE_URL = "https://opensource-demo.orangehrmlive.com";  
  
    static BROWSER = "Chrome";  
  
    static getEnvironment() {  
        return "QA";  
    }  
}  
  
console.log(Config.BASE_URL);
```

```
console.log(Config.BROWSER);  
  
console.log(Config.getEnvironment());
```

//Difference Between Static and Non-Static

```
/*  
Static Method  
-----  
Belongs to Class  
  
No Object Required  
  
Called Using:  
ClassName.method()  
  
Example:  
MathUtils.add(10,20)
```

```
Non-Static Method  
-----  
Belongs to Object  
  
Object Required  
  
Called Using:  
objectName.method()  
  
Example:  
c1.demo()  
*/
```

//Important Points

//Static members belong to the class itself

//No object creation required

//Access using ClassName.memberName()

//Static methods can access static properties

//Static methods cannot directly access non-static properties

//Static properties are shared across all objects

//Useful for:

//Utility Methods

//Helper Functions

//Constants

//Configuration Data

//Interview Questions

//Q. What is a Static Method?

//A static method belongs to the class itself rather than an object.

//It can be called directly using the class name.

//Q. Do we need to create an object to call a static method?

//No

//Call directly using:

//ClassName.methodName()

//Q. Can an object access a static method?

//No

//Static methods belong to the class, not the object.

//Q. Real-Time Examples of Static Methods?

//Math.max()

//Math.min()

//Math.random()

//Number.isInteger()

//One-Line Interview Answer

//Static members belong to the class itself and can be accessed without creating an object

// using ClassName.memberName().

Program 1

```
// =====
// THIS KEYWORD IN JAVASCRIPT
// =====

// this refers to the current object
// that is executing the code

class Person {

    // Constructor
    // Used to initialize object properties

    constructor(name) {
        this.name = name; // this refers to new object
    }

    // Method

    greet() {
        console.log(`Hi I'm ${this.name}`);
    }
}

// Object Creation

const p = new Person("Virat");

// Method Calling

p.greet();

// =====
// SUPER KEYWORD IN JAVASCRIPT
// =====

// super is used to call:
// 1. Parent Constructor
// 2. Parent Methods

class Animal {

    constructor(name) {
        this.name = name;
    }

    speak() {
        console.log(`${this.name} makes a sound`);
    }
}
```

```

// Child Class

class Dog extends Animal {
    constructor(name, breed) {
        // Call Parent Constructor
        super(name);
        this.breed = breed;
    }

    // Method Overriding
    speak() {
        // Call Parent Method
        super.speak();
        console.log(`${this.name} barks`);
    }
}

// Object Creation
const dog = new Dog("Tom", "Labrador");

// Method Calling
dog.speak();

// =====
// ACCESS MODIFIERS
// =====

// Public (Default)
// Accessible from anywhere

class Employee {
    constructor(name) {
        this.name = name;
    }
}

const emp = new Employee("Rahul");

console.log(emp.name);

// =====

```



```

// PRIVATE MEMBERS
// =====

// Private members start with '#'
// Accessible only inside the class

class BankAccount {
    #balance = 50000;

    getBalance() {
        return this.#balance;
    }
}

const account = new BankAccount();

console.log(account.getBalance());

// console.log(account.#balance);
// Error

// =====
// IMPORTANT POINTS
// =====

// this
// ✓ Refers to current object
// ✓ Used to access object properties

// super
// ✓ Calls parent constructor
// ✓ Calls parent methods

// extends
// ✓ Creates inheritance relationship

// Public Members
// ✓ Accessible from anywhere

// Private Members (#)
// ✓ Accessible only inside class

// Method Overriding
// ✓ Child class provides its own implementation

// Inheritance
// ✓ Reuse parent class code

// =====
// INTERVIEW QUESTIONS
// =====

```

```
// Q1. What is this keyword?
// this refers to the current object executing the code.

// Q2. What is super keyword?
// super is used to call parent constructor and methods.

// Q3. Why do we use super()?
// To initialize parent class properties.

// Q4. What is inheritance?
// Acquiring properties and methods from parent class.

// Q5. What is method overriding?
// Child class provides its own implementation of parent method.
```

Program 2

```
// =====
// HOSPITAL MANAGEMENT SYSTEM
// =====

// Parent Class
// Common properties shared by Doctor and Patient

class Person {
    constructor(name, age, phone) {
        this.name = name;
        this.age = age;
        this.phone = phone;
    }
}

// =====
// DOCTOR CLASS
// =====

// Doctor inherits Person class

class Doctor extends Person {
    constructor(name, age, phone, specialization, fee) {
        // Call Parent Constructor
        super(name, age, phone);

        this.specialization = specialization;
        this.fee = fee;
    }
}
```

```

        // Store doctor appointments

        this.appoinments = [];
    }

    // Book Appointment
    addAppoinment(patient, date) {

        this.appoinments.push({
            patient,
            date
        });

        console.log(
            `Appointment booked : ${patient.name} with Dr.$
{this.name} on ${date}`
        );

        // patient = Patient Object
        // patient.name = Accessing patient name
    }
}

// =====
// PATIENT CLASS
// =====

// Patient inherits Person class
class Patient extends Person {

    constructor(name, age, phone, disease) {

        // Call Parent Constructor
        super(name, age, phone);

        this.disease = disease;

        // Store prescriptions

        this.prescriptions = [];
    }

    // Add Medicine Prescription
    addPrescriptions(medicine, dosage) {

        this.prescriptions.push({
            medicine,
            dosage,

```

```
        date: new Date()
    });
}

// =====
// OBJECT CREATION
// =====

// Creating Doctor Object

const doctor1 = new Doctor(
    "Sharma",
    45,
    "84747474744",
    "Cardiologist",
    1000
);

// Creating Patient Object

const patient1 = new Patient(
    "Rohan",
    30,
    "8765434567",
    "Heart Problem"
);

// =====
// METHOD CALLING
// =====

// Book Appointment

doctor1.addAppoinment(
    patient1,
    "11-06-2026"
);

// Add Prescription

patient1.addPrescriptions(
    "Paracetamol",
    "500mg"
);

// Print Complete Doctor Object

console.log(doctor1);

// Access Patient Name
```

```
console.log(patient1.name);

// =====
// OOPS CONCEPTS USED
// =====

// Class
// → Person
// → Doctor
// → Patient

// Object
// → doctor1
// → patient1

// Constructor
// → Initializes object properties

// Inheritance
// → Doctor extends Person
// → Patient extends Person

// super()
// → Calls Parent Constructor

// this
// → Refers to current object

// Encapsulation
// → Data and methods grouped together

// Array Inside Object
// → appointments
// → prescriptions

// Object Inside Array
// → { patient, date }
// → { medicine, dosage, date }

// Object Composition
// → Doctor stores Patient object
// → Appointment contains Patient details

// =====
// IMPORTANT POINTS
// =====

// Inheritance
// ✓ Reuse parent class properties
// ✓ Avoid duplicate code

// super()
```

```

// ✓ Must be called before using this
// ✓ Calls parent constructor

// this
// ✓ Refers to current object

// Object Composition
// ✓ One object can contain another object

// Real-world Example
// ✓ Doctor has Patients
// ✓ Patient has Prescriptions
// ✓ Hospital manages both

// =====
// INTERVIEW QUESTIONS
// =====

// Q1. What is Inheritance?
// Acquiring properties and methods
// from parent class.

// Q2. What is super()?
// Calls parent class constructor.

// Q3. Why use Inheritance?
// Code reusability and maintainability.

// Q4. What is Object Composition?
// One object contains another object.

// Q5. Which OOPS concepts are used here?
// Class
// Object
// Constructor
// Inheritance
// super()
// Encapsulation
// this Keyword

// =====
// BANKING MANAGEMENT SYSTEM
// =====

// Private Property (#balance)
// Accessible only inside the class

class BankAccount {
    #balance = 0;

```

```

// Constructor
// Used to initialize account details

constructor(accountHolderName, balance) {
    this.accountHolderName = accountHolderName;
    this.#balance = balance;
}

// Deposit Method
// Adds money to account

deposit(amount) {
    this.#balance += amount;

    console.log(`Deposited ₹${amount}`);
}

// Withdraw Method
// Deducts money from account

withhdraw(amount) {
    if (amount > this.#balance) {
        console.log("Insufficient Balance");
        return;
    }

    this.#balance -= amount;

    console.log(`Withdrawn ₹${amount}`);
}

// Getter Method
// Used to access private balance

getBalance() {
    return this.#balance;
}
}

// =====
// SAVING ACCOUNT
// =====

// SavingAccount inherits BankAccount

class SavingAccount extends BankAccount {
    // Saving Account Specific Method

```

```

    calculateInterest() {
        console.log("Calculating Saving Interest");
    }

    // Method Overriding

    accountType() {
        console.log("Saving Account");
    }
}

// =====
// CURRENT ACCOUNT
// =====

// CurrentAccount inherits BankAccount
class CurrentAccount extends BankAccount {

    // Method Overriding

    accountType() {
        console.log("Current Account");
    }
}

// =====
// OBJECT CREATION
// =====

// Creating Saving Account Object

const s1 = new SavingAccount(
    "Rohit",
    50000
);

// Method Calling

s1.deposit(5000);

s1.withdraw(1000);

s1.calculateInterest();

s1.accountType();

console.log(
    "Balance : ",
    s1.getBalance()
);

```



```
console.log();

// Creating Current Account Object

const c1 = new CurrentAccount(
    "Virat",
    100000
);

c1.accountType();

// =====
// OOPS CONCEPTS USED
// =====

// Class
// → BankAccount
// → SavingAccount
// → CurrentAccount

// Object
// → s1
// → c1

// Constructor
// → Initializes account details

// Inheritance
// → SavingAccount extends BankAccount
// → CurrentAccount extends BankAccount

// Encapsulation
// → #balance is hidden from outside world

// Private Property
// → #balance

// Getter Method
// → getBalance()

// Method Overriding
// → accountType()

// this Keyword
// → Refers to current object

// Polymorphism
// → Same method accountType()
// behaves differently in child classes
```

```
// =====  
// IMPORTANT POINTS  
// =====  
  
// #balance  
// ✓ Private Property  
// ✓ Cannot be accessed outside class  
  
// Encapsulation  
// ✓ Hides sensitive data  
// ✓ Provides controlled access  
  
// Inheritance  
// ✓ Reuse parent class code  
// ✓ Reduces duplication  
  
// Method Overriding  
// ✓ Child class provides its own implementation  
  
// Polymorphism  
// ✓ One method, multiple behaviors  
  
// Getter Method  
// ✓ Used to access private data safely  
  
// =====  
// INTERVIEW QUESTIONS  
// =====  
  
// Q1. What is Encapsulation?  
  
// Wrapping data and methods together  
// and restricting direct access to data.  
  
// Q2. What is a Private Property?  
  
// Property that can only be accessed  
// inside the same class.  
  
// Q3. How do you create a private property?  
  
// Using # symbol  
  
// Example:  
// #balance = 0  
  
// Q4. What is Inheritance?
```

```
// Child class acquires properties
// and methods from parent class.

// Q5. What is Method Overriding?

// Child class provides its own version
// of parent method.

// Q6. What is Polymorphism?

// Same method behaves differently
// in different classes.

// =====
// REAL-WORLD MAPPING
// =====

// BankAccount
// → Common Account Features

// SavingAccount
// → Interest Calculation

// CurrentAccount
// → Business Account

// #balance
// → Secure Banking Data

// getBalance()
// → Authorized Access to Balance

// =====
// FUNCTIONS IN JAVASCRIPT
// =====

// What is a Function?
// A function is a reusable block of code
// that performs a specific task.

// Why Functions are Needed?
// ✓ Avoid code duplication
// ✓ Improve code reusability
// ✓ Make code easy to maintain
// ✓ Improve readability

// Without Function
```

```
// Same code needs to be written multiple times

// console.log("Launching Browser");
// console.log("Opening URL");
// console.log("Entering Username");
// console.log("Entering Password");
// console.log("Clicking Login Button");

// console.log("Launching Browser");
// console.log("Opening URL");
// console.log("Entering Username");
// console.log("Entering Password");
// console.log("Clicking Login Button");

// console.log("Launching Browser");
// console.log("Opening URL");
// console.log("Entering Username");
// console.log("Entering Password");
// console.log("Clicking Login Button");

// =====
// FUNCTION WITHOUT PARAMETERS
// =====

// Syntax

// function functionName() {
//     code
// }

function login() {
    console.log("Launching Browser");
    console.log("Opening URL");
    console.log("Entering Username");
    console.log("Entering Password");
    console.log("Clicking Login Button");
}

// Function Calling

login();
login();
login();

// Write Once
// Use Multiple Times
```

```
// =====
// FUNCTION WITH PARAMETERS
// =====
```

```

// Parameters
// Values received by function

function loginUser(username, password) {

    console.log(username);
    console.log(password);
}

// Arguments
// Values passed while calling function

loginUser("Admin", "admin123");
loginUser("StgAdmin", "admin1234");
loginUser("ProdAdmin", "admin12345");


// =====
// FUNCTION WITH RETURN VALUE
// =====

// return sends output back to caller

function add(a, b) {

    return a + b;
}

let result = add(10, 20);
console.log(result);


// Example

function getPageTitle() {
    return "OrangeHRM";
}

let title = getPageTitle();

console.log("OrangeHRM" == title);


// =====
// TYPES OF FUNCTIONS
// =====

// 1. Function Declaration (Normal Function)

function launchBrowser() {
    console.log("Browser Launched");
}

```

```

}

launchBrowser();

// =====
// 2. FUNCTION EXPRESSION
// =====

// Function stored inside a variable

const loginPage = function () {
    console.log("Login");
};

loginPage();

// =====
// 3. ARROW FUNCTION
// =====

// Modern ES6 Syntax

const loginTest = () => {
    console.log("Login");
};

loginTest();

// =====
// ARROW FUNCTION WITH PARAMETERS
// =====

const multiply = (a, b) => {
    return a * b;
};

console.log(multiply(10, 5));

// =====
// IMPORTANT TERMS
// =====

// Function Definition
// Creating a function

function demo() {
}

// Function Call / Invocation
// Executing a function

```

```
demo();

// Parameter
// Variable in function definition

function test(name) {
}

// Argument
// Actual value passed

test("Rahul");

// Return
// Sends value back from function


// =====
// REAL-TIME PLAYWRIGHT EXAMPLE
// =====

function loginToApplication(username, password) {
    console.log("Enter Username:", username);
    console.log("Enter Password:", password);
    console.log("Click Login");
}

loginToApplication(
    "admin",
    "admin123"
);


// =====
// INTERVIEW QUESTIONS
// =====

// Q1. What is a function?

// A reusable block of code that
// performs a specific task.

// Q2. Why are functions used?

// To avoid code duplication and
// improve reusability.

// Q3. What is the difference between
// parameter and argument?
```

```
// Parameter
// Variable declared in function

// Argument
// Actual value passed to function

// Q4. What is return keyword?
// Used to send value back from function.

// Q5. Types of functions in JavaScript?

// 1. Function Declaration
// 2. Function Expression
// 3. Arrow Function

// =====
// FUNCTION VS METHOD
// =====

// Function
// Independent block of code

function display() {
}

// Method
// Function inside object/class

class Test {
    display() {
    }
}

// =====
// SUMMARY
// =====

// Function
// → Reusable block of code

// Parameter
// → Input variable

// Argument
// → Actual value passed

// Return
// → Output from function

// Function Declaration
// → Traditional Function
```



```
// Function Expression  
// → Function stored in variable  
  
// Arrow Function  
// → Modern ES6 Function
```